
Fundamentals of Embedded Systems

(Embedded Devices, Microprocessors and
Microcontrollers, DSP, ...)

Microprocessor Market Share

General-purpose computers



98%



Embedded Systems

2%

Introduction to Embedded Systems

Introduction to Embedded Systems

Several Definitions:

Wayne Wolf:

Any **device** that **includes** a **programmable computer** but is not itself intended to be a general purpose computer.

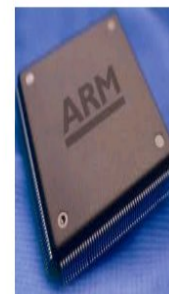
Jonathan W. Valvano:

An embedded computer system **includes** a **microcomputer** with **mechanic chemical and electrical devices** attached to it, programmed for a **specific dedicated purpose**, and packaged as a complete system.

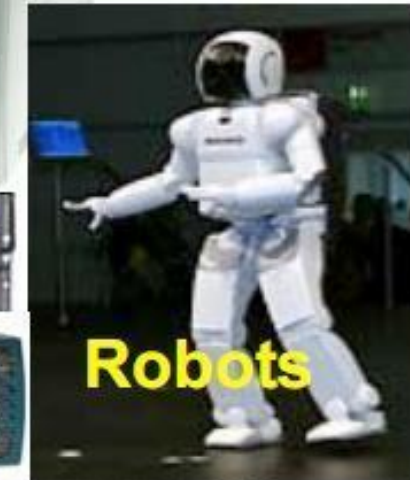
Raj Kamal:

An embedded system is one that has **computer-hardware with software embedded in it** as one of its most important component.

In a nutshell: Embedded Systems => Computers inside a product (electronic devices).



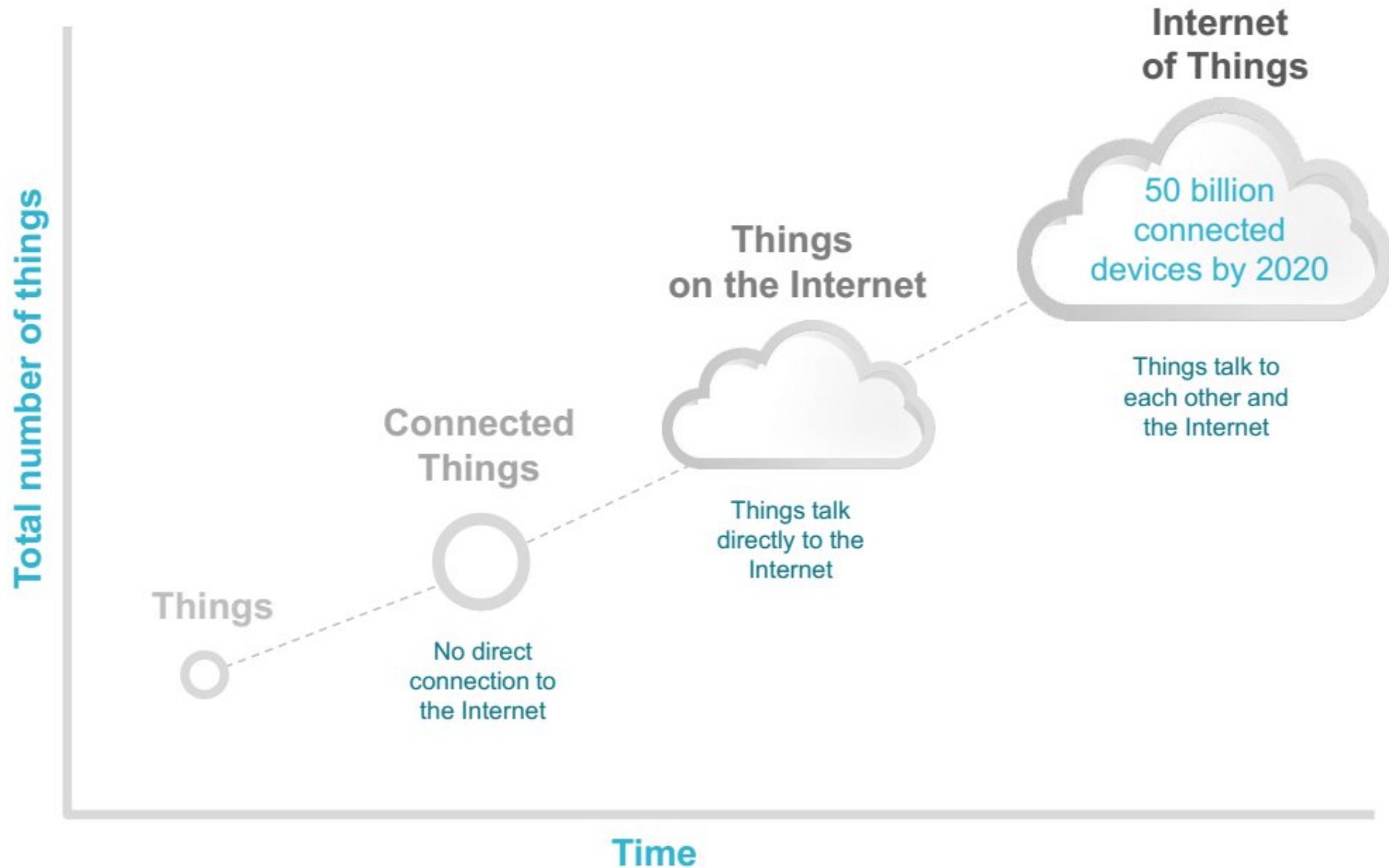
Computing Devices



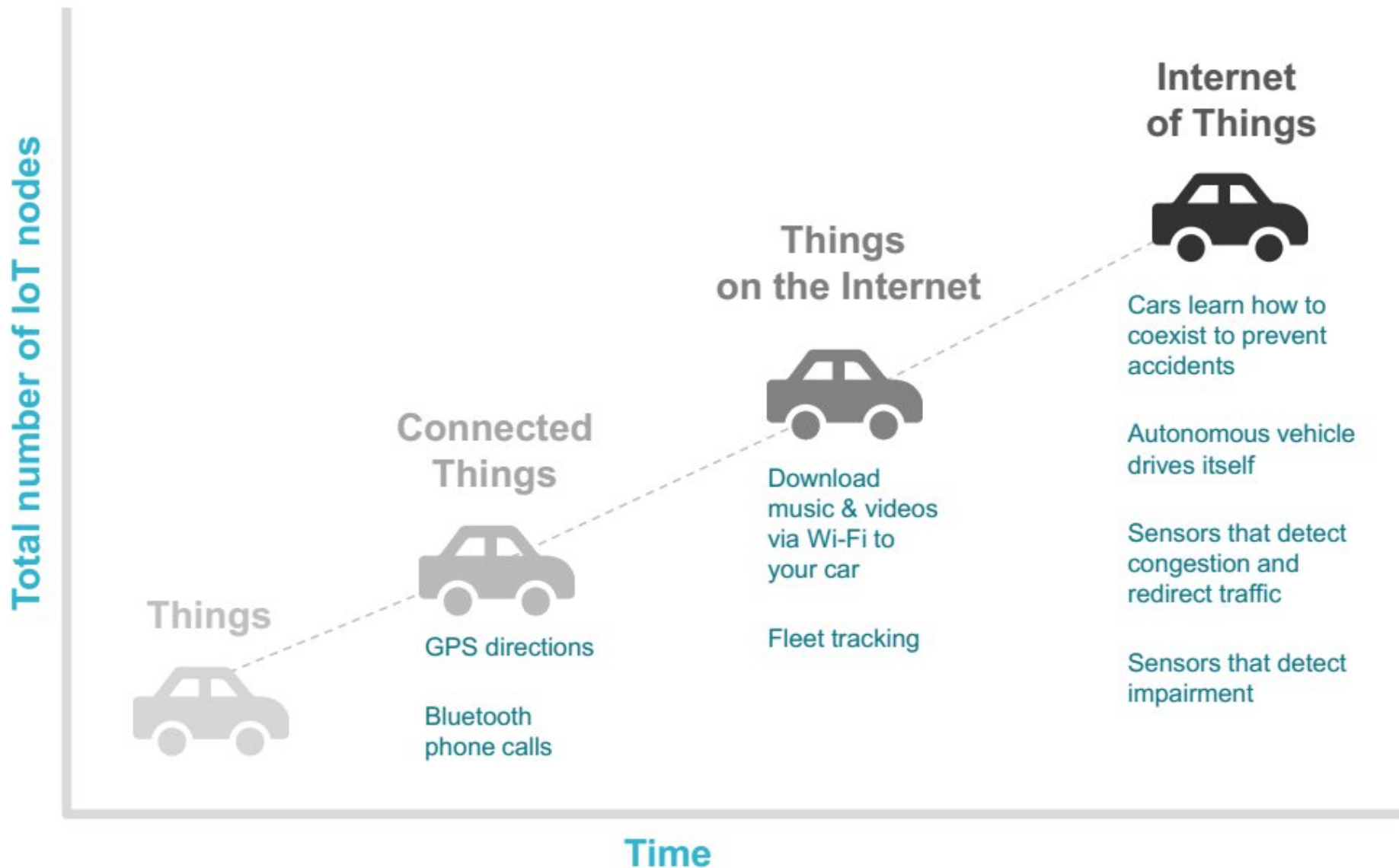
1/22/2008

CS152-S

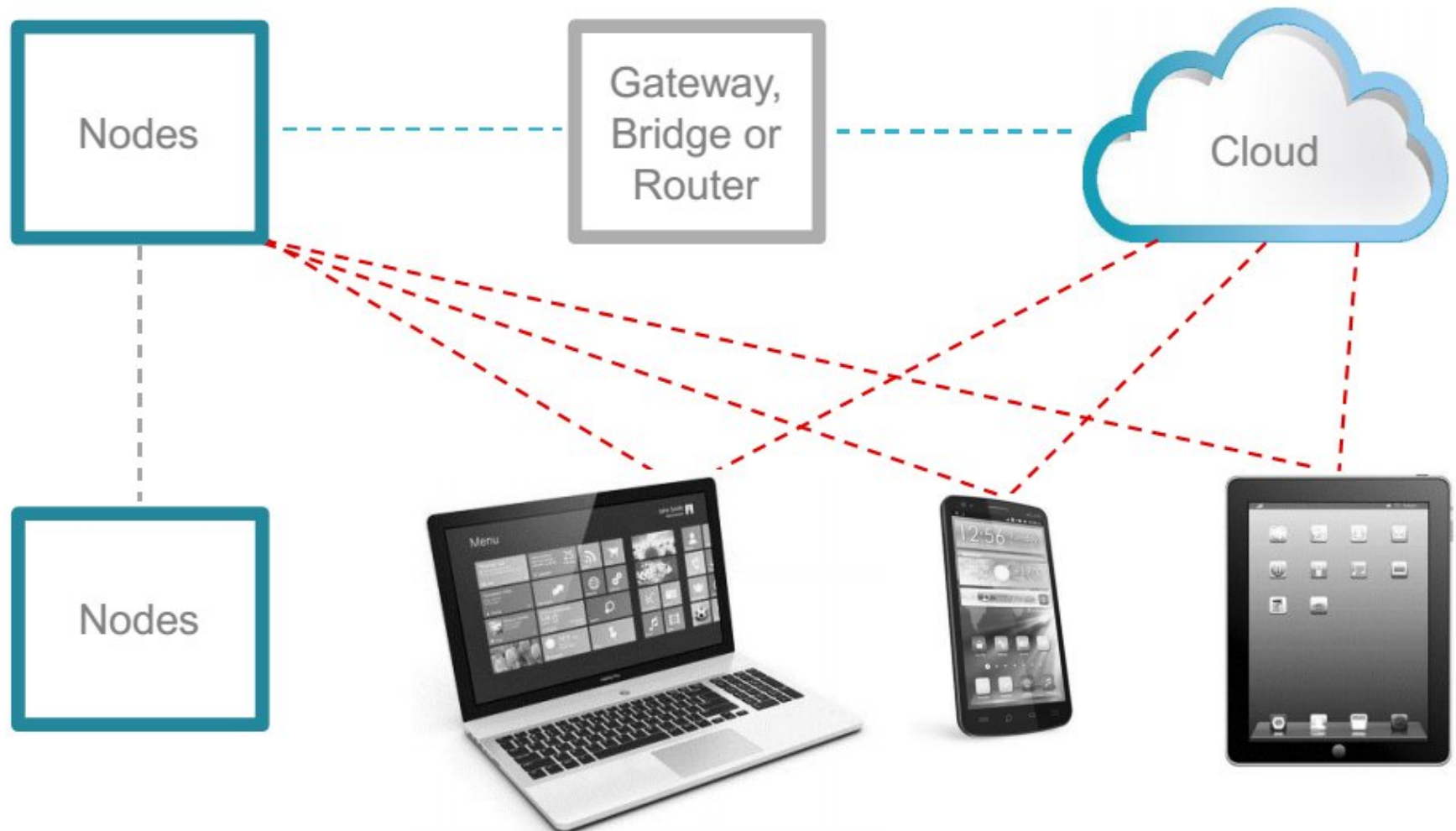
The Evolution of the IoT



The Evolution of the IoT



How the IoT works



Industrial in the IoT

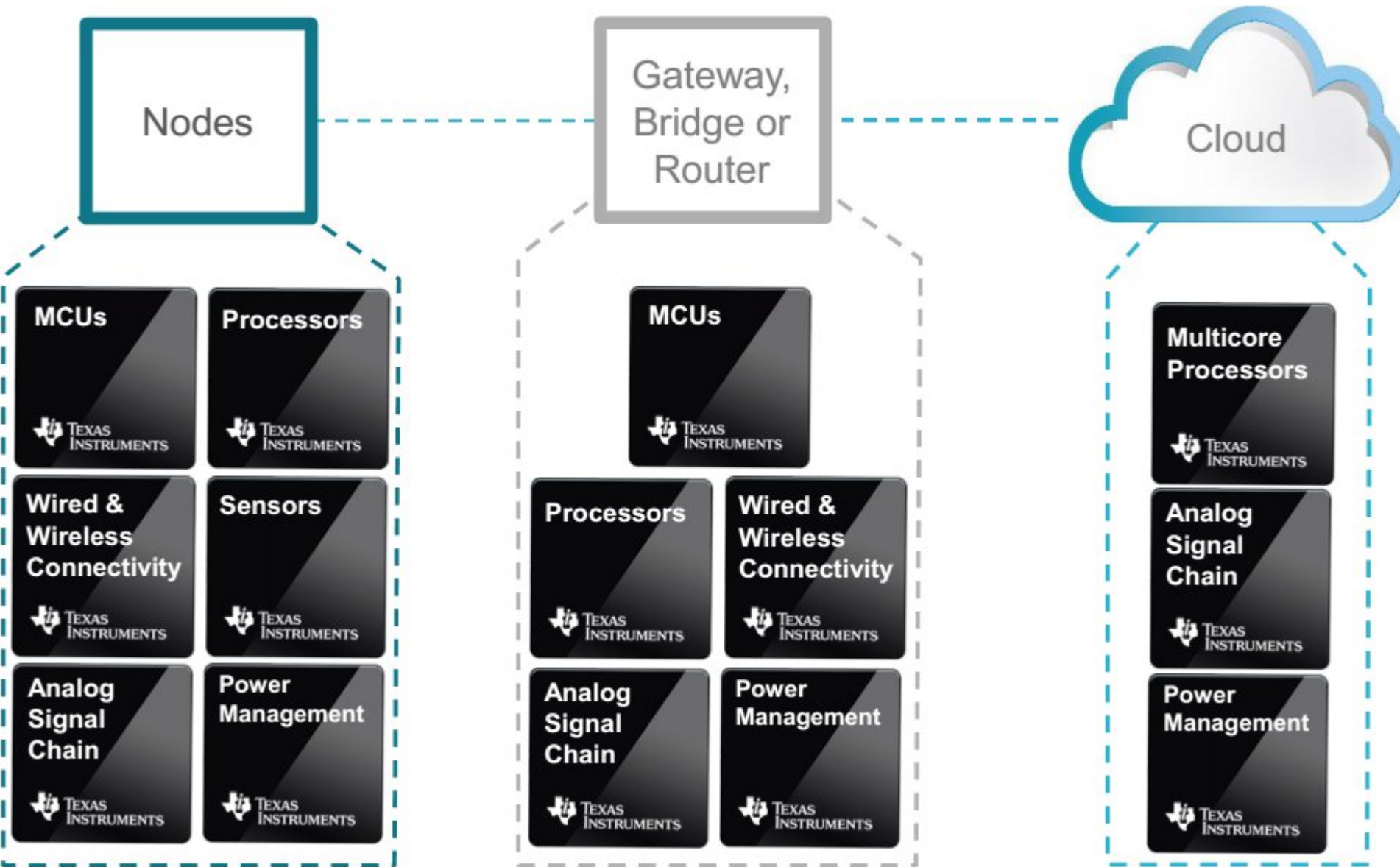


Wirelessly connected LED
lighting & energy
management



Flow & pressure
sensors

IoT building blocks



Homes in the IoT



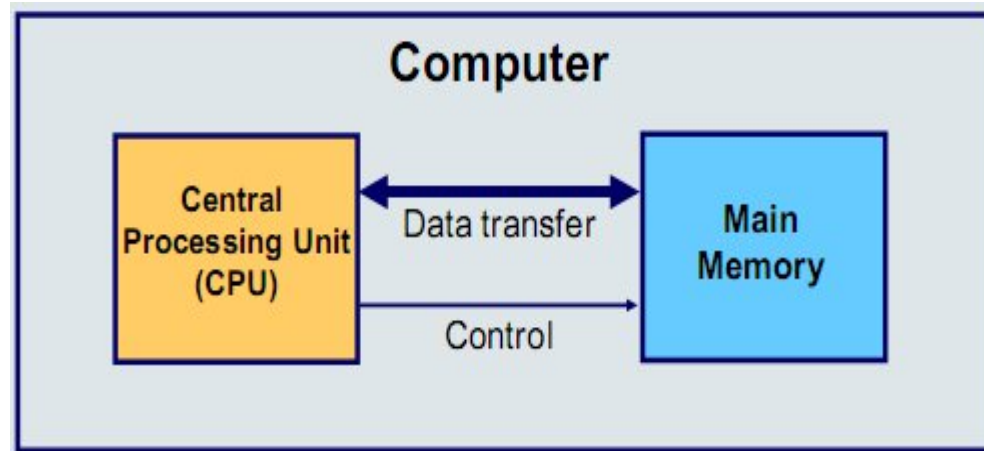
Security & safety
system, sensors



Smart home energy
gateway, thermostats,
sensors

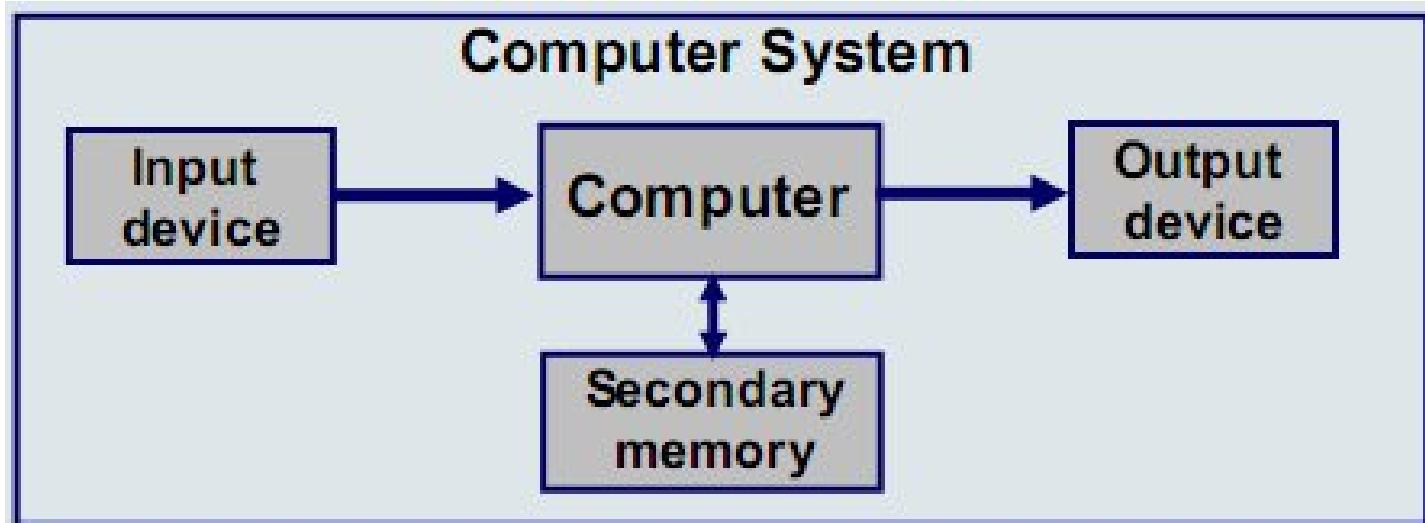
What A Computer?

- A computer is a data processing machine which is operated automatically under the control of a list of instructions (called a program) stored in its main memory.
- How about embedded devices?



A Computer System

- A computer system consists of a computer and its peripherals.
- Computer peripherals include input devices, output devices, and secondary memories.

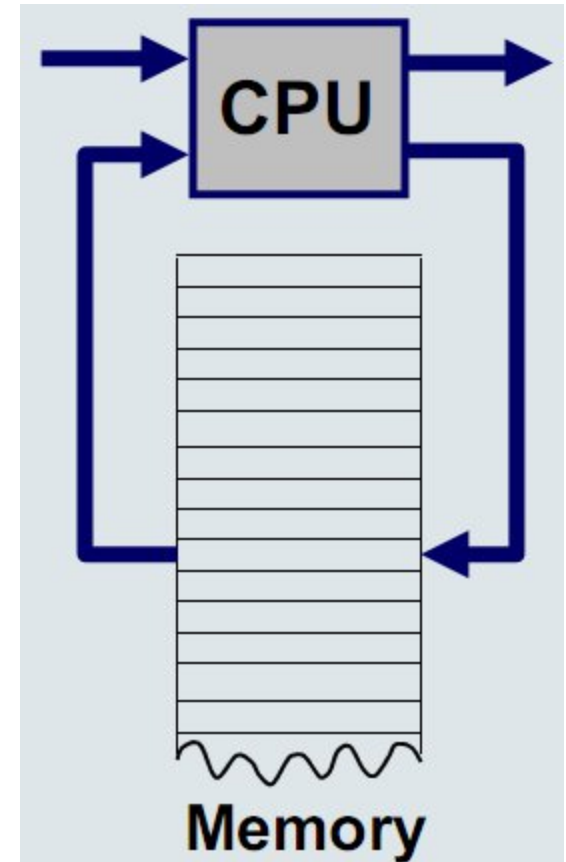


Computer Functionalities

- A computer executes repeatedly a series of instructions (called programs) stored in its main memory:
 - It performs data processing operations specified by the programs.
 - It runs the programs automatically, with no need for human intervention.
 - It can perform the operations in extremely high speed.
 - It can store and manipulate a large amount of data.
 - It can communicate with each other and with users in an efficient way.
 - It represents program and data in the same way, which leads to flexibility.

Basic Principles of Computers

- Virtually all modern computer designs are based on the Von Neumann architecture principles:
 - Data and instructions are stored in a single read/write memory.
 - The contents of this memory are addressable by location, without regard to what are stored there.
 - Instructions are executed sequentially (from one instruction to the next) unless the order is explicitly modified.



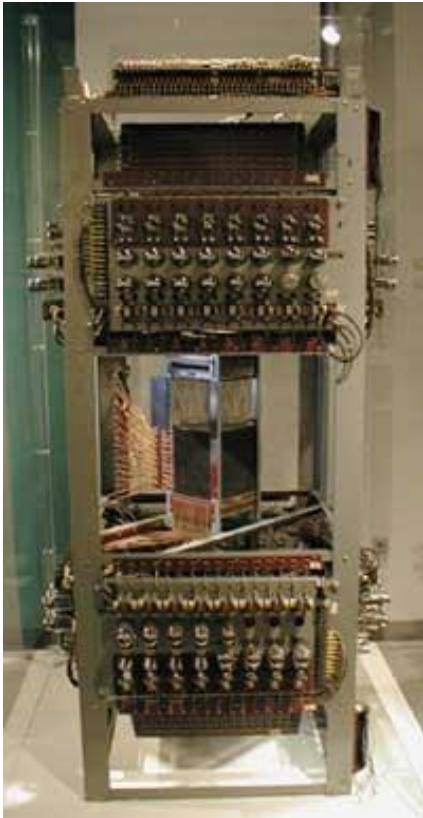
Classes of Computers

- Desktop computers/workstations
 - General purpose, variety of software
 - Subject to cost/performance tradeoff
- Server computers
 - Network based Network based
 - High capacity, performance, reliability
 - Range from small servers to building sized
- Embedded computers/Embedded devices
 - Hidden as components of systems
 - Stringent power/performance/cost constraints

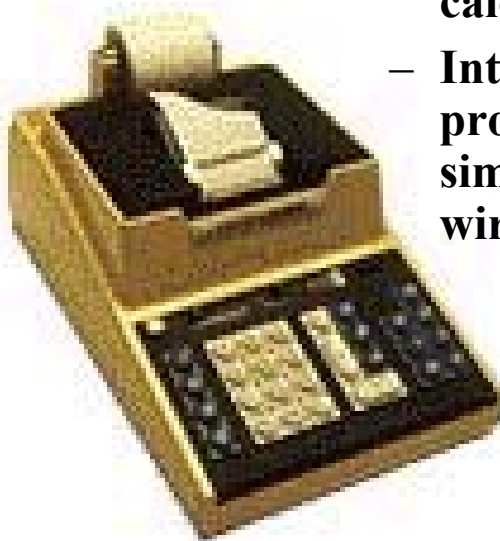
What is an Embedded Computer/Devices?

- ❑ **A computer not used to run general-purpose programs, but instead used as a component of a larger system.** Usually, user does not change the computer program (except for manufacturer upgrades).
 - ❑ Example applications:
 - Toasters
 - Cellphone
 - Digital camera (some have several processors)
 - Games machines
 - Set-top boxes (DVD players, personal video recorders, ...)
 - Televisions
 - Dishwashers
 - Car (some have dozens of processors)
 - Internet router (some have hundreds to thousands of processors)
 - Cellphone basestation
 - many more
-

Early Embedded Computing Examples



- **MIT Whirlwind, 1946-51**
 - Initially developed for real-time flight simulator
 - IBM later manufactured versions for SAGE air defence network, last used in 1983
- **Intel 4004, 1971**
 - developed for Busicom 141-PF printing calculator
 - Intel engineers decided that building a programmable computer would be simpler and more flexible than hard-wired digital logic



What is different about embedded computers?

- Embedded processors usually optimized to perform one fixed task with software from system manufacturer
 - General-purpose processors designed to run flexible, extensible software systems with code from third-party suppliers
 - applications not known at design time
 - Note, many products contain both embedded and general-purpose processors
 - e.g., smartphone has embedded processors for radio baseband signal processing, and general-purpose processors to run third-party software applications
-

Lesser emphasis on software portability in embedded applications

- Embedded systems
 - can usually recompile/rewrite source code for different ISA, and/or use assembler code for new application-specific instructions
 - processor pipeline microarchitecture and memory capacity and hierarchy known to programmer/compiler
 - mix of tasks known to writer of each task, usually static: uses custom run-time system
 - each task usually “trusts” others, can run in same address space
- General-purpose systems
 - must have standard binary interface for third-party software
 - compiler doesn’t know about this particular microarchitecture or memory capacity or hierarchy (compiled for general model)
 - unknown mix of tasks, tasks dynamically added and deleted from mix: uses general-purpose operating system
 - tasks written by various third-parties, mutually distrustful, need separate address spaces or protection domains

Embedded application requirements & constraints

❑ Real-time performance

- *hard real-time*: if deadline missed, system has failed (car brakes!)
- *soft real-time*: missing deadline degrades performance (skipping frames on DVD playback)

❑ Real-world I/O with multiple concurrent events

- sensor and actuators require continuous I/O (can't batch process)
- non-deterministic concurrent interactions with outside world

❑ Cost

- includes cost of supporting structures, particularly memory
- static code size very important (cost of ROM/RAM)
- often ship millions of copies (worth engineer time to optimize cost down)

❑ Power

- expensive package and cooling affects cost, system size, weight, noise, temperature
-

Processors for real-time software

- Simpler pipelines and memory hierarchies make it easier (possible?) to determine the worst-case execution time (WCET) of a piece of code
 - Would like to guarantee task completed by deadline
- Out-of-order execution, caches, prefetching, branch prediction, make it difficult to determine worst-case run time
 - Have to pad WCET estimates for unlikely but possible cases, resulting in over-provisioning of processor (wastes resources)

x86 Evolution (1)

- 8080
 - first general purpose microprocessor
 - 8 bit data path
 - Used in first personal computer – Altair
- 8086 – 5MHz – 29,000 transistors
 - much more powerful
 - 16 bit
 - instruction cache, prefetch few instructions
 - 8088 (8 bit external bus) used in first IBM PC
- 80286
 - 16 Mbyte memory addressable
 - up from 1Mb
- 80386
 - 32 bit
 - Support for multitasking
- 80486
 - sophisticated powerful cache and instruction pipelining
 - built in maths co-processor

x86 Evolution (2)

- Pentium
 - Superscalar
 - Multiple instructions executed in parallel
 - Pentium Pro
 - Increased superscalar organization
 - Aggressive register renaming
 - branch prediction
 - data flow analysis
 - speculative execution
 - Pentium II
 - MMX technology
 - graphics, video & audio processing
 - Pentium III
 - Additional floating point instructions for 3D graphics
-

x86 Evolution (3)

- Pentium 4
 - Note Arabic rather than Roman numerals
 - Further floating point and multimedia enhancements
 - Core
 - First x86 with dual core
 - Core 2
 - 64 bit architecture
 - Core 2 Quad – 3GHz – 820 million transistors
 - Four processors on chip
 - x86 architecture dominant outside embedded systems
 - Organization and technology changed dramatically
 - Instruction set architecture evolved with backwards compatibility
 - ~1 instruction per month added
 - 500 instructions available
 - See Intel web pages for detailed information on processors
-

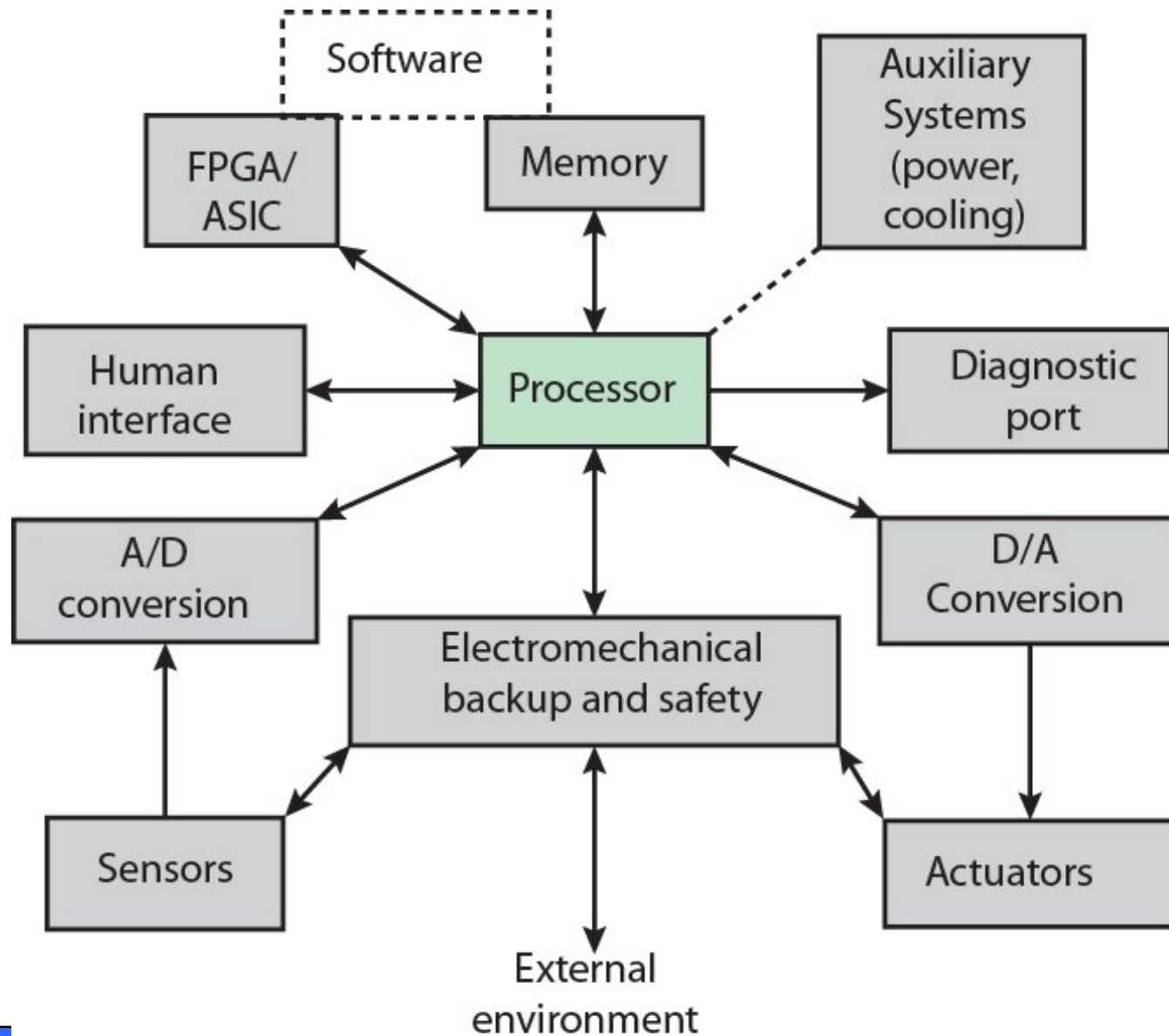
Embedded Systems Requirements

- Different sizes
 - Different constraints, optimization, reuse
 - Different requirements
 - Safety, reliability, real-time, flexibility, legislation
 - Lifespan
 - Environmental conditions
 - Static v dynamic loads
 - Slow to fast speeds
 - Computation v I/O intensive
 - Discrete event v continuous dynamics
-

Embedded Systems - ARM

- ARM evolved from RISC design
- Used mainly in embedded systems
 - Used within product
 - Not general purpose computer
 - Dedicated function
 - E.g. Anti-lock brakes in car

Possible Organization of an Embedded System



ARM Evolution

- Designed by ARM Inc., Cambridge, England
 - Licensed to manufacturers
 - High speed, small die, low power consumption
 - PDAs, hand held games, phones
 - E.g. iPod, iPhone
 - Acorn produced ARM1 & ARM2 in 1985 and ARM3 in 1989
 - Acorn, VLSI and Apple Computer founded ARM Ltd.
-

ARM Systems Categories

- Embedded real time
- Application platform
 - Linux, Palm OS, Symbian OS, Windows mobile
- Secure applications

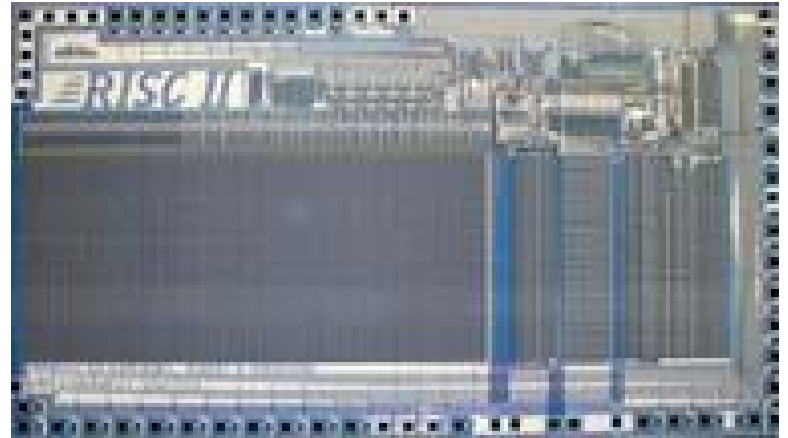
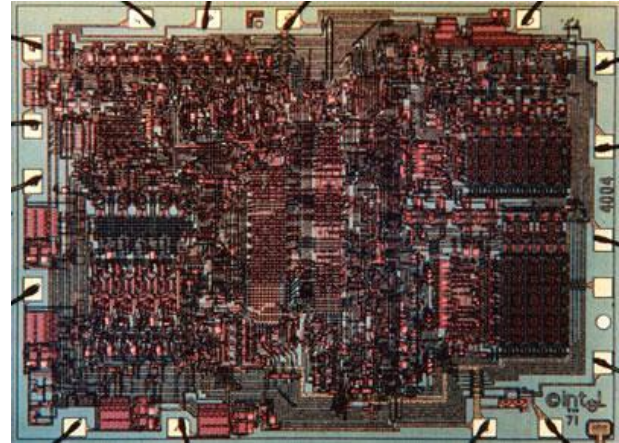
General Purpose Processors

Processor	Clock speed	Periph.	Bus Width	MIPS	Power	Trans.	Price
General Purpose Processors							
Intel PIII	1GHz	2x16 K L1, 256K L2, MMX	32	~900	97W	~7M	\$900
IBM PowerPC 750X	550 MHz	2x32 K L1, 256K L2	32/64	~1300	5W	~7M	\$900
MIPS R5000	250 MHz	2x32 K 2 way set assoc.	32/64	NA	NA	3.6M	NA
StrongARM SA-110	233 MHz	None	32	268	1W	2.1M	NA
Microcontroller							
Intel 8051	12 MHz	4K ROM, 128 RAM, 32 I/O, Timer, UART	8	~1	~0.2W	~10K	\$7
Motorola 68HC811	3 MHz	4K ROM, 192 RAM, 32 I/O, Timer, WDT, SPI	8	~.5	~0.1W	~10K	\$5
Digital Signal Processors							
TI C5416	160 MHz	128K, SRAM, 3 T1 Ports, DMA, 13 ADC, 9 DAC	16/32	~600	NA	NA	\$34
Lucent DSP32C	80 MHz	16K Inst., 2K Data, Serial Ports, DMA	32	40	NA	NA	\$75

Sources: Intel, Motorola, MIPS, ARM, TI, and IBM Website/Datasheet; Embedded Systems Programming, Nov. 1998

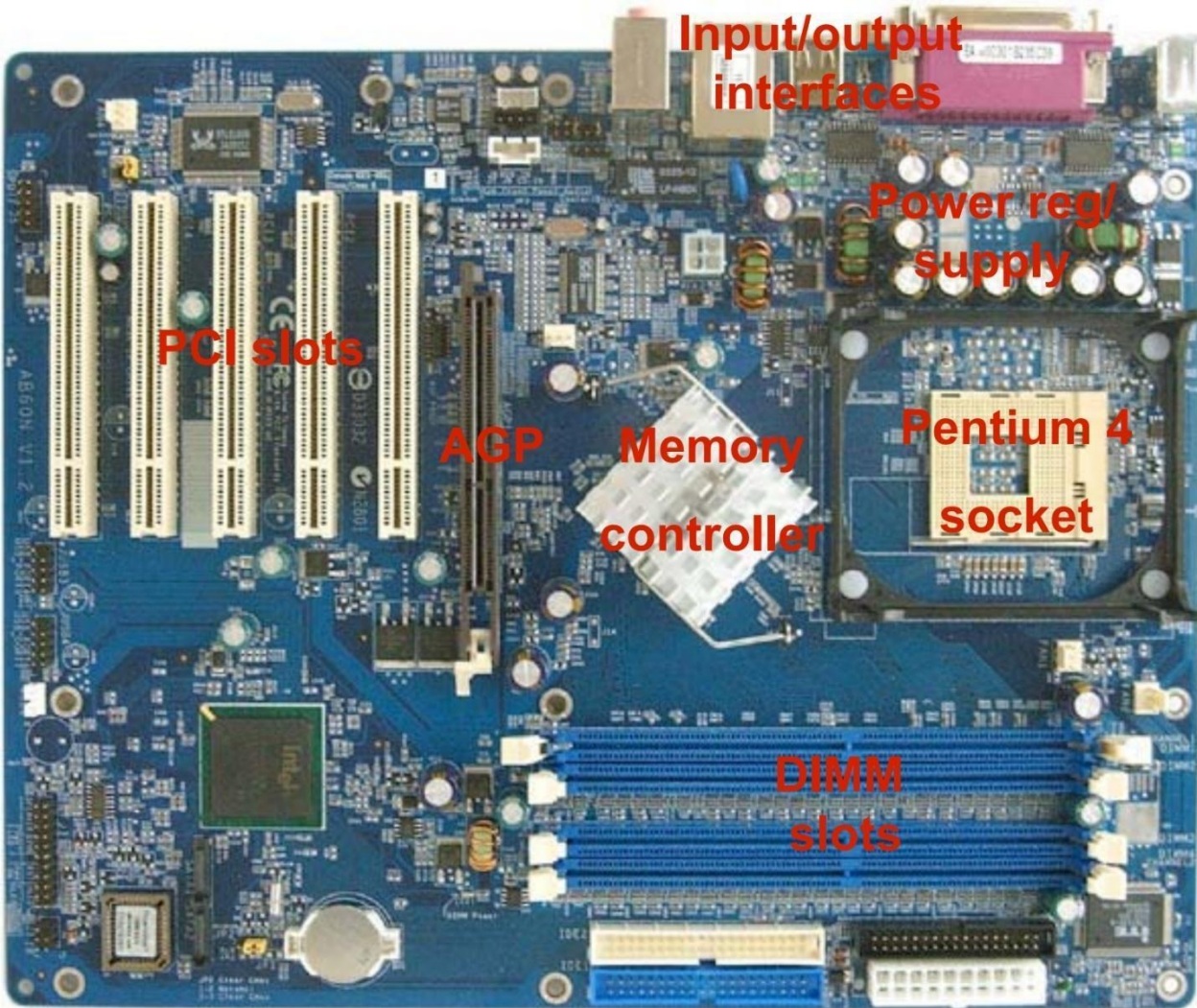
Sea Change in Chip Design

- Intel 4004 (1971): 4-bit processor, 2312 transistors, 0.4 MHz, 10 micron PMOS, 11 mm² chip
- RISC II (1983): 32-bit, 5 stage pipeline, 40,760 transistors, 3 MHz, 3 micron NMOS, 60 mm² chip
- 125 mm² chip, 0.065 micron CMOS
= 2312 RISC II+FPU+Icache+Dcache
 - RISC II shrinks to ~ 0.02 mm² at 65 nm
 - Caches via DRAM or 1 transistor SRAM?



- **Processor is the new transistor?**

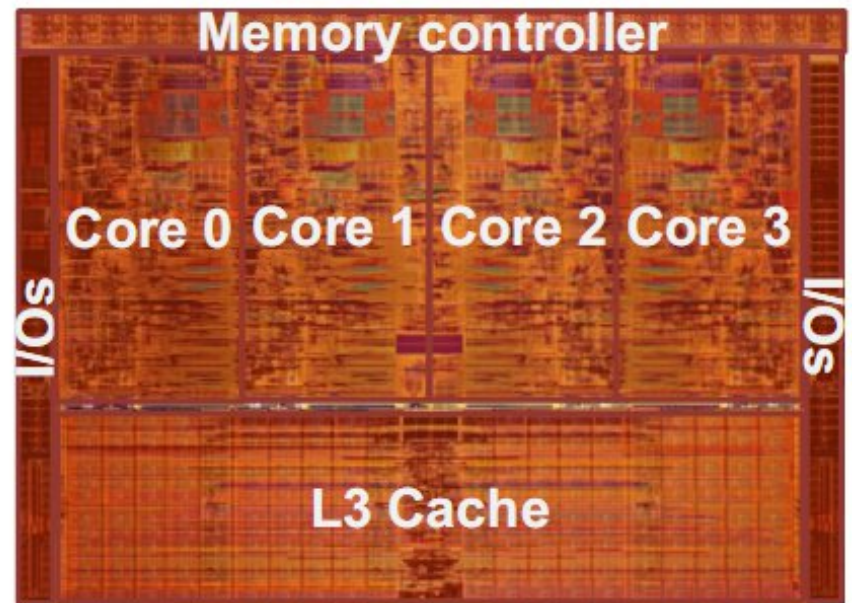
Pentium 4 PC Mother Board



- Pentium 4 2.66 GHz
 - 8KB Data cache, 12 KB Instruction cache
 - 512 KB L2 Cache
 - 533 MHz System Bus
 - 68 Watts
- Memory system
 - 4 DDR DIMM slots
 - Up to 4 GB
- I/O interfaces
 - Ethernet
 - USB
 - Serial ATA (disk)
 - Serial port
 - Parallel port

Intel Core i7

- 4 cores/chip
- 16 pipeline stages, ~3GHz
- 4-wide superscalar
- Out of order, 128-entry reorder buffer
- 2-level branch predictors
- Caches:
 - L1: 32KB I + 32KB D
 - L2: 256KB
 - L3: 8MB, shared



Execution Units	L1 Data Cache	L2 Cache & Interrupt Servicing
	Memory Ordering & Execution	Paging
Out-of-Order Scheduling & Retirement	Instruction Decode & Microcode	Branch Prediction
		Instruction Fetch & L1 I-Cache

PS2 Mother Board

PS2 Motherboard

64 b MIPS CPU 300 MHz
Behavioral synthesis,
geometry processing, main
system control

Sony
#CXD9542GB
Emotion Engine
Processor (Toshiba)

Toshiba
#RM718GB
RAMBUS™
DRAM

Sony
#CXD2934GB
Graphics Synthesizer

Rendering
Texture
Frame-
buffer ops

Sony
#CXD9566R
No Data Available

32 b MIPS CPU 34 MHz
IO processing
PS1 emulation

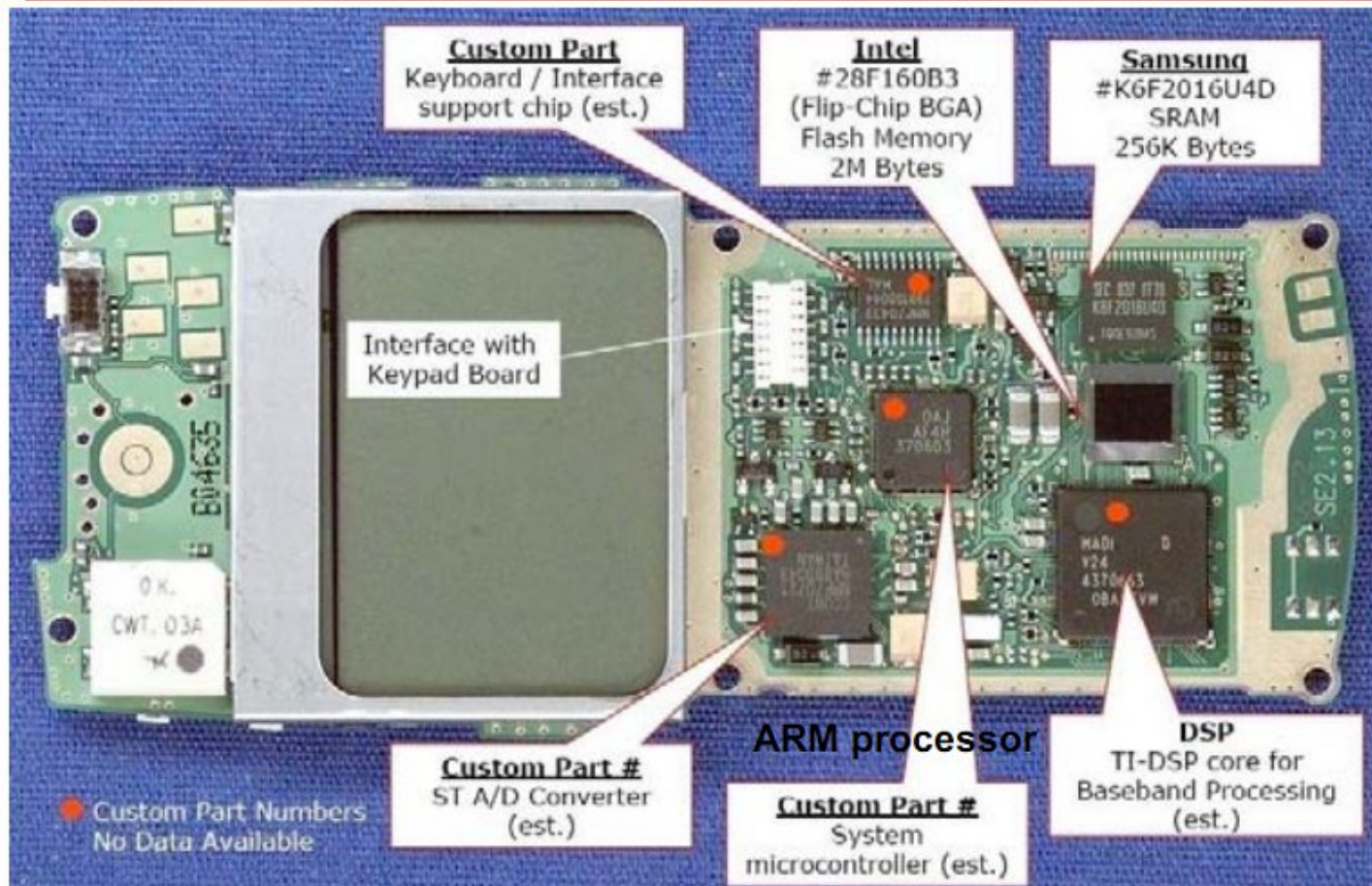
Sony
#CXD9553GB

Sony
#CXD2942R

OKI
#M51V18165
DRAM
2M Bytes

Digital Cell Phone

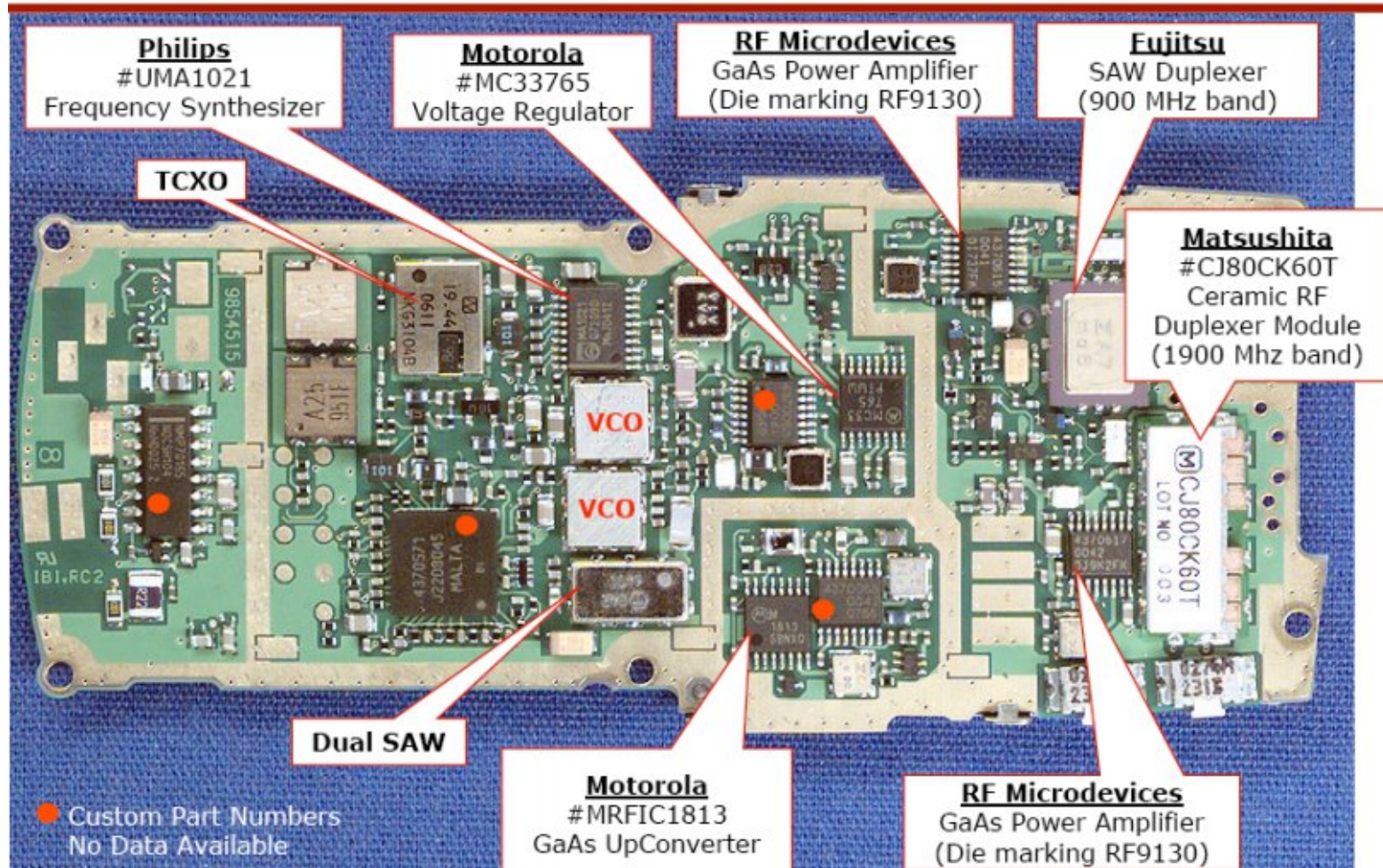
Digital Cell Phone (Nokia 8260) Front Side



- Battery
900 mAhr
- 3.5 hr talk
– ~ 1 W
- 8 days
standby
– ~ 1mW

Digital Cell Phone

Digital Cell Phone (Nokia 8260) Back Side

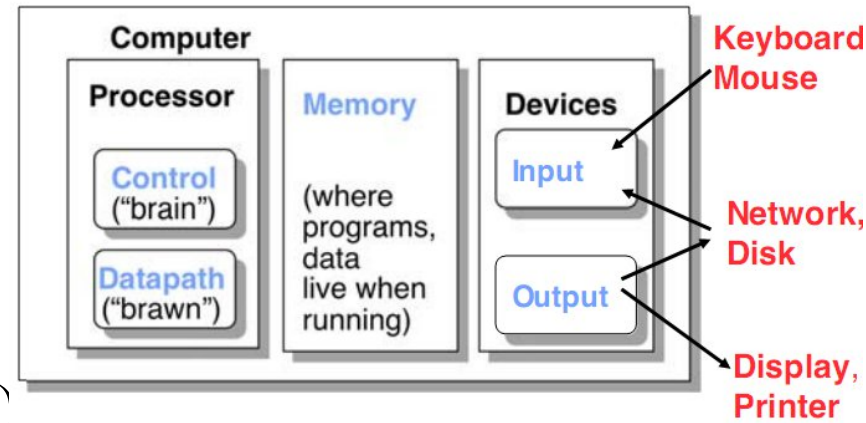


How Computers (All Digital Systems) Works?

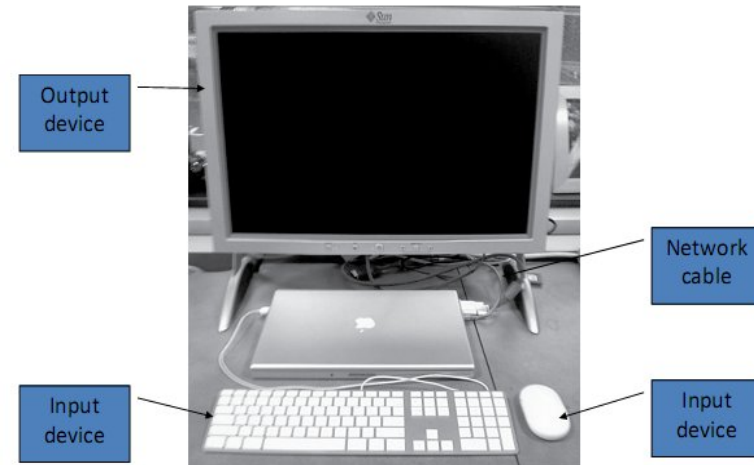
- Computers only work with binary signals
 - Signal on a wire is either 0, or 1
 - Usually called a “bit”
 - More complex stuff (numbers, characters, strings, pictures)
 - Must be built from multiple bits
- Built out of simple logic gates that perform boolean logic
 - AND, OR, NOT, ...
- and memory cells that preserve bits over time
 - Flip-flops, registers, SRAM cells, DRAM cells, ...
- To get hardware to do anything, need to break it down to bits
 - Strings of bits that tell the hardware what to do are called **instructions**
 - A sequence of instructions called **machine language** program (machine code)

Computer Components and Abstraction

- Components:
 - input (mouse, keyboard)
 - output (display, printer)
 - memory
(disk drives, DRAM, SRAM, CD)
 - network
 - **Processor (datapath and control):
our primary focus.**
- They are implemented using millions of transistors
 - Impossible to understand by looking at each transistor
 - We need “**Abstraction**” such as ISA

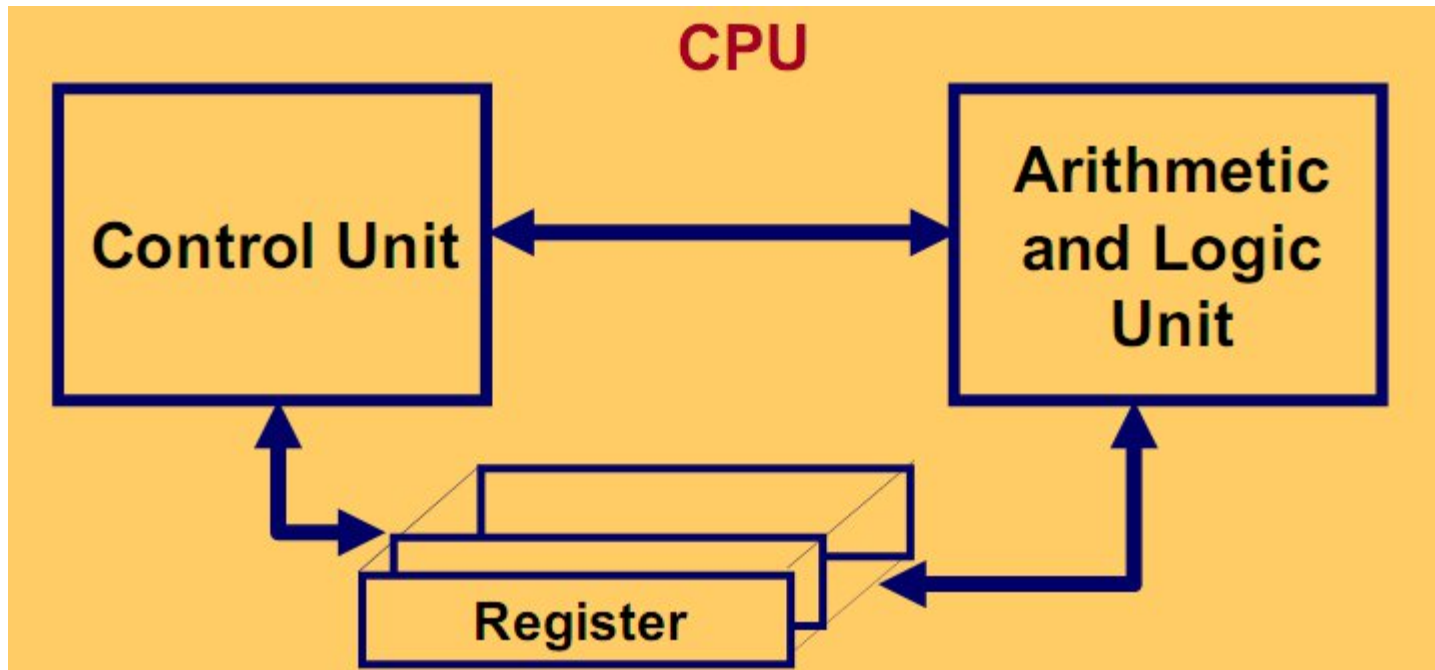


Anatomy of a Computer



Central Processing Unit (CPU)

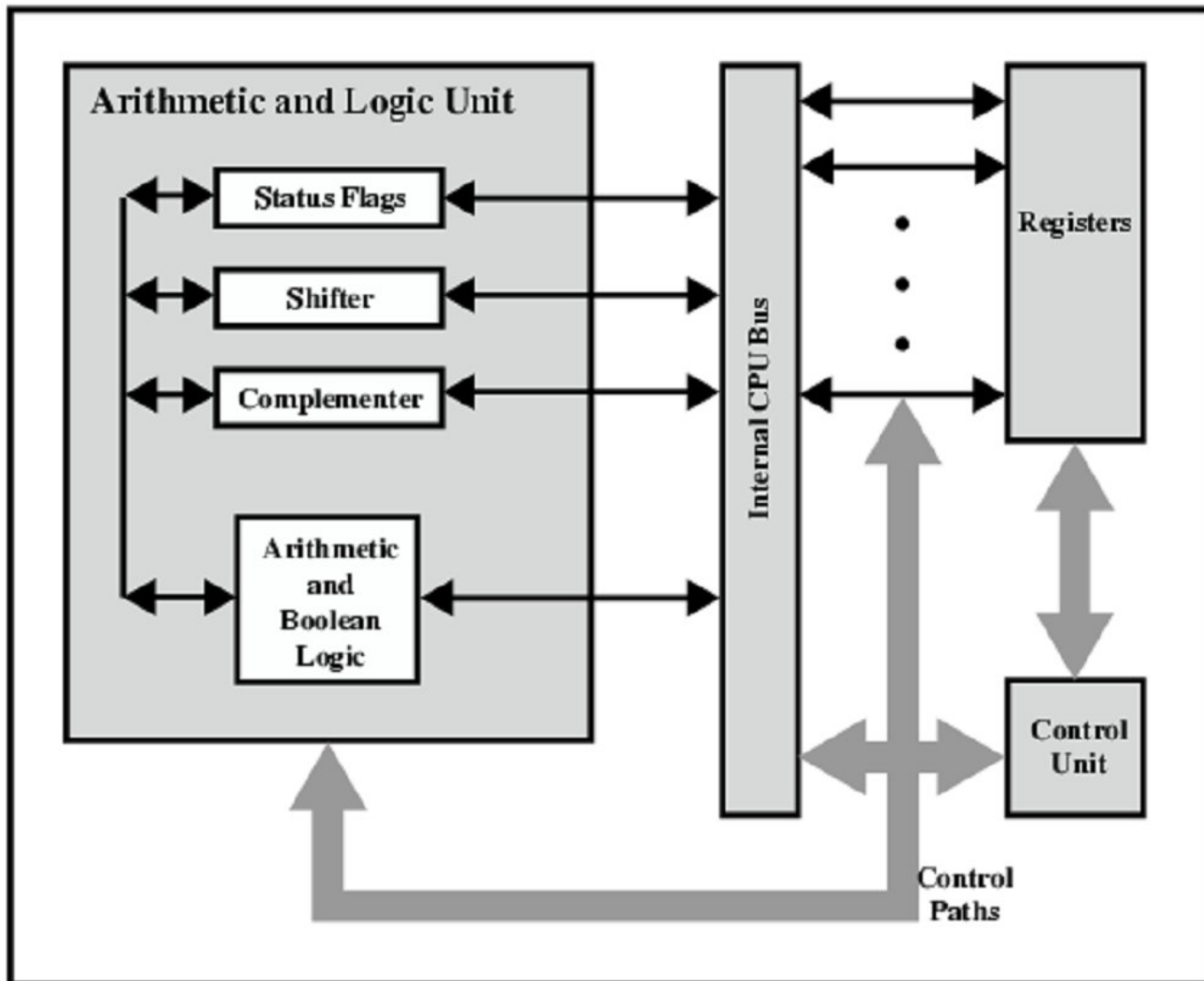
- The Central Processing Unit (CPU), also called processor, includes two main units:
 - A program control unit, and
 - An Arithmetic and Logic Unit (ALU).



Central Processing Unit (CPU)

- The primary function of a CPU is to execute the instructions stored in the main memory.
 - An instruction tells the CPU to perform one of its basic operations.
 - The CPU includes also a set of registers, which are temporary storage devices used to hold control information, key data, and intermediate results.
 - It includes also an internal bus infrastructure, which provides data movement paths among the control unit, ALU, and registers.
 - The CU is the one which interprets (decodes) the instruction to be executed and "tells" the other components what to do.
-

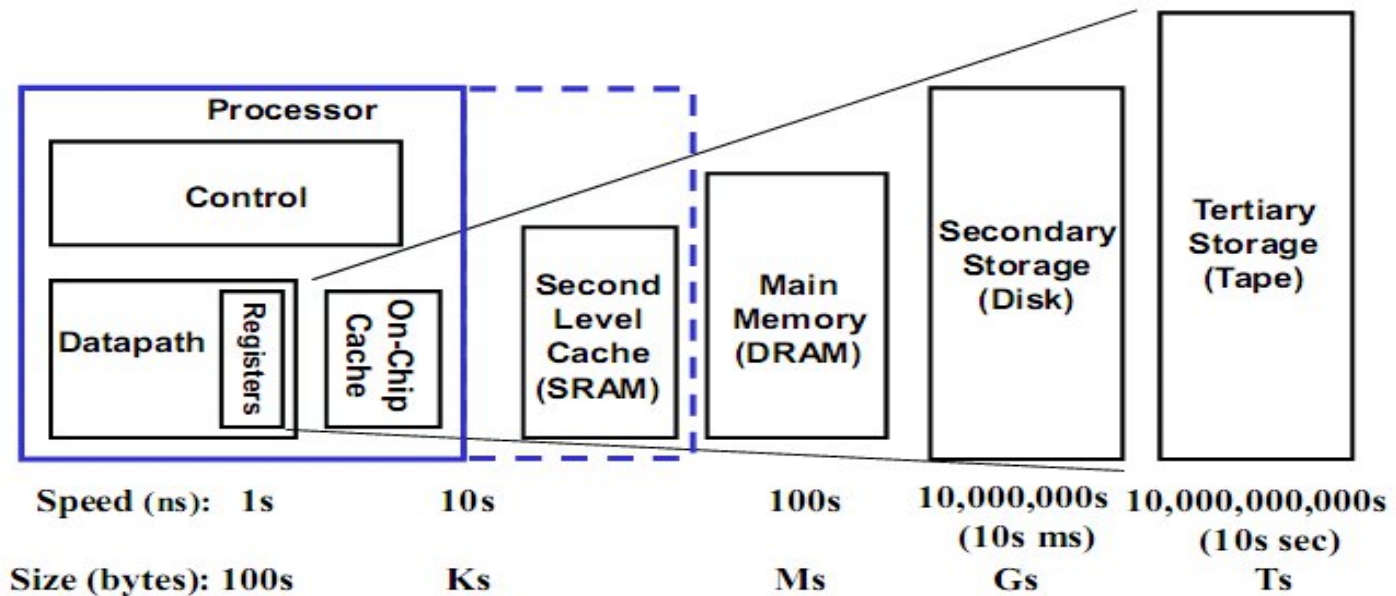
CPU Internal Structure



Registers

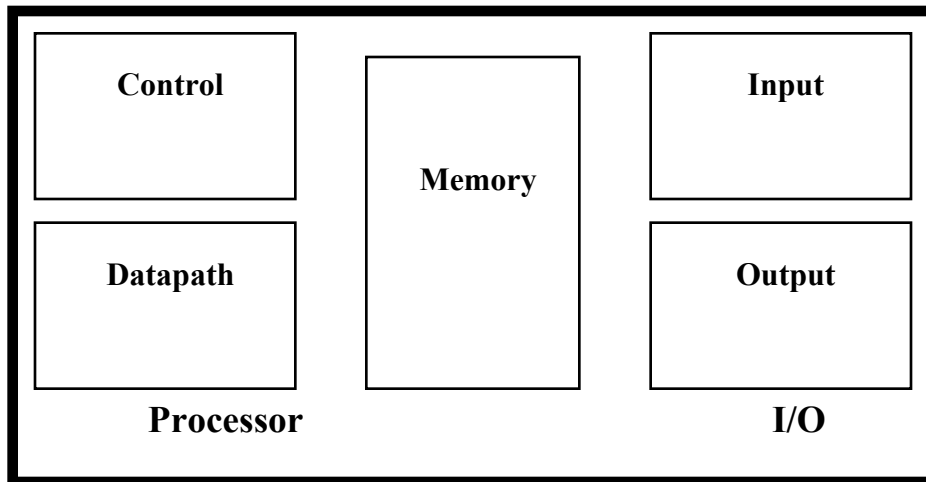
- CPU must have some working space (temporary storage).
- These storage units are called registers.
- They are the top level component in the memory hierarchy.
- Number and function of the registers vary between different computers.
- Register organization is one of the major design decisions

Memory Hierarchy of a Modern Computer System



Registers vs. Memory

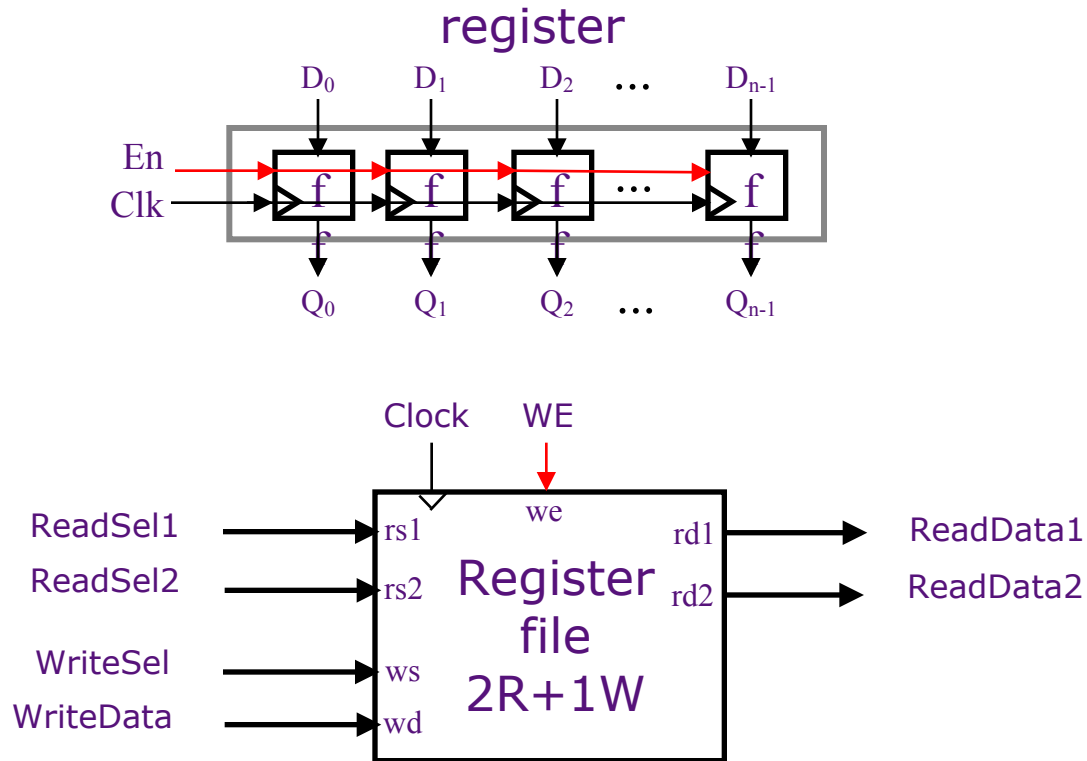
- Arithmetic instruction operands must be registers (RISC),
 - only a limited number (8, 16, 32, 64) of registers provided
- Compiler associates variables with registers
- What about programs with lots of variables
 - Register reuse?
 - Who does it?



Register Organization

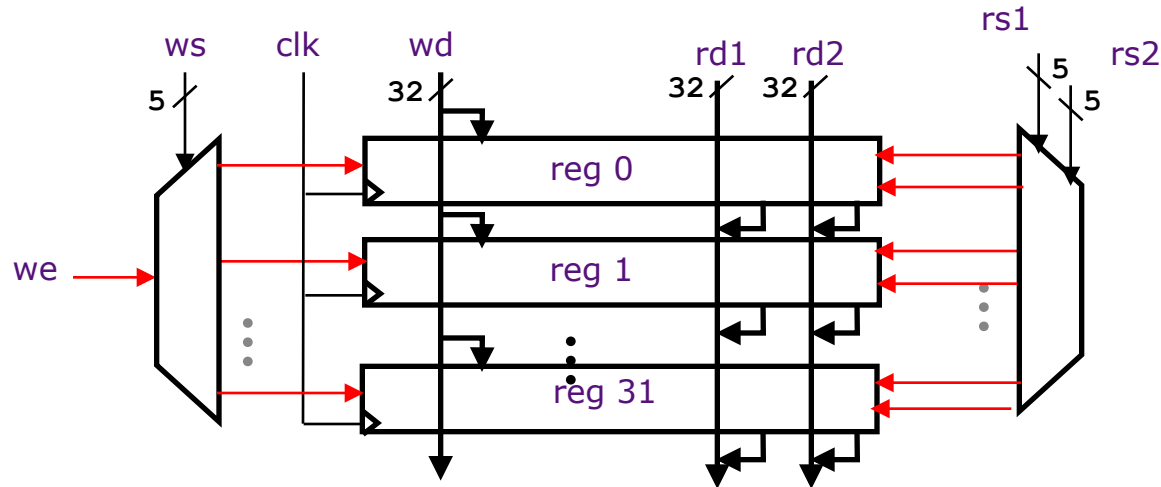
- The registers serve two main functions:
 - User-Visible Registers: used by machine or assembly language programmers to minimize memory access.
 - General-purpose registers
 - Data registers
 - Address registers
 - Condition code registers
 - Control and Status Registers: used by the control unit to control the operation of the CPU, and by the operating system to control the execution of programs.

Register Organization: Register File



- Reads are combinational

Register File Implementation



- Register files with a large number of ports are difficult to design
 - Almost all MIPS instructions have exactly 2 register source operands
 - *Intel's Itanium, GPR File has 128 registers with 8 read ports and 4 write ports!!!*

Memory Organization

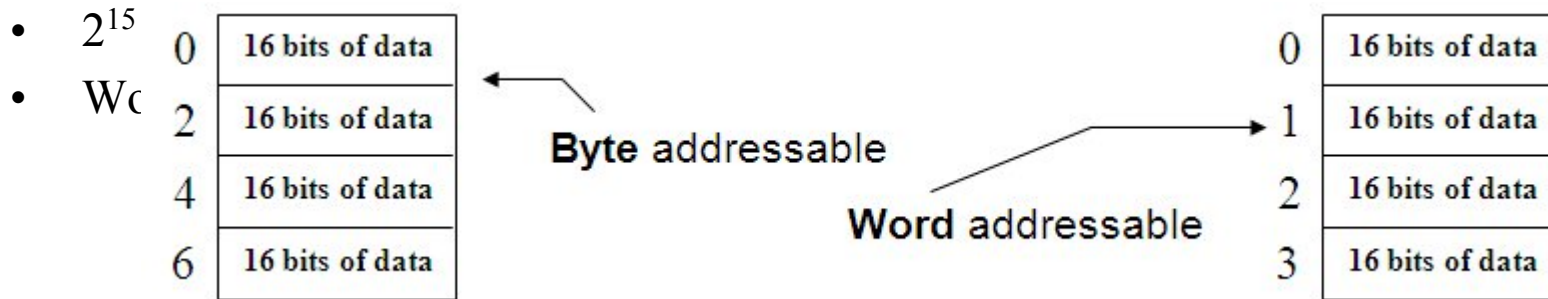
- Viewed as a large, single-dimension array, with an address.
- A memory address is an index into the array.
- "Byte addressing" means that the index points to a byte of memory.
- "Word addressing" means the index points to the first byte of a word

0	8 bits of data
1	8 bits of data
2	8 bits of data
3	8 bits of data
4	8 bits of data
5	8 bits of data
6	8 bits of data

...

Memory Organization

- Bytes are useful for MM apps, but most data items use larger "words"
- For LC-3, a word is 16 bits (2 bytes) [for MIPS, a word is 32 bits or 4 bytes].
- 2^{16} bytes with byte addresses from 0 to $2^{16}-1$

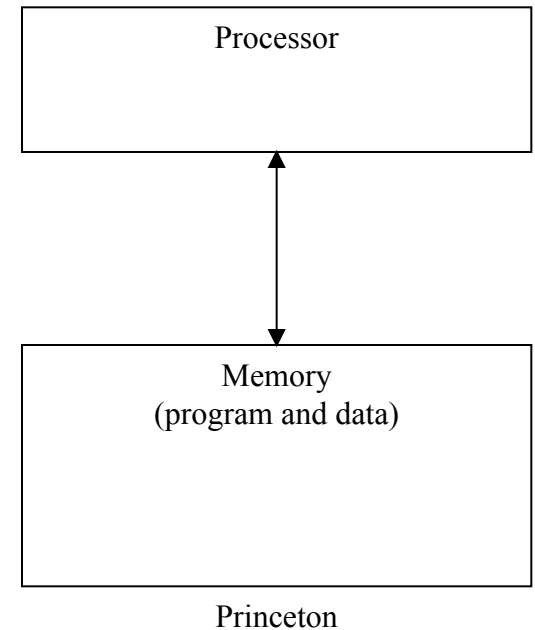
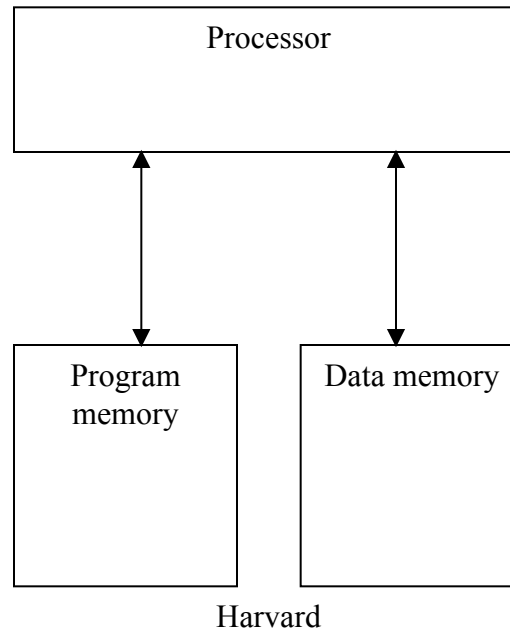


Registers vs Memory

- Registers are faster to access than memory
- Operating on memory data requires loads and stores
 - More instructions to be executed
- Compiler must use registers for variables as much as possible
 - Only spill to memory for less frequently used variables
 - Register optimization is important!

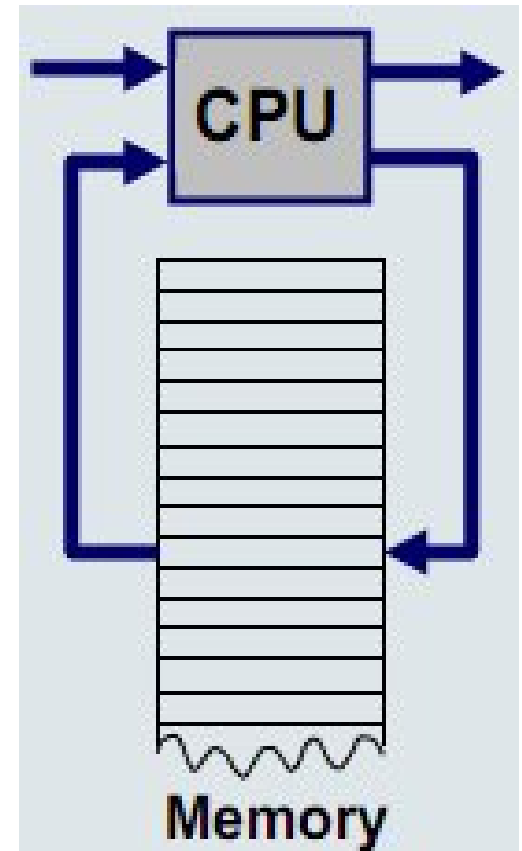
Two Memory Architectures

- Princeton (Von-Neumann)
 - Fewer memory wires
- Harvard
 - Simultaneous program and data memory access



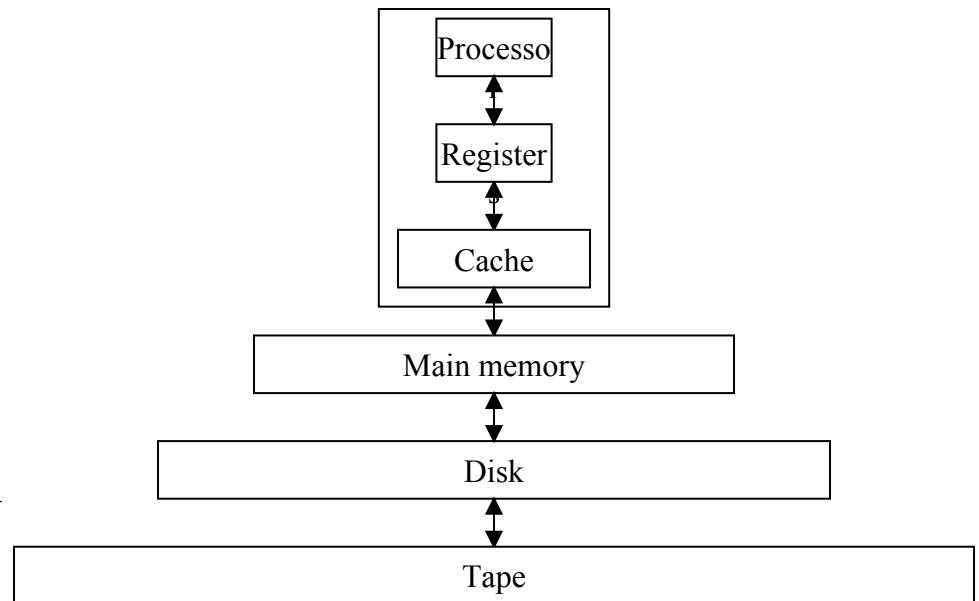
Why Von-Neumann Architecture in Computers?

- General-purpose, programmable.
 - They can solve very different problems by executing different programs.
- Instruction execution is done automatically.
- It can be built with very simple electronics components:
 - Data processing function is performed by electronic gates.
 - Data storage function is provided by memory cells.
 - Data communication is achieved by electrical wires.



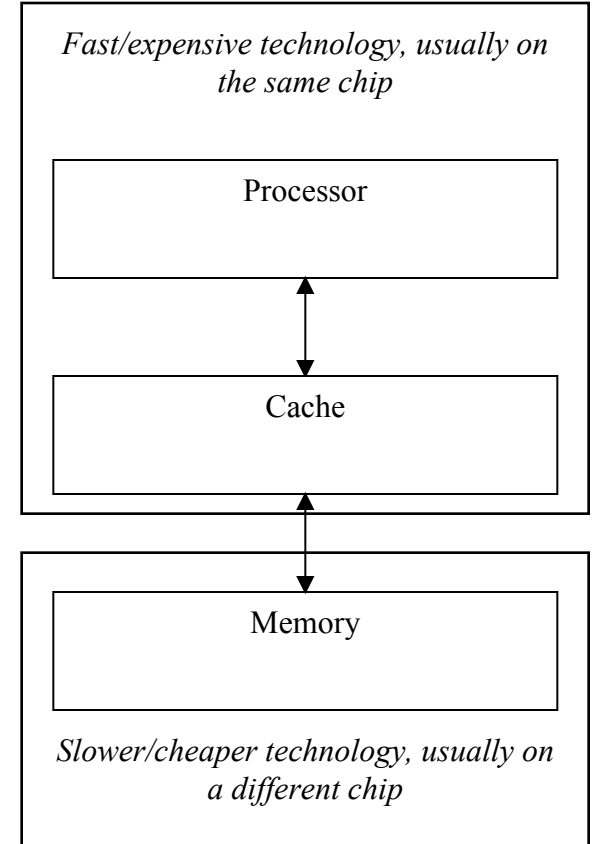
Memory hierarchy

- Want inexpensive, fast memory
- Main memory
 - Large, inexpensive, slow memory stores entire program and data
- Cache
 - Small, expensive, fast memory stores copy of likely accessed parts of larger memory
 - Can be multiple levels of cache



Cache Memory

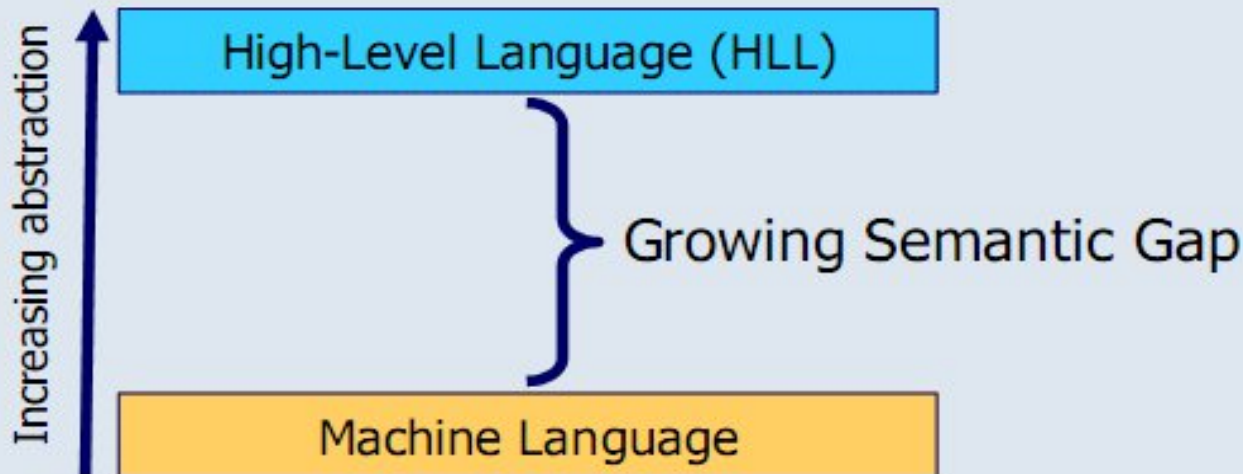
- Memory access may be slow
- Cache is small but fast memory close to processor
 - Holds copy of part of memory
 - Hits and misses



Semantic Gap

The Semantic Gap

- In order to improve efficiency of software development, new and powerful high-level programming languages have been developed (e.g., Ada, C++, Java), which provide high level of abstraction.
- This evolution has increased the semantic gap between programming languages and machine languages.

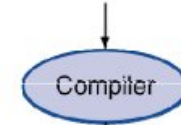


Hierarchical Layers of Program Code

- High-level language
 - Level of abstraction closer to problem domain
 - Provides for productivity and portability
- Assembly language
 - Textual representation of instructions
- Hardware representation
 - Binary digits (bits)
 - Encoded instructions and data

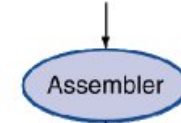
High-level
language
program
(in C)

```
swap(int v[], int k)
{
    int temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```



Assembly
language
program
(for MIPS)

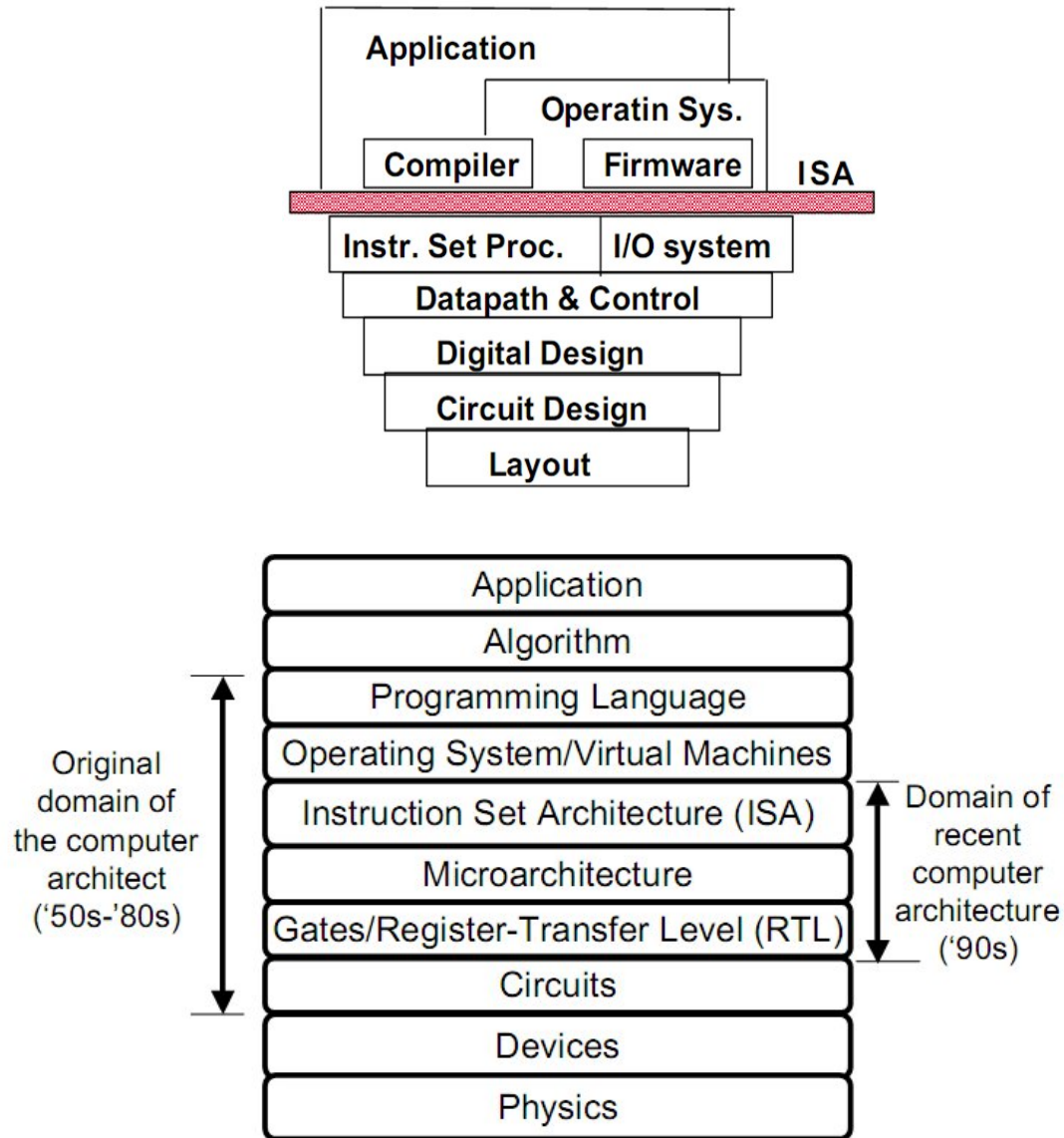
```
swap:
    muli $2, $5, 4
    add $2, $4, $2
    lw $15, 0($2)
    lw $16, 4($2)
    sw $16, 0($2)
    sw $15, 4($2)
    jr $31
```



Binary machine
language
program
(for MIPS)

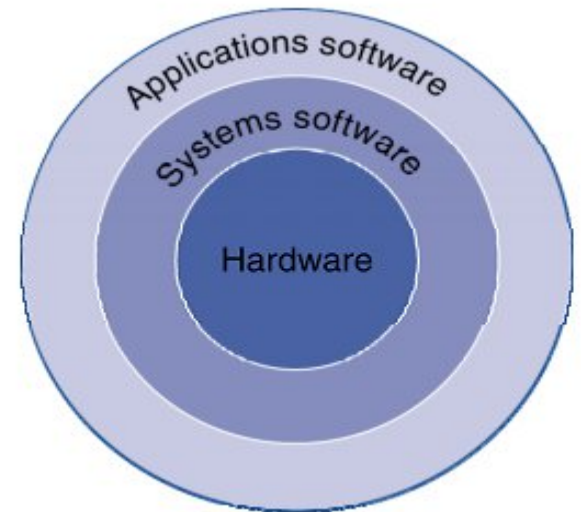
```
00000000101000010000000000000000
00000000000001100000001100000100001
1000110001100010000000000000000000
1000110011110010000000000000000100
1010110011110010000000000000000000
1010110001100010000000000000000100
0000001111100000000000000000001000
```

Levels of Abstraction



Hierarchical Layers of Program Code

- Application software
 - Written in high-level language
- System software
 - Compiler: translates HLL code to machine code
 - Operating System: service code
 - Handling input/output
 - Managing memory and storage
 - Scheduling tasks & sharing resources
- Hardware
 - Processor, memory, I/O controllers



Levels of Representation

High Level Language
Program

Compiler

Assembly Language
Program

Assembler

Machine Language
Program

Machine Interpretation

Control Signal Spec

```
temp = v[k];  
v[k] = v[k+1];  
v[k+1] = temp;
```

```
lw $15, 0($2)
```

```
lw $16, 4($2)
```

```
sw $16, 0($2)
```

```
sw $15, 4($2)
```

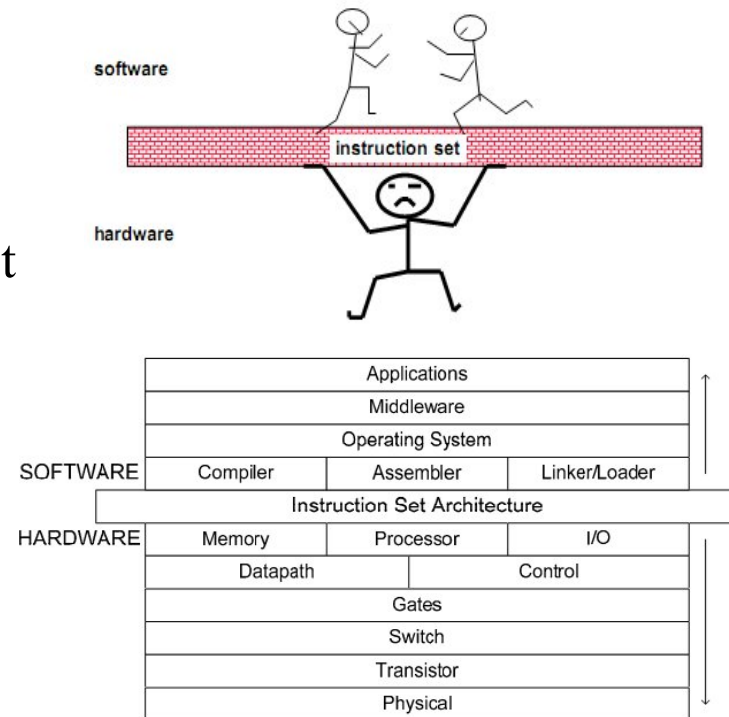
Instruction Set Architecture

- ISA defines what instructions do
- A very important abstraction
 - interface between hardware and low-level software
 - standardizes instructions, machine language bit patterns, etc.
 - advantage: different implementations of the same architecture
 - disadvantage: sometimes prevents using new innovations

True or False: Binary compatibility is extraordinarily important?

- Modern instruction set architectures:
 - 80x86/Pentium implement IA32
 - PowerPC, DEC Alpha, MIPS, SPARC,

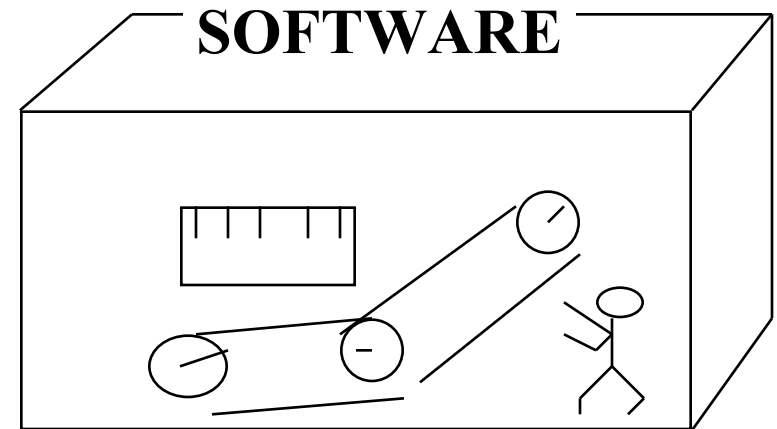
ARM, ...



Instruction Set Architecture

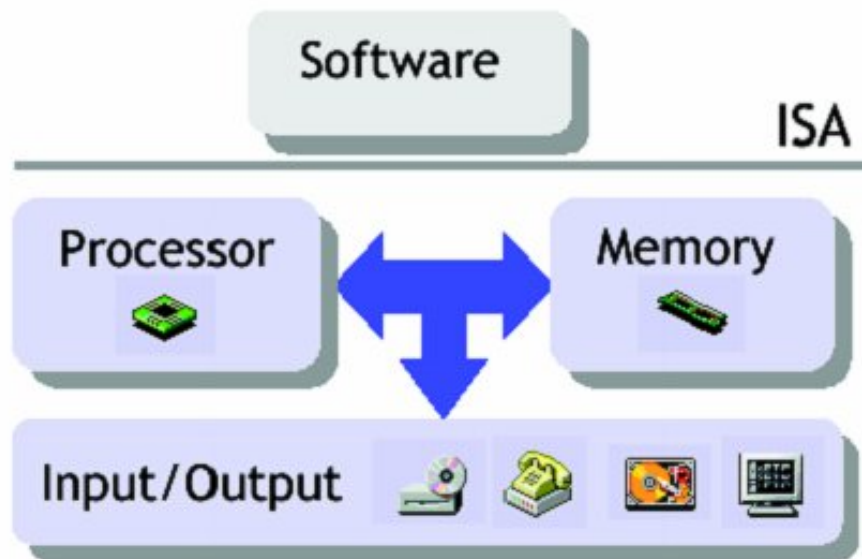
“... the attributes of a [computing] system as seen by the programmer, *i.e.* the conceptual structure and functional behavior, as distinct from the organization of the data flows and controls the logic design, and the physical implementation.” – Amdahl, Blaauw, and Brooks, 1964

- Organization of Programmable Storage
- Data Types & Data Structures: Encodings & Representations
- Instruction Formats
- Instruction (or Operation Code) Set
- Modes of Addressing and Accessing Data Items and Instructions
- Exceptional Conditions



Instruction Set Architecture

- An instruction set describes the basic functions that a processor can perform
- The ISA serves as an abstraction layer between the HW and SW
 - Software doesn't need to know how the processor is implemented
 - Any processor that implements the ISA appears equivalent
- Following ISA makes innovation of processor without changing software



Language of the hardware

- We constantly interact with computers
 - E.g., via apps on iPhone or via a Word Processor
- How does an application interact with the Hardware?

Language of the hardware

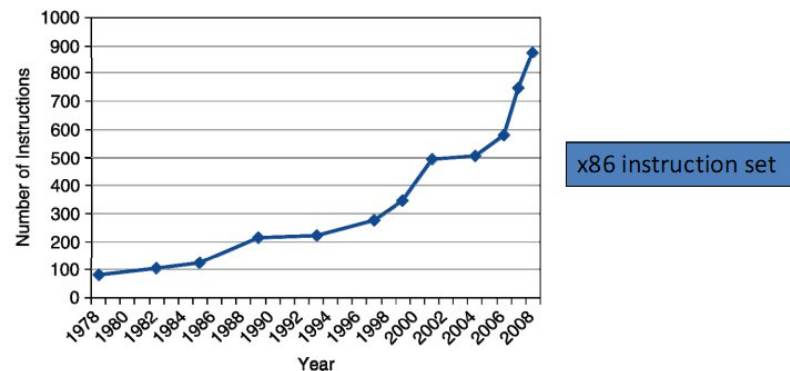
- The hardware understands on or off. Hence, the language of the hardware needs two symbols 0 and 1.
- Commonly referred to as binary digits or bits.
- 2 letters do not limit what computers can do (just like the finite letters in our languages)
- What our programs instruct the computer to do are the *instructions*:
 - Instructions are collections of bits that the computer understands
 - Hundreds/thousands/millions of lines of code (instructions) are hidden beneath our apps and programs

Assembly Language vs. Machine Language

- Assembly provides convenient symbolic representation
 - much easier than writing down numbers
 - e.g., destination first
- Machine language is the underlying reality
 - e.g., destination is no longer first
- Assembly can provide 'pseudo-instructions'
 - e.g., “move \$t0, \$t1” exists only in Assembly
 - would be implemented using “add \$t0,\$t1,\$zero”
- When considering performance you should count real instructions

Fallacies

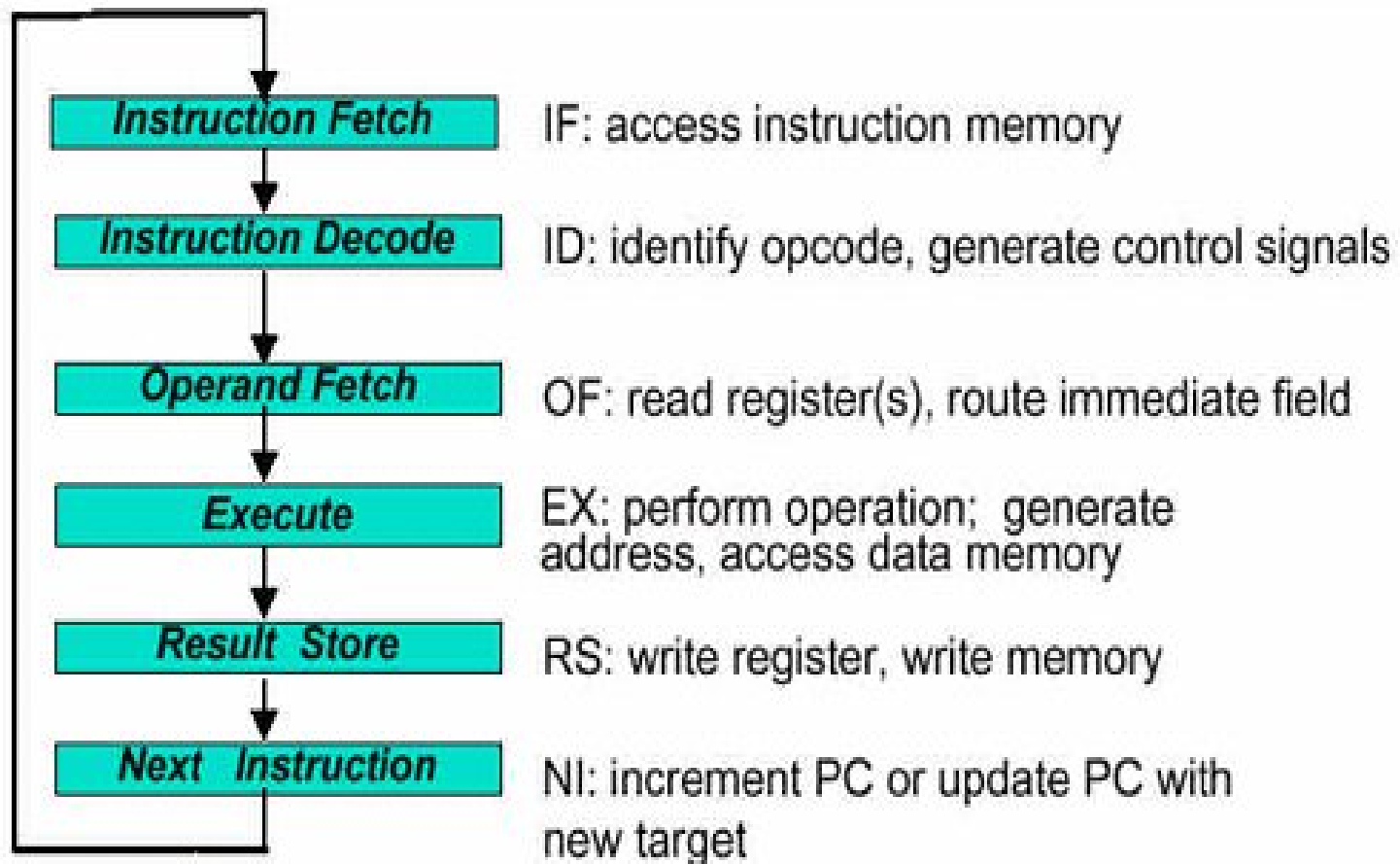
- Powerful instruction $\Rightarrow$ higher performance
 - Fewer instructions required
 - But complex instructions are hard to implement
 - May slow down all instructions, including simple ones
 - Compilers are good at making fast code from simple instructions
- Use assembly code for high performance
 - But modern compilers are better at dealing with modern processors
 - More lines of code $\Rightarrow$ more errors and less productivity
- Backward compatibility $\Rightarrow$ instruction set doesn't change
 - But they do accrete more instructions



Instruction Set Design

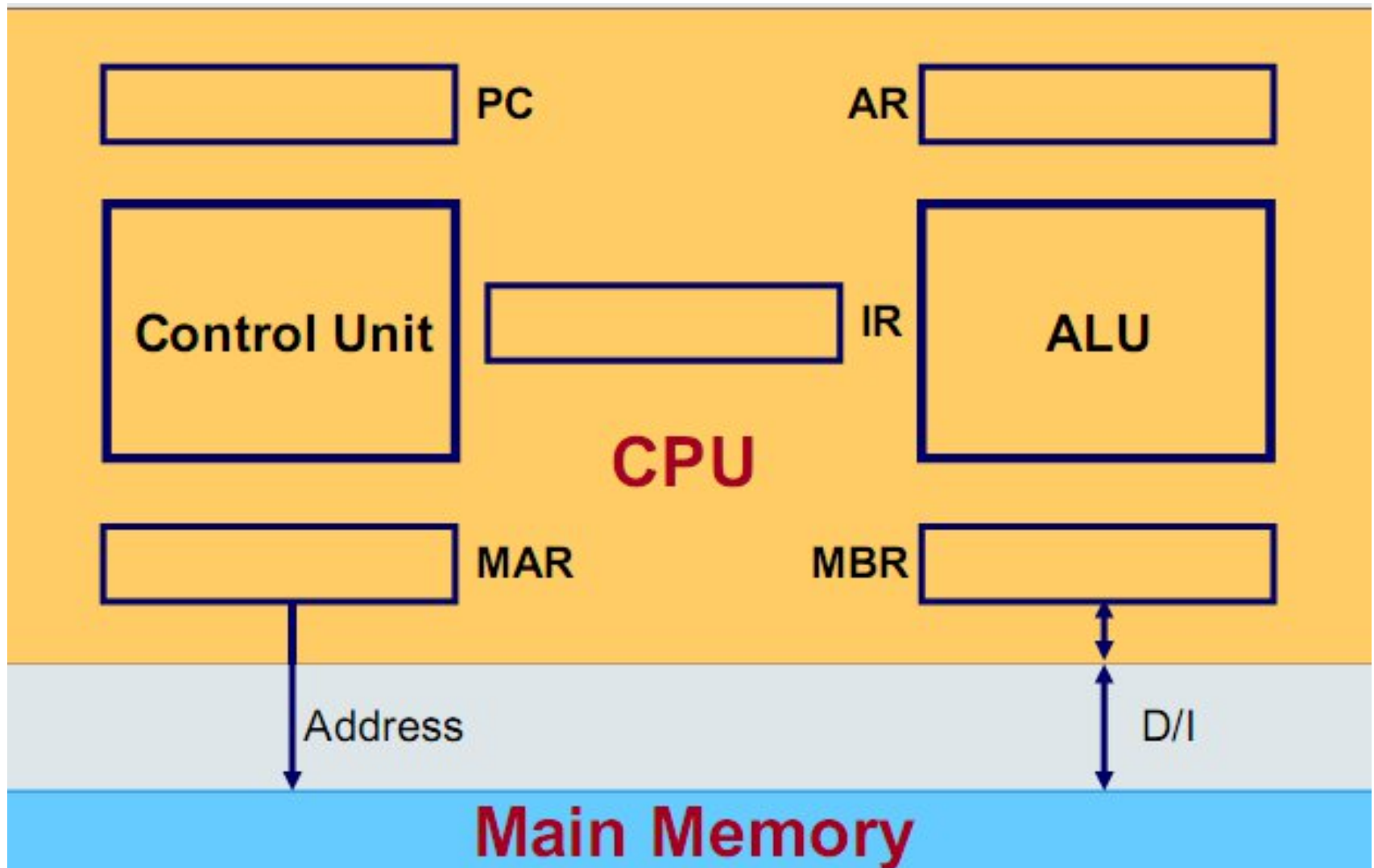
- The design of an instruction set is critical to the operations of a computer system. The most important issues are:
 - Operation repertoire — How many and which operations to provide, and how complex operations should be.
 - Data types — Which data types to be supported.
 - Instruction format — Length, number of addresses, size of various fields, etc.
 - Registers — Number of CPU registers and their use.
 - Addressing — Which modes to be provided.
- The issues are highly interrelated and must be considered together.

Instruction execution

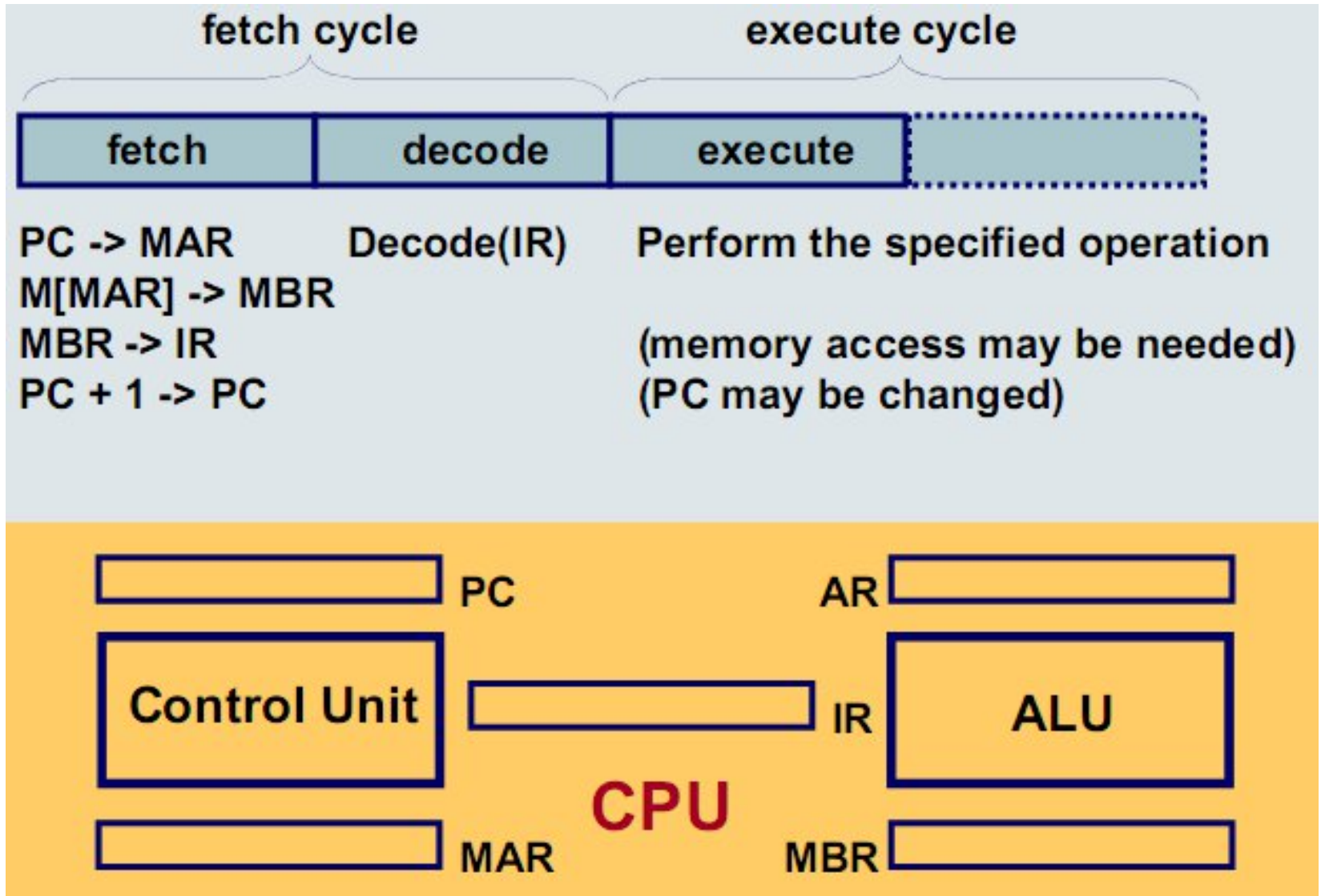


Note that each step does not necessarily correspond to a clock cycle. These only describe the basic flow of instruction execution. The details vary with instruction type.

Instruction Execution Mechanism



Instruction Execution



Machine Cycles

- The execution of an instruction is carried out in a machine cycle (instruction cycle).
- The CPU executes one instruction after the other, cycle by cycle, repeatedly.
- The machine cycle time (or instruction execution time) of a computer gives an indication of its performance (speed).
 - Ex. a computer can have a performance of 733 MIPS (Millions of Instructions Per Second).
- Since different instructions need different time to execute, the average instruction execution time is used to calculate performance

CISC Approach CISC Approach

- Complex Instruction Set Computer

- C code:

$g = h + A[8];$

- CISC

add a,32

- Compiled MIPS code:

lw \$t0, 32(\$s3) # load word

add \$s1, \$s2, \$t0

- Achieved by building complex hardware that loads value from memory into a register and then adds it to register a and stores the results in register a.

CISC vs RISC

- Compiler has to do little
 - Programming was done in assembly language
 - To make it easy, more and more complex instructions were added
- Length of the code is short and hence, little memory is required to store the code
 - Memory was a very costly real-estate
- E.g., Intel x86 machines powering several million desktops
- Each instruction needs only one clock cycle
- Hardware is less complex
- RISC processors, despite their advantages, took several years to gain market
 - Intel had a head start of 2 years before its RISC competitors
 - Customers were unwilling to take risk with new technology and change software products
 - They have a large market and hence, they can afford resources to overcome complexity

CISC vs RISC

- Very good slides on the topic
 - <https://www.cs.drexel.edu/~wmm24/cs281/lectures/pdf/RISCvsCISC.pdf>
- Patterson's blog:
 - <http://blogs.arm.com/software-enablement/375-risc-versus-cisc-wars-in-the-prepc-and-pc-eras-part-1/>
- Interview with Hennessy
 - <http://www-cs-aculty.stanford.edu/~eroberts/courses/soco/projects/risc/about/interview.html>

CISC vs RISC: Concluding Remarks

- Design principles
 - 1.Simplicity favors regularity
 - 2.Smaller is faster
 - 3.Make the common case fast
 - 4.Good design demands good compromises
- Layers of software/hardware
 - Compiler, assembler, hardware
- MIPS: typical of RISC ISAs
 - c.f. x86

RISC machine examples

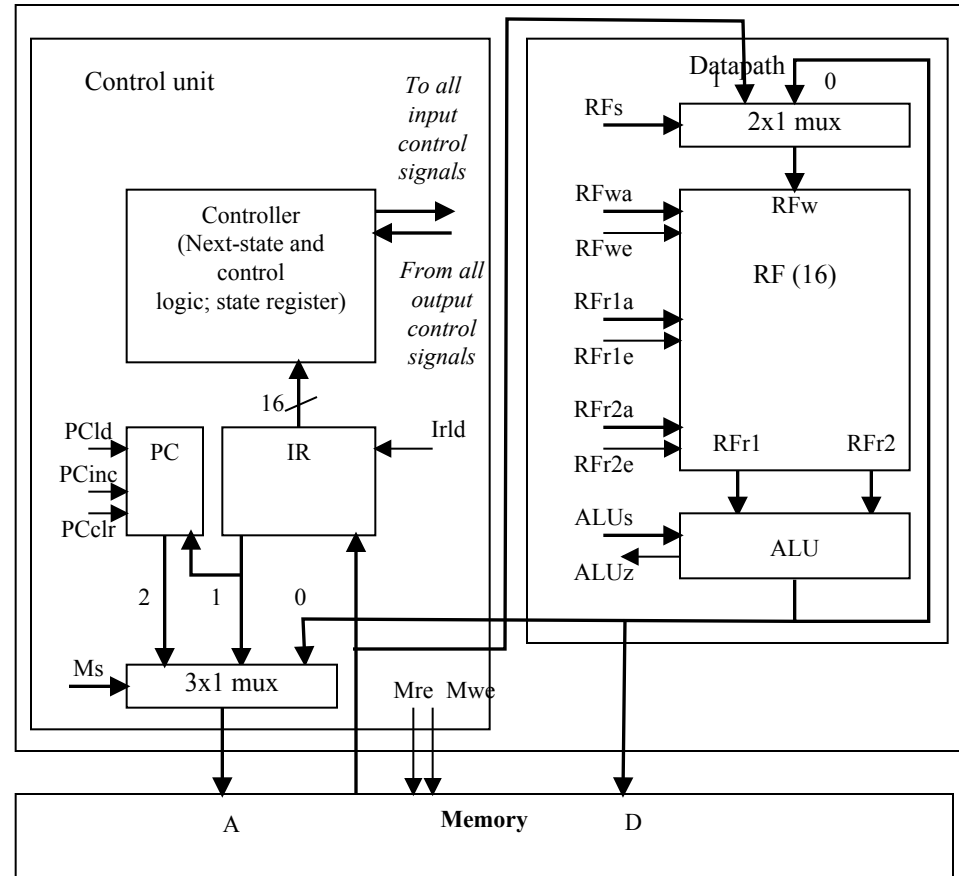
3/3

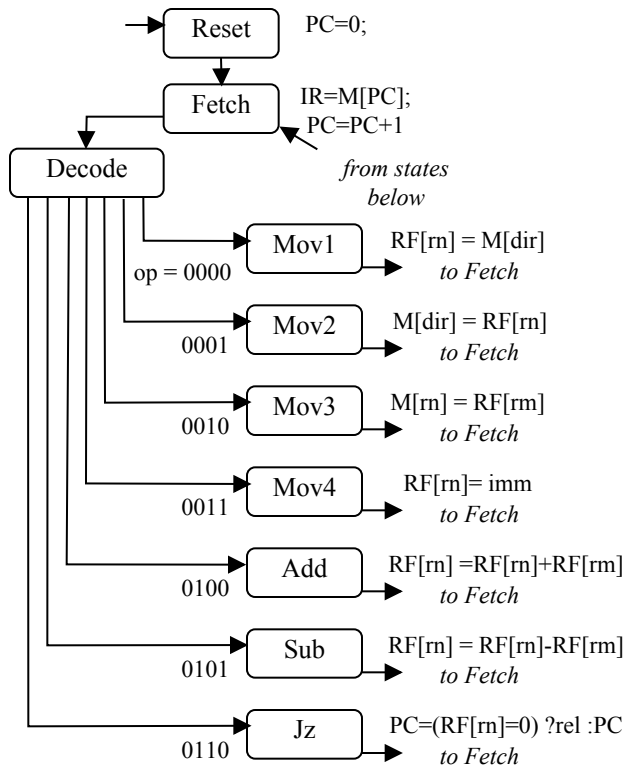
- Summary of 80386 and MIPS R2000 architectures [17]

	MIPS R2000	Intel 80386
Date announced	1986	1985
Instruction size (bits)	32	Variable
Address space (size, model)	32 bits, flat	32 bits, segmented with paging support
Data alignment	Aligned	No
Data addressing modes	2	11
Protection	Page	Segmented Scheme
Integer registers (number, model, size)	31 GPR*32 bits	8 GPR*32 bits, 6 segment registers*16 bits, 2 other * 16 bits
Separate floating-point registers	16*32 or 16*64 bits	8*80 bits
Floating-point format	IEEE 754 single, double	IEEE 754 single, double, extended

Architecture of a Simple Microprocessor

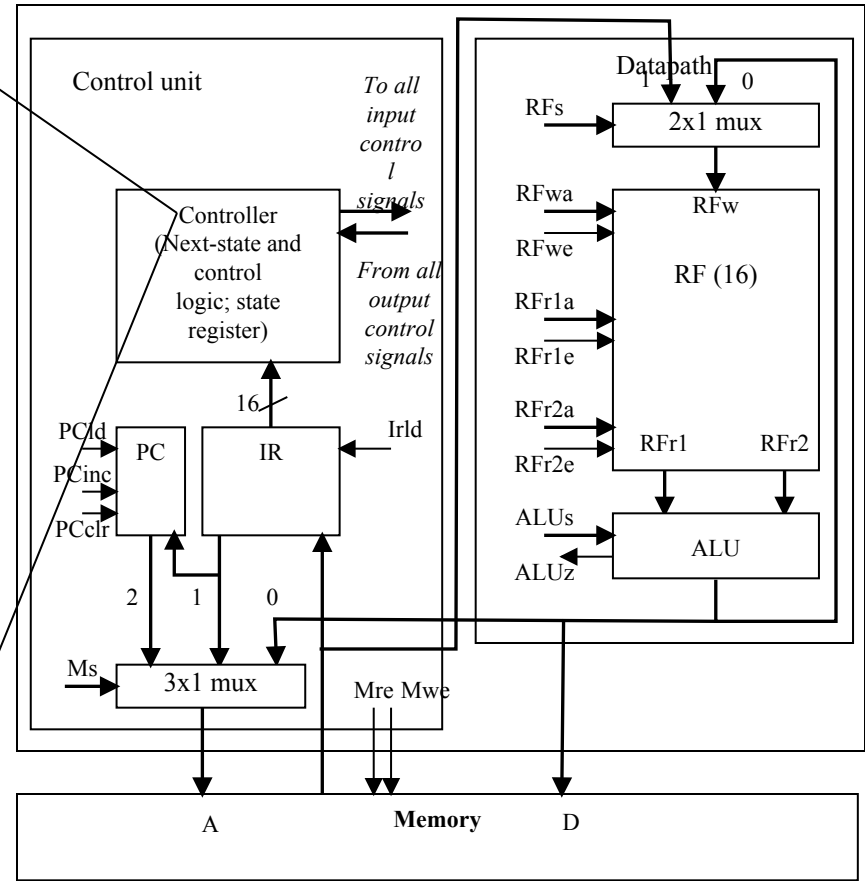
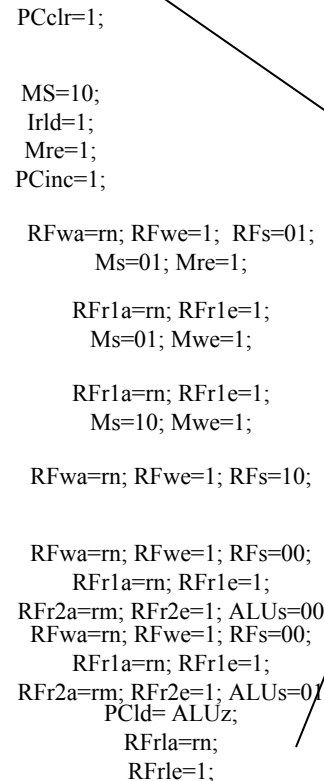
- Storage devices for each declared variable
 - register file holds each of the variables
- Functional units to carry out the FSMD operations
 - One ALU carries out every required operation
- Connections added among the components' ports corresponding to the operations required by the FSM
- Unique identifiers created for every control signal



 $FSMD$

You just built a simple microprocessor!

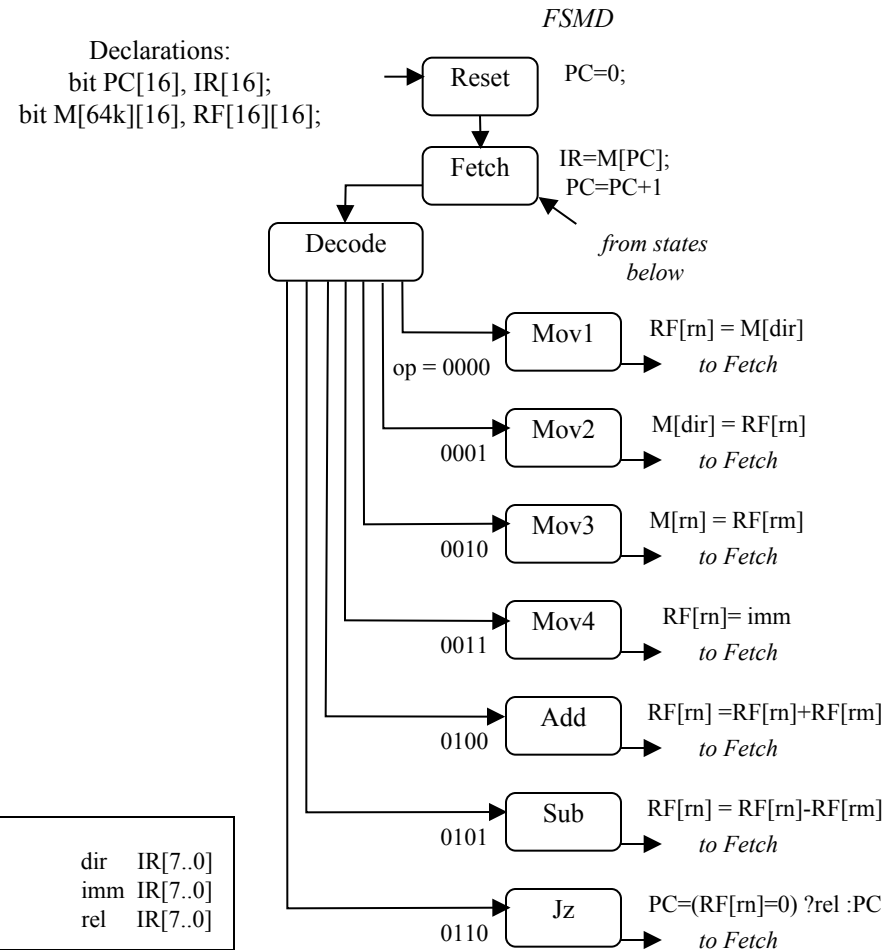
FSM operations that replace the FSM operations after a datapath is created



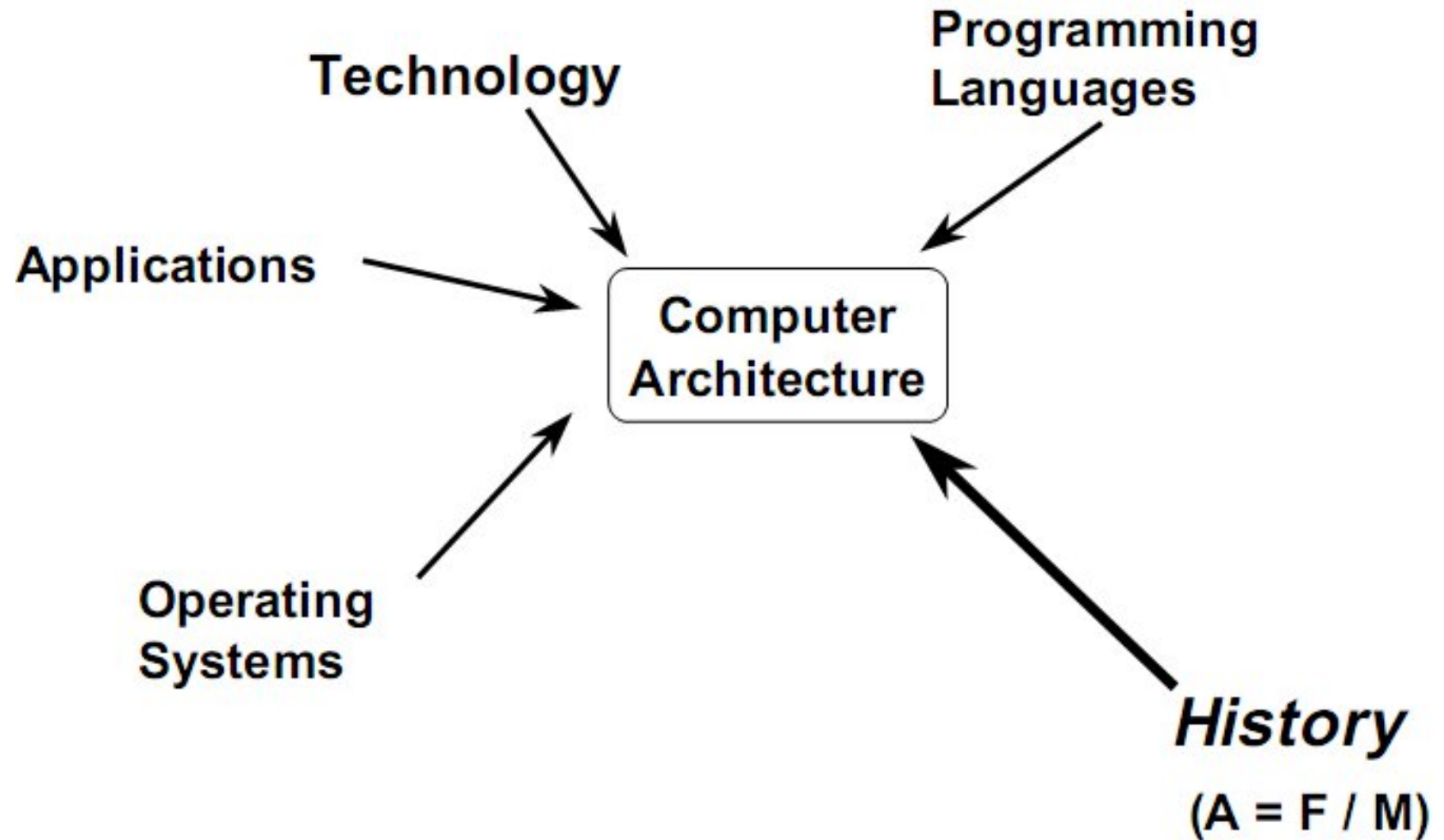
Designing a General Purpose Processor

- Not something an embedded system designer normally would do
 - But instructive to see how simply we can build one top down
 - Remember that real processors aren't usually built this way
 - Much more optimized, much more bottom-up design

Aliases:			
op	IR[15..12]	dir	IR[7..0]
rn	IR[11..8]	imm	IR[7..0]
rm	IR[7..4]	rel	IR[7..0]



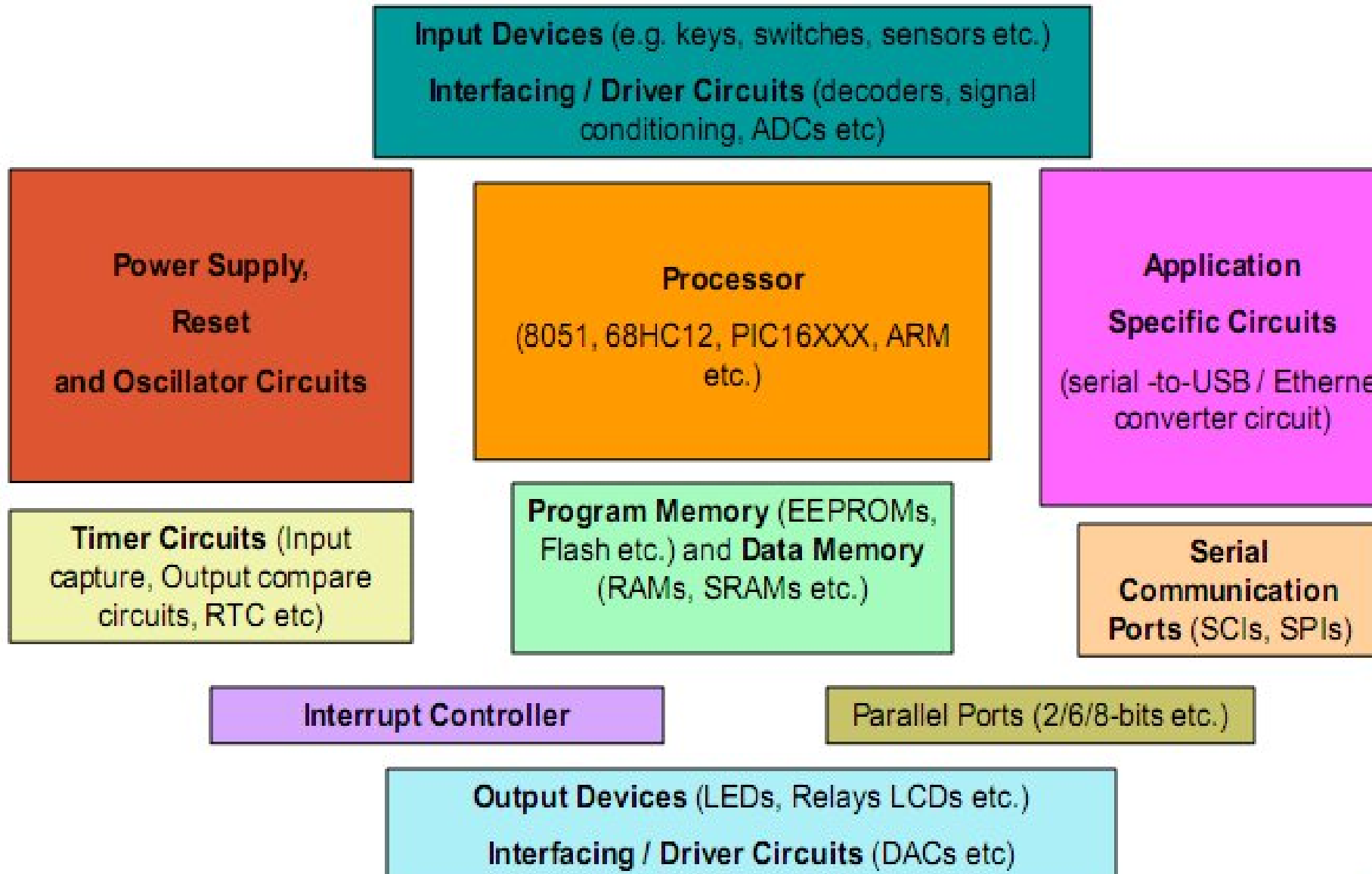
Forces on Computer Architecture



Key Technology Trends

- Semiconductors (processor & memory chips)
 - Moore's Law: # of devices per chip doubles every ~1.5 years
 - Device speed improves by <15% per year
 - Long on-chip wires get worse; voltages have stopped scaling
- Storage (hard disks)
 - Disk capacity increases by 100x per decade
 - Disk throughput improves by 10x per decade
 - Disk cost/bit is 100x better than that of DRAM
- Network
 - Gilder's law: deployed bandwidth triples every year
 - Link bandwidth improves 4x every 3 years
- Implications?

Components of Embedded Systems (Hardware)



Application Areas of Embedded Systems

Automatic chocolate vending machine	Anti-lock brakes	Auto-focus cameras	Automatic teller machines	Automatic toll-booth systems	Automatic transmission / reception	Avionic systems
Battery chargers	Camcorders	Cell phones	Cell-phone base stations	Cordless phones	Cruise control systems	Curbside check-in systems
Digital cameras	Disk drives	Data Acquisition systems	Electronic card / tape readers / drives	Electronic instruments	Electronic toys / games	Electronic Control Units (ECUs)
Factory control	Fax machines	Fingerprint identifiers		Home security systems	LCD TVs / Monitors	Life-support systems
Medical testing systems	Modems	MPEG decoders	Mobile Phones	Network cards	Network switches/ routers	Night vision cameras
On-board navigation		Pagers	Photocopiers	Point-of-sale systems	Portable video games	Printers
Robots	RFID Tags	Satellite phones	Scanners	Smart Ovens /dishwashers	Smart Rooms / Buildings	Speech recognizers
Stereo systems	Teleconferencing systems	Telemonitoring medical systems	Televisions	Temperature controllers	Theft tracking systems	
VCR's, DVD players	Video game consoles	Video phones		Washers and dryers	Wireless Devices	

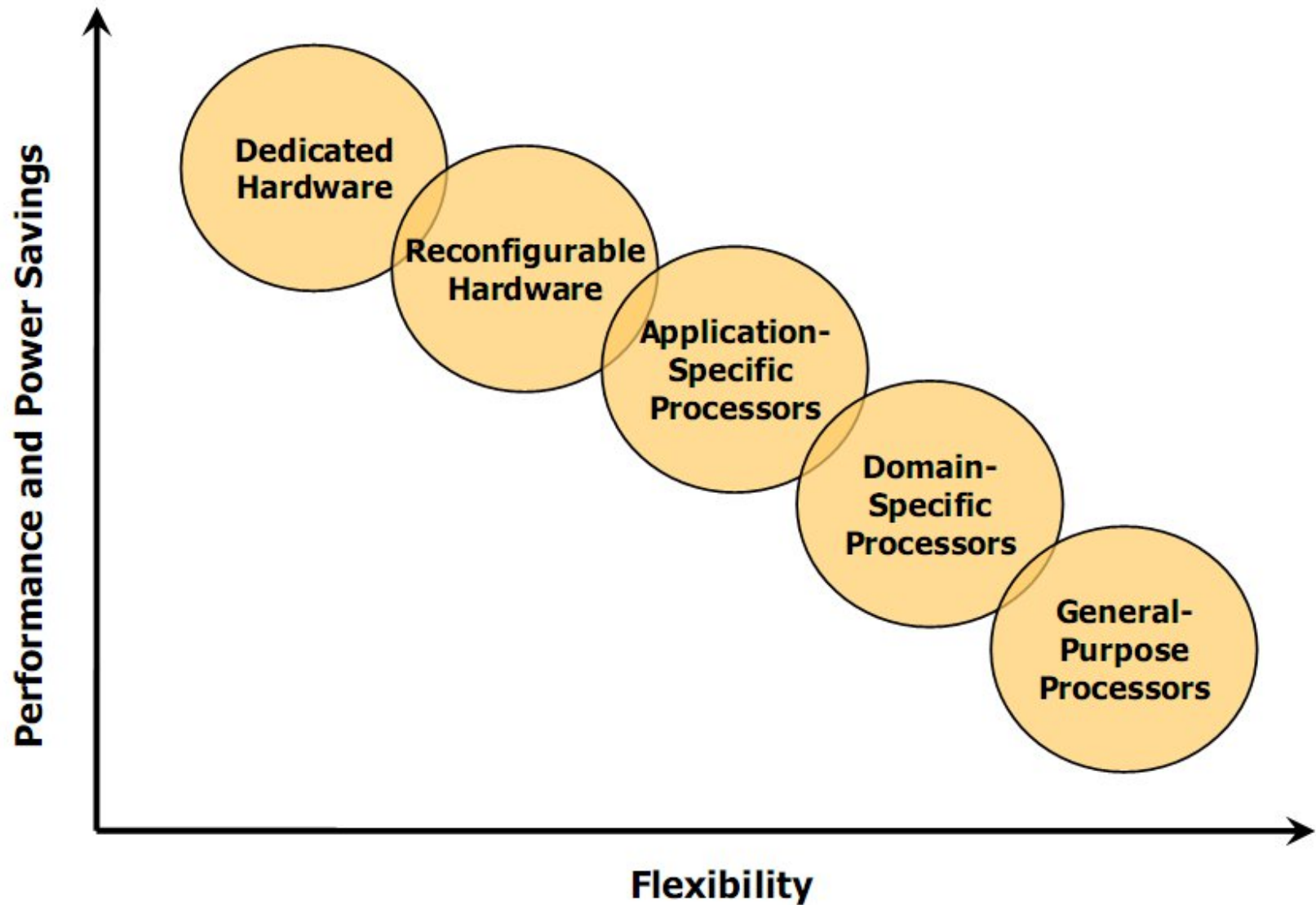
Introduction

- Embedded system functionality aspects
 - Processing
 - Transformation of data
 - Implemented using processors
 - Storage
 - Retention of data
 - Implemented using memory
 - Communication
 - Transfer of data between processors and memories
 - Implemented using buses
 - Called *interfacing*

Embedded Processor

- What is an embedded processor?
 - A programmable processor ‘embedded’ in an embedded system (on a chip)
 - An embedded processor can be a general purpose processor, DSP, microcontroller, or a processor optimized to a specific application.
 - Programming self-contained computers
 - End-user, application developer, system integrator, and component manufacturer are all separated.
 - Programming embedded processors
 - Most software is provided by system integrator.

Tradeoffs due to architectural choice



Hardware Units in Embedded Systems

- Power Source
- Clock Oscillator Circuit and Clocking Unit(s)
- Real Time Clock (RTC) and Timers for Various Timing and Counting Needs of the System
- Reset Circuit, Power-up Reset and Watchdog–Timer Reset
- Memories
- Input, Output and I/O Ports, I/O Buses and I/O Interfaces
- Interrupts Handler
- DAC (Using a PWM) and ADC
- LCD and LED Displays
- Keypad / Keyboard
- Pulse Dialer, Modem and Transceiver
- GPIB (IEEE 488) Link
- Linking and Interfacing Buses and Units of the Embedded System Hardware

Hardware tools

- Development boards, EVB
- Reference boards
- Platforms
- Bread boards
- Hardware debuggers, analyzers
- Emulators
- Etc...

Components of Embedded Systems (Software)

- Main application software:
 - performs series of tasks or multiple tasks concurrently.
 - constrained due to low memory, low processing power requirement etc.
- Real Time Operating System (RTOS):
 - supervises the application software.
 - provides a mechanism to let the processor run a process as per scheduling.
 - performs context-switching between various processes (tasks).
 - organizes access to a resource in sequence of the series of tasks of the system.
 - schedules their working and execution by following a plan to control the latencies and to meet the deadlines.
 - sets the rules during the execution of the application software.

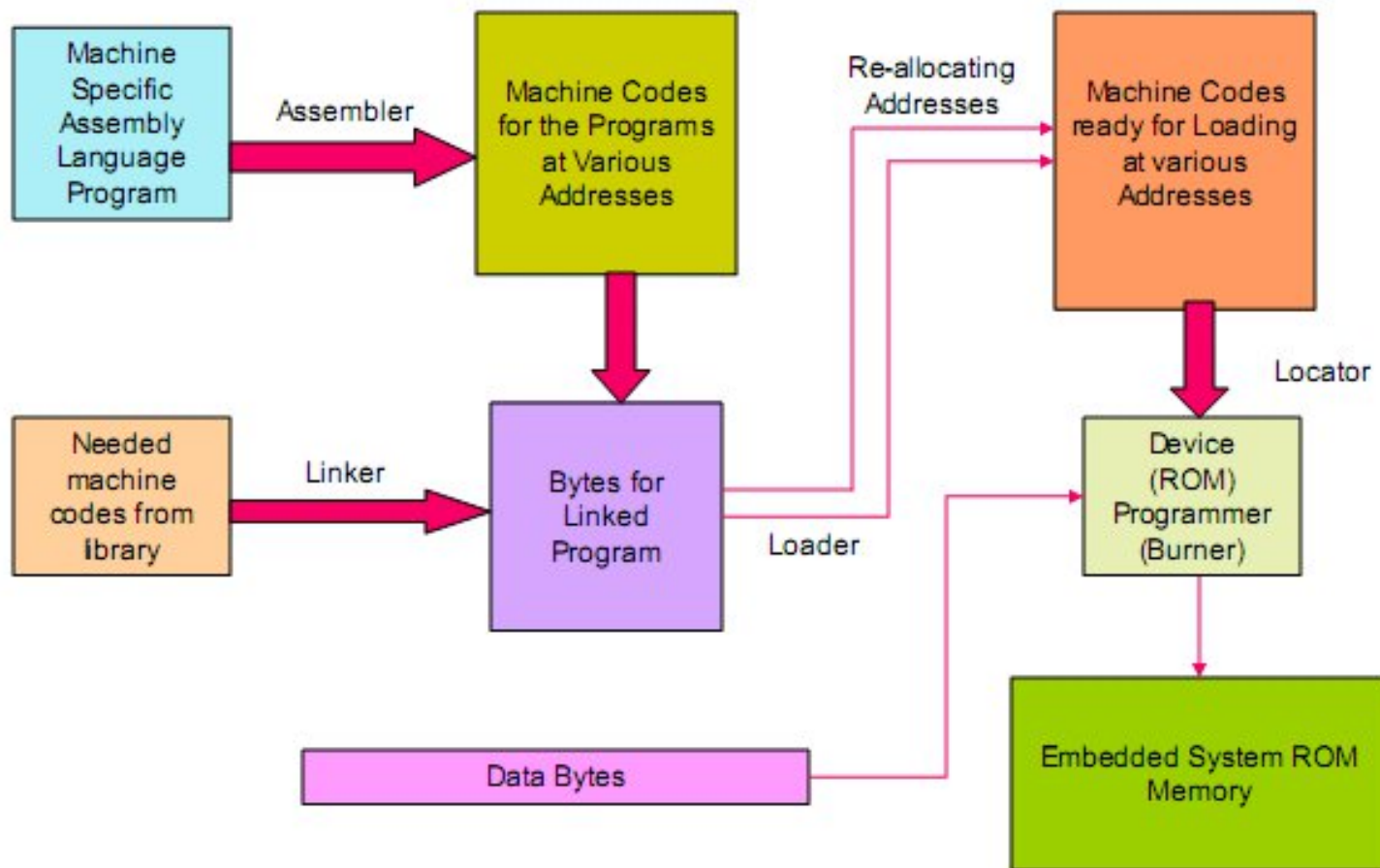
Software for Embedded Systems

- ❑ **Instructions:** A binary pattern designed inside a microprocessor to perform a specific function.
- ❑ **Instruction Set:** An entire group of instructions.
- ❑ **Mnemonic:** A combination of letters to suggest the operation of an instructions (Mnemonic = mindful)
- ❑ **Assembly Language Program:** A set of instructions written in mnemonics of a given microprocessor / microcontroller or its family such as 8085, 8086 etc.
- ❑ **Machine Language:** The binary medium of communication with a computer through a designed set of instructions.

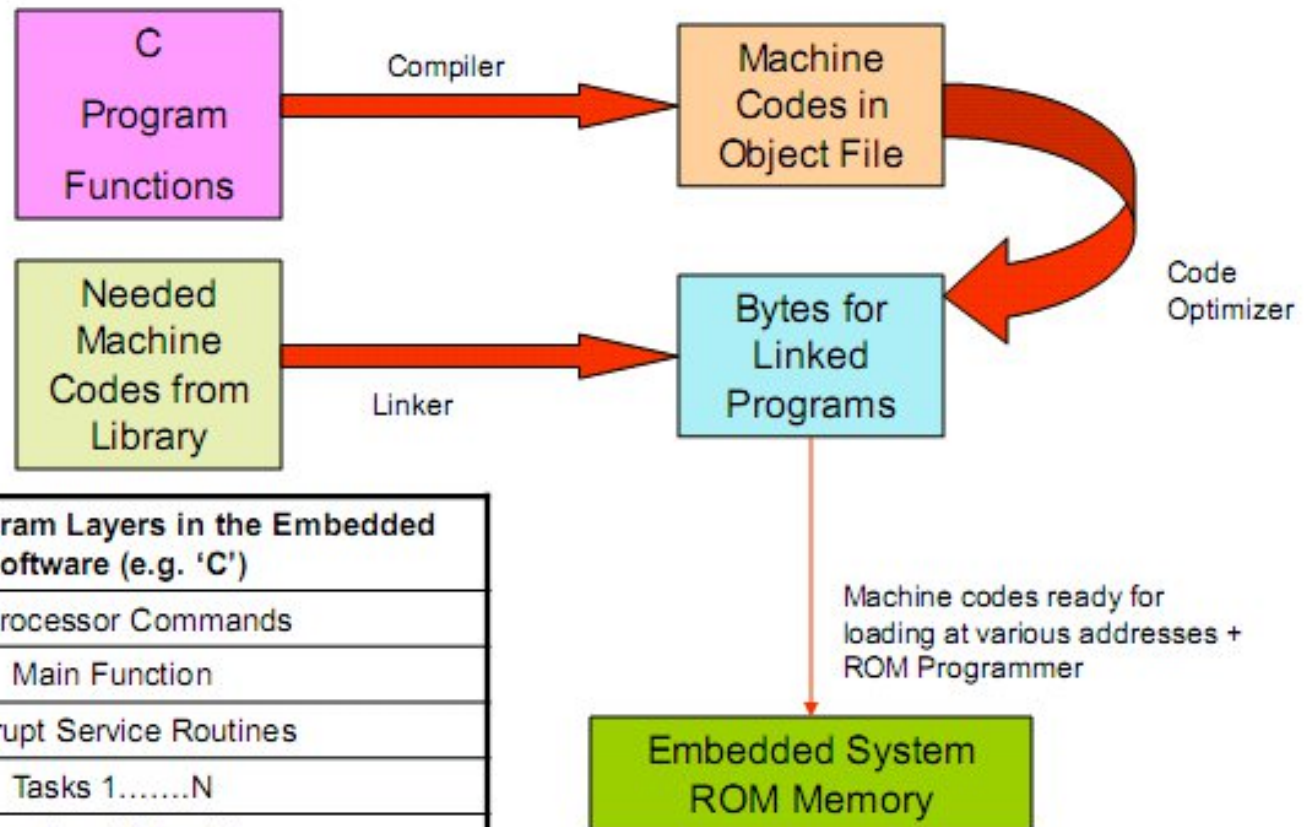
Software for Embedded Systems

- ❑ **Low-Level Language:** A medium of communication with a computer in which programs are written in mnemonics. An assembly language is specific to a given computer.
- ❑ **High-Level Language:** A medium of communication that is independent of a given computer. Programs are written in English-like words and they can be executed on machine using a translator (a compiler or Interpreter).
- ❑ **Compiler:** This is a program that translates English-like words, of a high-level language in the machine language of a computer.
- ❑ **Interpreter:** An interpreter is a program that translates the English-like statements of a high-level language into the machine language of a computer.
- ❑ **Assembler:** A computer program that translates an assembly language program from mnemonics to the binary machine code of a computer.
- ❑ **Linker:** links the machine codes generated by assembler with the other required assembled codes. Generates a file known as linked file or executable '.exe' file to run on a computer.
- ❑ **Loader:** reallocates the codes after finding the physical RAM addresses available at a given instant. It is a part of the OS and places codes into the memory after reading the '.exe' file.
- ❑ **Locator:** reallocates the linked file and creates a file for permanent location of codes.

From ALP to ROM Image



Software in High Level Language



Different Program Layers in the Embedded Software (e.g. 'C')

Preprocessor Commands

Main Function

Interrupt Service Routines

Tasks 1.....N

Kernel and Scheduler

Standard Library Functions Including Network Protocol Functions for Sending Stack and Receiving Stack

OS and RTOS

- ❑ **Operating System (OS)** allocate resources in the computing system among programs that request them. It also schedules processes to meet performance requirements and provide services beyond CPU scheduling.
- ❑ **Multitasking:** Processing codes for the different tasks as directed by the scheduler.
- ❑ **Kernel:** A program with functions for memory allocation and de-allocation, task scheduling, inter-process communication, effective management of shared memory access by using the signals, exception (error) handling signals, queues, mailboxes, pipes and sockets, I/O management, interrupts control, device drivers and device management.
- ❑ **Real Time Operating Systems (RTOS):** An OS for real-time programming and scheduling, process and memory manager, device drivers, device management and multitasking.
- ❑ RTOS has to be **scalable** in embedded systems.
- ❑ **Scalable OS** is one in which memory is optimized by having only that part of features that are needed and is associated with the final software.

Software Design using RTOS

- ❑ Each Task has a distinct memory location
- ❑ Tasks are made up of one or more procedures
- ❑ Shared Memory Architecture (Graphic chips doesn't have own dedicated memory, instead shares main system RAM with CPU and other components)
- ❑ Distributed memory Architecture – UMA/ NUMA
- ❑ Scheduling done by OS - takes care of constraints

- ❑ RTOS kernel schedules transitions of task
 - Ready state -> Running State
 - Block state -> Running State (pre-emptive scheduling)
 - Tasks can enter into ready state at any time
 - At a time only one task will be in running state

- ❑ RTOS also controls inter-process communication

Device Driver Concepts

Device Driver: a software for *controlling, receiving and sending* a byte or a stream of bytes from or to a device. Drivers control three functions

- (i) **Initializing** that is activated by placing appropriate bits at the control register or word.
- (ii) **Calling** an ISR on interrupt or on setting a status flag in the status register and run (drive) the ISR.
- (iii) **Resetting** the status flag after interrupt service. (e.g. device drivers for accessing a serial port, keyboard, mouse etc.)

Device Driver ISR: Consider a device driver for handling port inputs. It does the following:

- (i) Resets the device-buffer-full flag (in status register) and thus prepares the device for the next read. [The flag must be reset when the corresponding ISR starts execution.]
- (ii) Reads the input buffer (s) by emptying the buffer and storing the byte (s) in memory or using the bytes received as per system requirement.

Before driving the device must be configured. A device driver routine differs from other ISRs.

Timer Device Driver: Following steps are needed

- (i) Write in a register that holds the timer reset value, the number of count-inputs, *numTicks* for RTC
- (ii) Write in status register the timer status flag (s) = reset (in case the device does reset automatically this flag (s) on a read of the status flag (s)).
- (iii) Write each bit present in the control register (s). Write interrupt secondary and primary level enable bits = true in control register, write other bits according to their uses.

Software for Device Drivers

- ❑ **Virtual devices** are used for programming (file, pipe etc.). They are programmed for opening and closing and for reading and writing, such as a program for attaching and detaching a physical device and for input and output.
- ❑ **File:** A data structure which sends the records (characters or words) to a data sink (for e.g. a program structure) and which stores the data from the data source (for e.g. a program structure). A file in computer may also be stored at the hard disk.
- ❑ **Pipe:** A data structure which is sent as byte stream from a data source (for e.g. a program structure) and which delivers the byte stream to the data sink (for e.g. a program structure).
- ❑ Control registers, input data buffers, output data buffers and status registers form part of the device hardware.
- ❑ **Device Management Software:** manages multiple devices and drivers.

Software tools

- Editor
- Interpreter / Compiler
- Assembler / Cross-assembler
- Simulator
- RTOS
- Logic Analyzer
- Prototyper
- Locator
- EDA packages like Altium, Orcad
- IDE like KeilC, AVR Studio,
- Etc...

Emulators

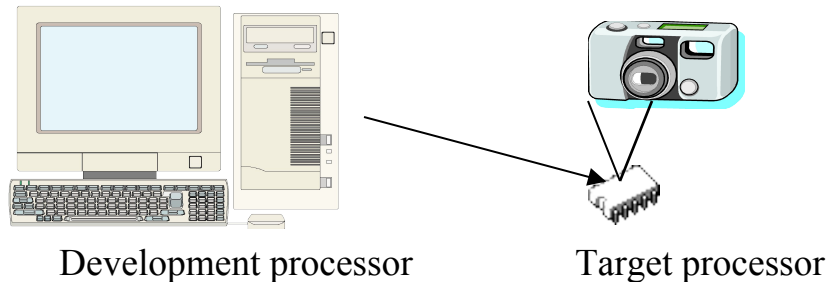
- General physical device system mapped to
 - Microprocessor emulator
 - Microprocessor IC with some monitoring, control circuitry
 - SPP emulator
 - FPGAs (10s to 100s)
 - Usually supports debugging tasks
- Created to help solve simulation disadvantages
 - Mapped relatively quickly
 - Hours, days
 - Can be placed in real environment
 - No environment setup time
 - No incomplete environment
 - Typically faster than simulation
 - Hardware implementation

Disadvantages

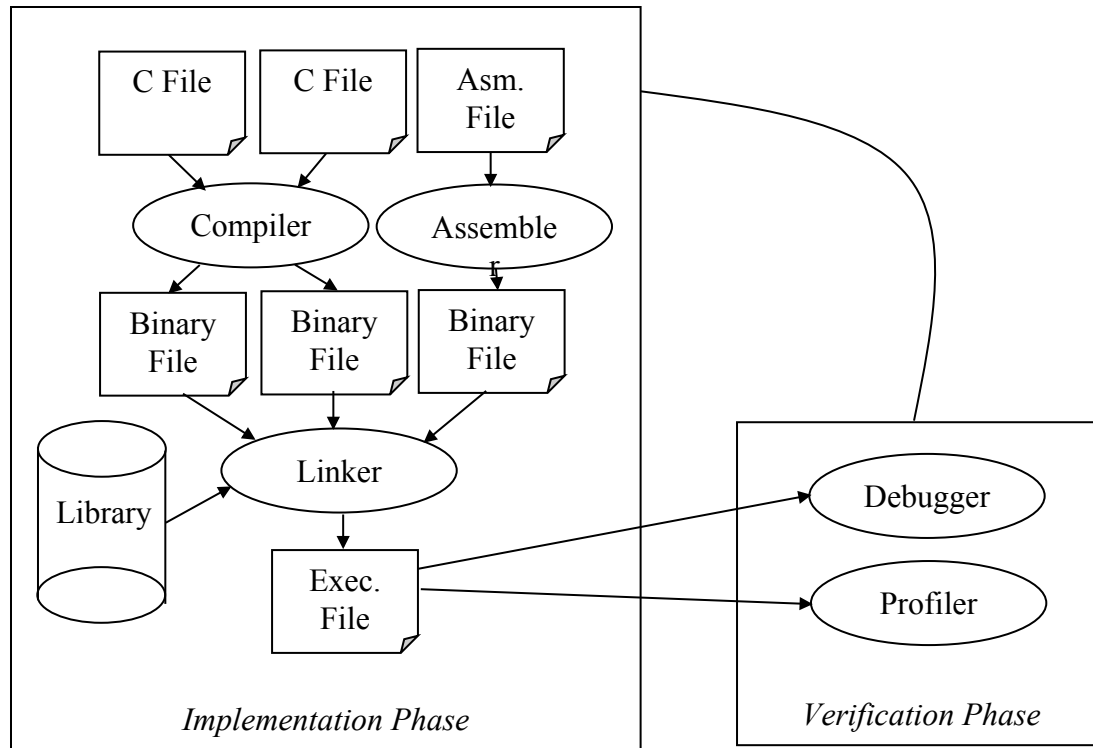
- Still not as fast as real implementations
 - E.g., emulated cruise-control may not respond fast enough to keep control of car
- Mapping still time consuming
 - E.g., mapping complex SOC to 10 FPGAs
 - Just partitioning into 10 parts could take weeks
- Can be very expensive
 - Top-of-the-line FPGA-based emulator: \$100,000 to \$1mill
 - Leads to resource bottleneck
 - Can maybe only afford 1 emulator
 - Groups wait days, weeks for other group to finish using

Development Environment

- Development processor
 - The processor on which we write and debug our programs
 - Usually a PC
- *Target processor*
 - The processor that the program will run on in our embedded system
 - Often different from the development processor



Software Development Process

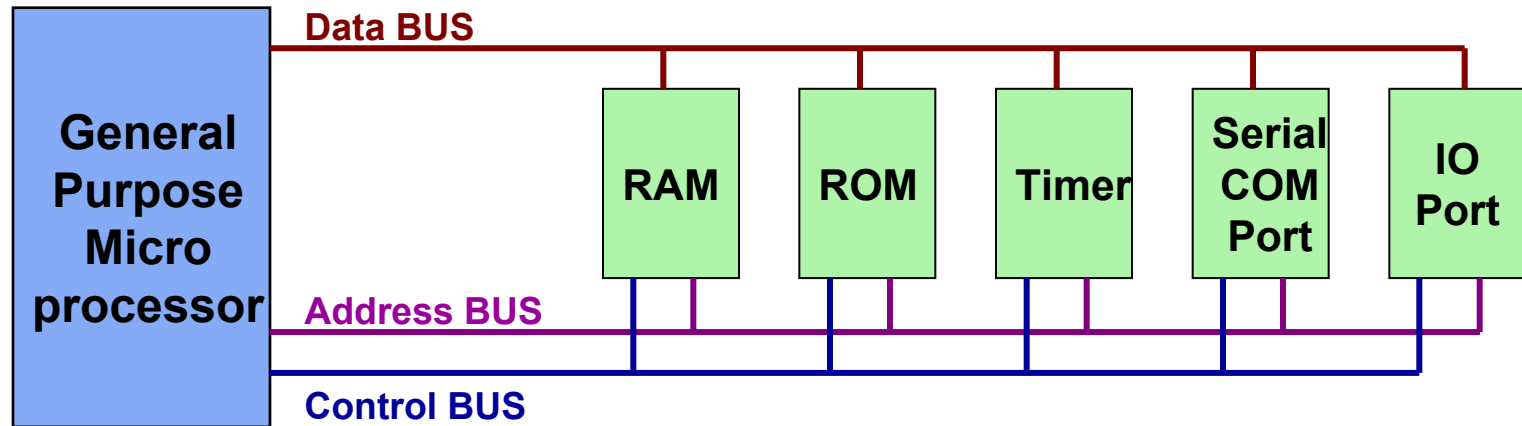


- Compilers
 - Cross compiler
 - Runs on one processor, but generates code for another
- Assemblers
- Linkers
- Debuggers
- Profilers

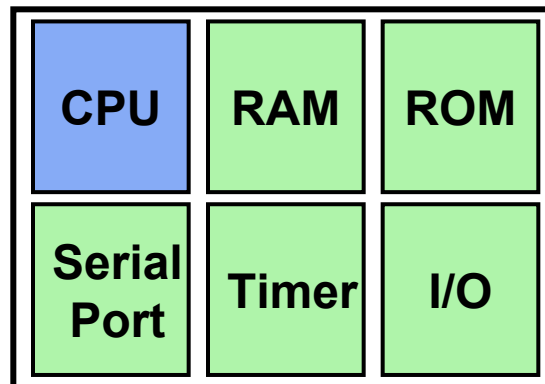
uP and MCU

Microprocessors vs. Microcontrollers

- General Purpose Microprocessors



- Microcontrollers



Microprocessor

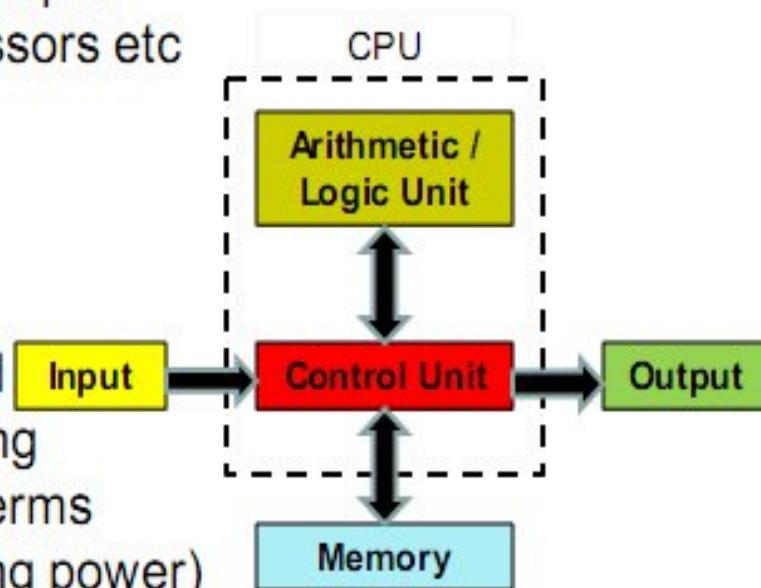
What is a Microprocessor ?

A microprocessor is a multipurpose, clock-driven, register-based electronic device that reads binary instructions from a storage device called memory, accepts binary data as input and processes data according to the instructions given and provides results as output.

Ex: 8085, 8086, Z80, 6800, Pentium processors etc

Why do we use microprocessors ?

- ❑ efficiently implements digital systems (executes programs very efficiently)
- ❑ easier to design families of products and can be extended to meet rapidly changing market demands. (optimized design in terms of achieving greater speed i.e. processing power)





Microprocessor (Cont...)

Instruction set includes instructions for performing	Other functional units besides CPU	Embedded Systems incorporated in GHz processors
- Data Transfer Operations - ALU Operations - Stack Operations - Input and Output (I/O) Operations - Program Control Operations - Sequencing Operations - Supervising Operations	- cache memories - floating point processing arithmetic unit - pipelining unit - super-scaling unit	- Gbps transceiver - Encryption engine - Graphic accelerator - Disk controllers - Network Interface Cards (NICs)
A microprocessor is used when large embedded software is to be located in the external memory chips. RISC core microprocessors are used when intensive computations are to be performed as it provides speedy processing of instructions (in a single clock-cycle)		

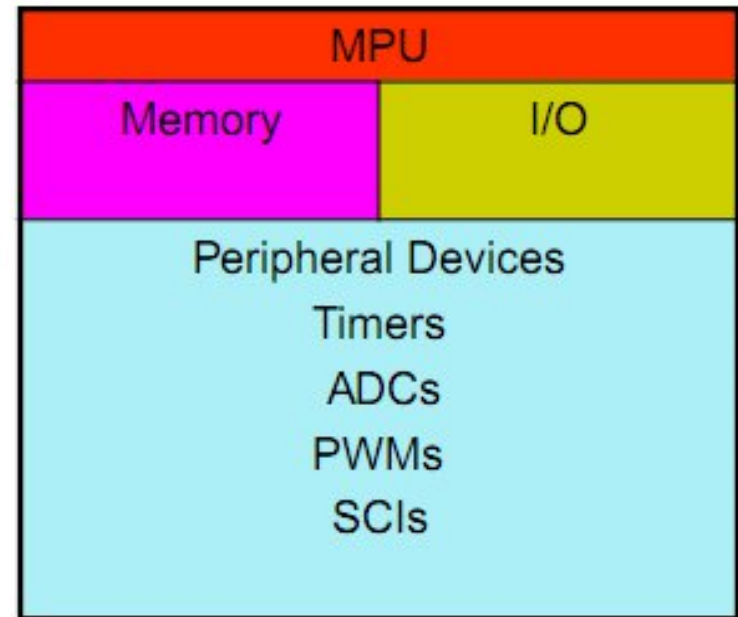
Microcontroller

What is a Microcontroller?

A microcontroller is essentially an entire computer on a single chip.

Ex: Intel's 8051, 8096, Motorola M68HC11 XX / M68HC12XX, PIC 16XX series etc.

Most essential component of a control or communication unit



Commonly used microcontrollers

Small scale embedded systems	Medium scale embedded systems	Large scale embedded systems
- 68HC05 / 08 - PIC 16F8X - 8051	- 8051 / 251 - 80x86 - 68HC11XX / 12XX - 80196	- Intel 80960CA - ARM 7 - Power PC MPC 604

Microcontroller (Cont...)

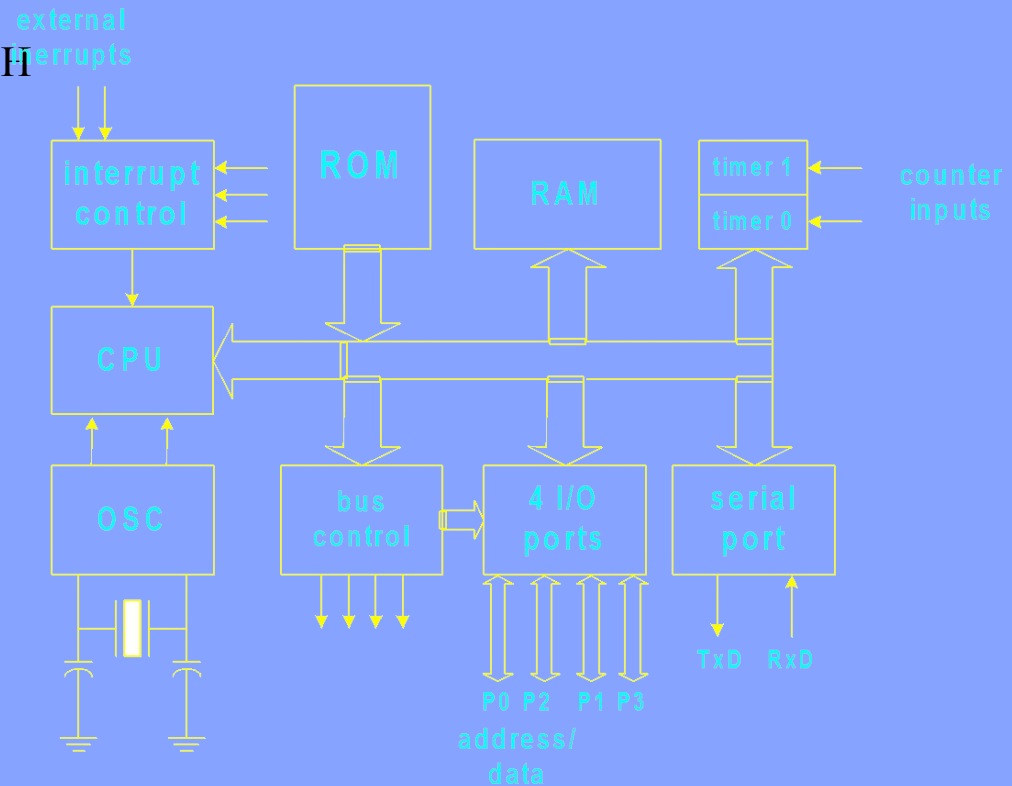
Functional circuits in a chip or core of a microcontroller

- Processor - Data and Stack in internal RAM - Timers and Watch dog timers - External Memories Interface - Non volatile PROM / ROM / EPROM - Interrupt controller - I/O ports control and Interface / Drivers - Serial UART communication port - Serial synchronous communication port	Application Specific Circuits in Specific Versions
	--DMA Controllers -- Network Driver Stack and Interface -- A/D Converter -- LAN controller -- PWM circuit for D/A -- Printer Controller -- Modem -- DTMF circuit

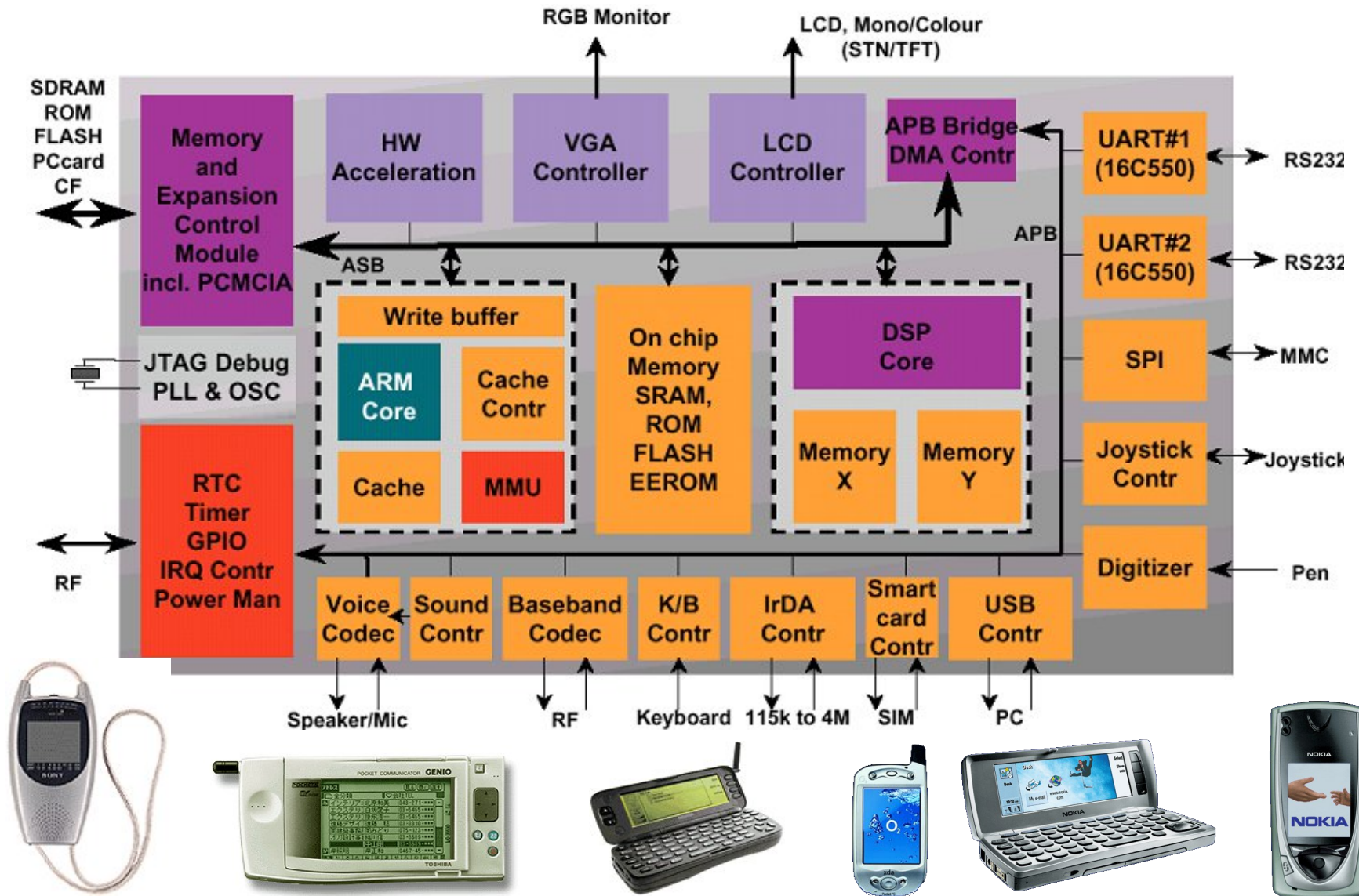
A **microcontroller** is used when a **small part of the embedded software** has to be **located in internal memory** and when the **on-chip functional units** like interrupt handler, port, timer, ADC and PWM are **needed**.

Microcontrollers

- Basic parts are:
 - Central Processing Unit
 - RAM
 - EPROM/PROM/ROM or FLASH Memory
 - I/O serial or/and parallel
 - timers
 - interrupt controller
- Optional parts are:
 - Watch Dog Timer
 - AD Converter
 - LCD driver
 - etc.



Advanced MicroController



Microprocessors and Microcontrollers

Microprocessor

- Data is in byte (8-bit) or word (16-bit) format
- Many instruction types and modes
- Hardware includes CPU only
- Mainly for processing
- Larger I/O scale – use system buses
- Memory is byte-oriented
- Instruction sets are more robust

Microcontroller

- Data is in bits or byte format
- Less instruction types and modes
- Hardware includes CPU and peripherals
- Mainly for control
- Smaller I/O scale – use parallel and serial ports
- Memory is bit-oriented
- Instruction sets are more compact

Other Processors (FPGA, PSoC)

	Embedded Processor (for a complex system)	Digital Signal Processor (DSP) (Microprocessor + MAC units)
Used for	Fast, precise and intensive calculations	Processing of signals
Application area	Complex real time applications for e.g. in an electronic warfare system's advanced warning and control systems (AWCS)	Image processing, multimedia, audio, video, HDTV, DSP modem and telecommunication processing systems Also for recognizing an image pattern or a DNA sequence fast
Features	-Fast context switching (lower latencies of the tasks in complex real time applications) -Atomic ALU operations (no shared data problem) -RISC core for fast, more precise and intensive calculations by the embedded s/w	- A typical DSP has 16x32 MAC unit - Provides fast, discrete-time, signal processing instructions - Very Large Instruction Word (VLIW) processing capabilities - Single Instruction Multiple Data (SIMD) instructions - Discrete Cosine Transformations (DCT) and inverse DCT (IDCT)
examples	ARM family: ARM 7 and ARM 9 (available in single chip as well as file version for embedding on a VLSI chip or SOC) Intel family: i960 AMD family: 29050	TMS 320Cxx (Texas Ins.) SHARC (Analog Devices) 5600xx (Motorola)

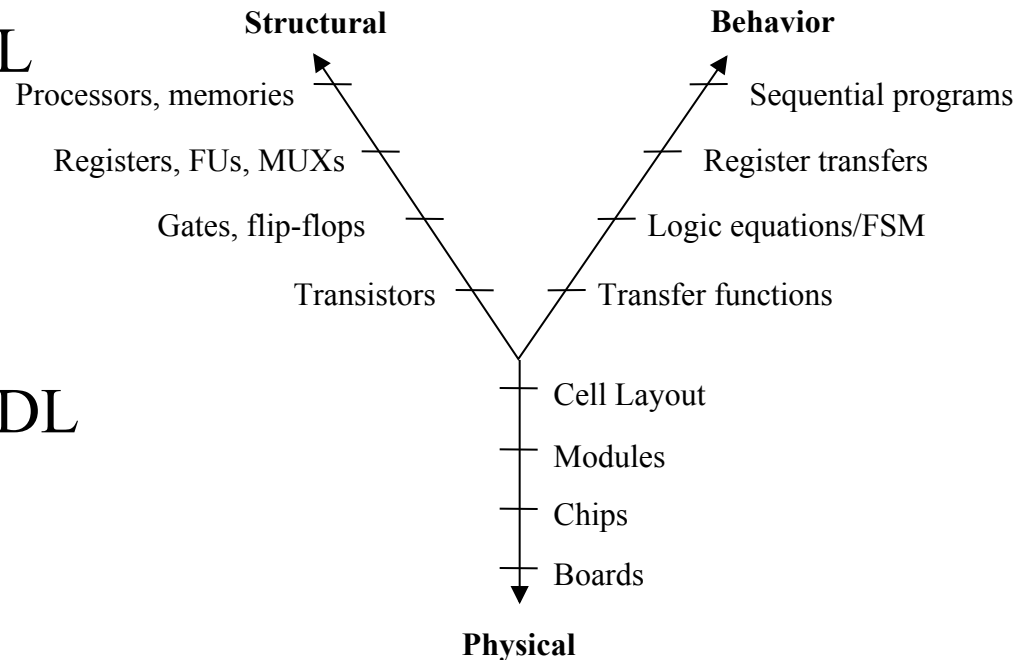
Reuse: intellectual property cores

- Commercial off-the-shelf (COTS) components
 - Predesigned, prepackaged ICs
 - Implements GPP or SPP
 - Reduces design/debug time
 - Have always been available
- System-on-a-chip (SOC)
 - All components of system implemented on single chip
 - Made possible by increasing IC capacities
 - Changing the way COTS components sold
 - As intellectual property (IP) rather than actual IC
 - Behavioral, structural, or physical descriptions
 - Processor-level components known as cores
 - SOC built by integrating multiple descriptions

Cores

- Soft core
 - Synthesizable behavioral description
 - Typically written in HDL (VHDL/Verilog)
- Firm core
 - Structural description
 - Typically provided in HDL
- Hard core
 - Physical description
 - Provided in variety of physical layout file formats

Gajski's Y-chart



Advantages/disadvantages of hard core

- Ease of use
 - Developer already designed and tested core
 - Can use right away
 - Can expect to work correctly
- Predictability
 - Size, power, performance predicted accurately
- Not easily mapped (retargeted) to different process
 - E.g., core available for vendor X's 0.25 micrometer CMOS process
 - Can't use with vendor X's 0.18 micrometer process
 - Can't use with vendor Y

Advantages/disadvantages of soft/firm cores

- Soft cores
 - Can be synthesized to nearly any technology
 - Can optimize for particular use
 - E.g., delete unused portion of core
 - Lower power, smaller designs
 - Requires more design effort
 - May not work in technology not tested for
 - Not as optimized as hard core for same processor
- Firm cores
 - Compromise between hard and soft cores
 - Some retargetability
 - Limited optimization
 - Better predictability/ease of use

Most common microcontrollers

- 8-bit microcontrollers
 - AVR
 - PIC
 - 68HC
 - 8051
 - 32-bit microcontrollers
 - ARM
 - PIC32
-

Main Difference between AVR, ARM, 8051 and PIC Microcontrollers

	8051	PIC	AVR	ARM
Bus width	8-bit for standard core	8/16/32-bit	8/32-bit	32-bit mostly also available in 64-bit
Communication Protocols	UART, USART, SPI, I2C	PIC, UART, USART, LIN, CAN, Ethernet, SPI, I2S	UART, USART, SPI, I2C, (special purpose AVR support CAN, USB, Ethernet)	UART, USART, LIN, I2C, SPI, CAN, USB, Ethernet, I2S, DSP, SAI (serial audio interface), IrDA
Speed	12 Clock/instruction cycle	4 Clock/instruction cycle	1 clock/ instruction cycle	1 clock/ instruction cycle
Memory	ROM, SRAM, FLASH	SRAM, FLASH	Flash, SRAM, EEPROM	Flash, SDRAM, EEPROM
ISA	CISC	Some feature of RISC	RISC	RISC
Memory Architecture	Von Neumann architecture	Harvard architecture	Modified	Modified Harvard architecture
Power Consumption	Average	Low	Low	Low
Families	8051 variants	PIC16, PIC17, PIC18, PIC24, PIC32	Tiny, Atmega, Xmega, special purpose AVR	ARMv4,5,6,7 and series
Community	Vast	Very Good	Very Good	Vast
Manufacturer	NXP, Atmel, Silicon Labs, Dallas, Cyprus, Infineon, etc.	Microchip Average	Atmel	Apple, Nvidia, Qualcomm, Samsung Electronics, and TI etc.
Cost (as compared to features provide)	Very Low	Average	Average	Low
Other Feature	Known for its Standard	Cheap	Cheap, effective	High speed operation Vast
Popular Microcontrollers	AT89C51, P89v51, etc.	PIC18fXX8, PIC16f88X, PIC32MX	Atmega8, 16, 32, Arduino Community	LPC2148, ARM Cortex-M0 to ARM Cortex-M7, etc.

	STM32L0	PIC32MX	SAMD21	EFM32Z	LPC812M	MSP430F	ATmega-328P
Vendor	ST	Microchip	Atmel	Silicon Labs	NXP	TI	Atmel
Arch.	ARM M0+	MIPS M4K	ARM M0+	ARM M0+	ARM M0+	16-bit RISC	megaAVR 8-bit
IDE	Arduino, IAR, Keil	MPLAB X IDE	Arduino, Atmel Studio	Simplicity Studio	LPCXpresso	Energia, CCS, IAR	Arduino, Atmel Studio
Max Freq.	32 MHz	80 MHz	48 MHz	24 MHz	30 MHz	16 MHz	20 MHz
Power	240 μ A/MHz @ 4 MHz	700 μ A/MHz @ 80 MHz	179 μ A/MHz @ 48 MHz	115 μ A/MHz @ 24 MHz	110 μ A/MHz @ 30 MHz	120 μ A/MHz @ 1 MHz	700 μ A/MHz @ 8 MHz
SRAM	8 KB	32 KB	32 KB	4 KB	1 KB	0.5 KB	2 KB
Comms	SPI, I2C, JTAG	SPI, I2C, JTAG	SPI, I2C, SDIO, JTAG	SPI, I2C, SDIO, JTAG	SPI, I2C, SDIO, JTAG	SPI, I2C, JTAG	SPI, I2C, JTAG
ADC	12-bit 1.1 MS/s	10-bit 500 KS/s	12-bit 350 KS/s	None	None	10-bit 200 KS/s	10-bit 77 KS/s
DAC	12-bit	None	12-bit	12-bit	None	None	None
PWM	4 x 16-bit	5 x 16- or 32-bit	8 x 16-bit	6 x 16-bit	2 x 16-bit or 1 x 32-bit	3 x 16-bit	6 x 8-bit
GPIO	11-84	53-85	26-52	17-37	6-18	12-83	23

	8051	AT 89S52	68HC11	Atmega 328	PIC 16F84A	PIC 18F452	ARM7-TDMI-S LPC2148	ARM Cortex-M3 LPC1768
CPU	8-bit	8-bit	8-bit	8-bit	8-bit	8-bit	16-bit/ 32-bit	16-bit/ 32-bit
Data Bus / Instr bus	8-bit	8-bit	8-bit	8-bit	8-bit / 14-bit	8-bit / 16-bit	32-bit / 32-bit	32-bit / 32-bit
Address bus	16-bit	16-bit	16-bit	16-bit	14-bit	16-bit	32-bit	32-bit
Clock Speed	12 MHz	0-33 MHz	0-4 MHz	20 MHz	20 MHz	0-40 MHz	10-25 MHz	0-100 MHz
Memory								
Program Memory	Int - 4 KB Extl - 64 KB	8 KB (flash)	8 KB	32 KB	14-bits 1 KB	32 KB	512 kB	512 kB
Data Memory	128 B (64 KB ext)	256 B	256 B	2 KB	RAM - 68B EEPROM-64B	RAM-1536 B EEPROM-256B	32 KB + 8 KB	Up to 64 KB
Interrupts	5	8	18	2	4	18	22	
DMA Channels	No	No	No	No	No	No	One (for USB)	8 Channel Gen purpose
I/O ports	4 x 8-bit 32 I/O lines	4 x 8-bit 32 I/O	40 I/O lines	23/28/32 N. Mathivanan	13	4 x 8-bit 1 x 3-bit	45 pins	70 / 52

Peripherals

	8051	AT 89S52	68HC11	Atmega 328	PIC 16F84A	PIC 18F452	ARM7-TDMI-S LPC2148	ARM Cortex-M3 LPC1768
ADC	No	No	One 8 channel 8-bit	One 8/6 Ch 10-bit	No	One 8 Channel 10-bit	Two 6+8 Chs, 10-bit	One 8 Chs, 12-bit
DAC	No	No	No	No	No	No	One 10-bit	One 10-bit
Timers	2 x 16-bit	3x16-bit	16-bit 3 Cap 5 Comp	2x8-bit, 1x16-bit, comp, cap	One 8-bit	4 8-bit / 16-bit	2 x 32-bit each with 4 Cap/Mat Chs	4
PWM Channels	No	No	8-bit-4 Ch, 16-bit-2Ch	6	No	Cap/comp PWM - 2	6	1 + 6
RTC	No	No	No	Yes	No	No	Yes	Yes
WDT	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes

Other Features								
	8051	AT 89S52	68HC11	Atmega 328	PIC 16F84A	PIC 18F452	ARM7-TDMI-S LPC2148	ARM Cortex-M3 LPC1768
Power Saving Modes	No	2 modes	2 modes, Wait & stop	6 modes	Yes	Yes	Yes Two	Yes Two
No. of Pins, Package	40/44/44 DIP/PLCC/ QFP	40/44/44 DIP/PLCC/ TQFP	48/52 DIP/PLCC	28/32/32p DIP/MLF/ TQFP	18/18/20 PDIP/SOIC/ SSOP	40/44/44 PDIP/TQFP/ PLCC	64 LQFP	100 LQFP 100 TFBGA 100WLCSP
No. of Instr.	255	?	109	131	35	75	?	?
CISC / RISC	CISC	CISC	CISC	RISC	RISC	RISC	RISC	RISC
Architecture	Von Neumann	Von Neumann	Von Neumann	----	Harvard (modified)	Harvard (modified)	Von Neumann	Harvard
Pipelining	-----	----	----	----	2 stage	2 stage	3 stage	3 stage
Special Features	-----	----	----	Multiplier, Analog Compar	----	Multiplier	Hardware Multiplier	Ethernet, Memory Management, QEI

Feature	8031	8051	8751	8032	8052	8752
Program memory	None ROM less	4K ROM	4K EPROM	None ROM less	8K ROM	8K EPROM
Data memory	128 Bytes	128 Bytes	128 RAM	256 Bytes	256 Bytes	256 Bytes
Timers/ Counters (16-bit)	2	2	2	3	3	3
I/O pins	32	32	32	32	32	32
Serial port	1	1	1	1	1	1
Interrupt sources (Reset not included)	5	5	5	6	6	6

	Baseline Architecture	Mid-Range Architecture	Enhanced Mid-Range Architecture	PIC18 Architecture
Pin Count	6-40	8-64	8-64	18-100
Interrupts	No	Single interrupt capability	Single interrupt capability with hardware context save	Multiple interrupt capability with hardware context save
Performance	5 MIPS	5 MIPS	8 MIPS	Up to 16 MIPS
Instructions	33, 12-bit	35, 14-bit	49, 14-bit	83, 16-bit
Program Memory	Up to 3 KB	Up to 14 KB	Up to 28 KB	Up to 128 KB
Data Memory	Up to 138 Bytes	Up to 368 Bytes	Up to 1,5 KB	Up to 4 KB
Hardware Stack	2 level	8 level	16 level	32 level
Features	• Comparator • 8-bit ADC • Data Memory • Internal Oscillator	In addition to Baseline: • SPI/I²C™ • UART • PWMs • LCD • 10-bit ADC • Op Amp	In addition to Mid-Range: • Multiple Communication Peripherals • Linear Programming Space • PWMs with Independent Time Base	In addition to Enhanced Mid-Range: • 8x8 Hardware Multiplier • CAN • CTMU • USB • Ethernet • 12-bit ADC
Highlights	Lowest cost in the smallest form factor	Optimal cost to performance ratio	Cost effective with more performance and memory	High performance, optimized for C programming, advanced peripherals
Total Number of Devices	16	58	29	193
Families	PIC10, PIC12, PIC16	PIC12, PIC16	PIC12FXXX, PIC16F1XX	PIC18

Microchip 8-bit PIC MCU architectures



* ADC with MATHPAK ** In addition to standard 8-bit and 16-bit timers

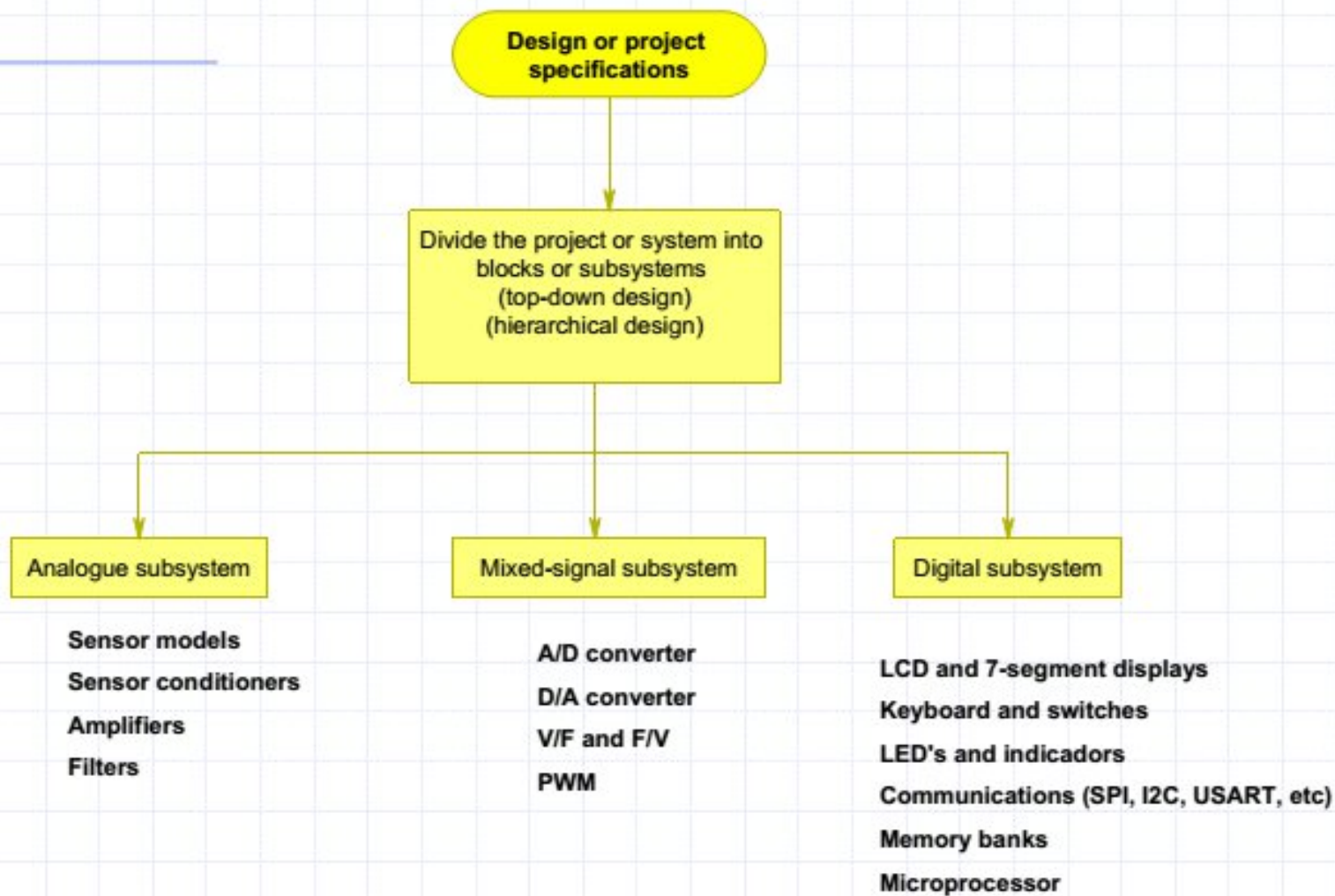
4.2 Product Selection Guide

	FLASH (KB)	EEPROM(Bytes)	RAM(Bytes)	I/O Pins	SPI	UART	I2C	USI	Hardware Multiplier	8-bit Timer	16-bit Timer	10-bit A/D Channels	Brown Out Detector	In System Prog. (I) + Self-Prog.(S)	Vcc(V)	Clock speed(MHz)	Packages (Pin)	Production
ATtiny11L	1	-	-	6	-	-	-	-	-	1	-	-	-	-	2.7-5.5	0-2	8 PDIP,SOIC	now
ATtiny11	1	-	-	6	-	-	-	-	-	1	-	-	-	-	4.0-5.5	0-6	8 PDIP,SOIC	now
ATtiny12V	1	64	-	6	-	-	-	-	-	1	-	-	Y	I	1.8-5.5	0-1	8 PDIP,SOIC	now
ATtiny12L	1	64	-	6	-	-	-	-	-	1	-	-	Y	I	2.7-5.5	0-4	8 PDIP,SOIC	now
ATtiny12	1	64	-	6	-	-	-	-	-	1	-	-	Y	I	4.0-5.5	0-8	8 PDIP,SOIC	now
ATtiny13	1	64	64	6	-	-	-	-	-	1	-	4	Y	I	1.8-5.5	0-16	8 PDIP,SOIC	Q4, 03
ATtiny15L	1	64	-	6	-	-	-	-	-	2	-	4	Y	I	2.7-5.5	1.6	8 PDIP,SOIC	now
ATtiny26L	2	128	128	16	-	-	-	1	-	2	-	11	Y	I	2.7-5.5	0-8	20 PDIP,SOIC, 32 MLF	now
ATtiny26	2	128	128	16	-	-	-	1	-	2	-	11	Y	I	4.5-5.5	0-16	20 PDIP,SOIC, 32 MLF	now
ATtiny28V	2	-	-	20	-	-	-	-	-	1	-	-	-	-	1.8-5.5	0-1	28 PDIP, 32 TQFP, MLF	now
ATtiny28L	2	-	-	20	-	-	-	-	-	1	-	-	-	-	2.7-5.5	0-4	28 PDIP, 32 TQFP, MLF	now
AT90S1200	1	64	-	15	-	-	-	-	-	1	-	-	-	I	2.7-6.0	0-4	20 PDIP,SOIC,SSOP	now
AT90S1200	1	64	-	15	-	-	-	-	-	1	-	-	-	I	4.0-6.0	0-12	20 PDIP,SOIC,SSOP	now
AT90S2313	2	128	128	15	-	1	-	-	-	1	1	-	-	I	2.7-6.0	0-4	20 PDIP,SOIC	now
AT90S2313	2	128	128	15	-	1	-	-	-	1	1	-	-	I	4.0-6.0	0-10	20 PDIP,SOIC	now
AT90LS2323	2	128	128	3	-	-	-	-	-	1	-	-	-	I	2.7-6.0	0-4	8 PDIP,SOIC	now
AT90S2323	2	128	128	3	-	-	-	-	-	1	-	-	-	I	4.0-6.0	0-10	8 PDIP,SOIC	now
AT90LS2343	2	128	128	5	-	-	-	-	-	1	-	-	-	I	2.7-6.0	0-1	8 PDIP,SOIC	now
AT90LS2343	2	128	128	5	-	-	-	-	-	1	-	-	-	I	2.7-6.0	0-4	8 PDIP,SOIC	now
AT90S2343	2	128	128	5	-	-	-	-	-	1	-	-	-	I	4.0-6.0	0-10	8 PDIP,SOIC	now
ATmega8L	8	512	1K	23	1	1	1	-	Y	2	1	8	Y	S	2.7-5.5	0-8	28 PDIP, 32 TQFP, MLF	now
ATmega8	8	512	1K	23	1	1	1	-	Y	2	1	8	Y	S	4.5-5.5	0-16	28 PDIP, 32 TQFP, MLF	now
ATmega8515L	8	512	512	35	1	1	-	-	Y	1	1	-	Y	S	2.7-5.5	0-8	40 PDIP, 44 PLCC, TQFP, MLF	now
ATmega8515	8	512	512	35	1	1	-	-	Y	1	1	-	Y	S	4.5-5.5	0-16	40 PDIP, 44 PLCC, TQFP, MLF	now
ATmega8535L	8	512	512	32	1	1	1	-	Y	2	1	8	Y	S	2.7-5.5	0-8	40 PDIP, 44 PLCC, TQFP, MLF	now
ATmega8535	8	512	512	32	1	1	1	-	Y	2	1	8	Y	S	4.5-5.5	0-16	40 PDIP, 44 PLCC, TQFP, MLF	now
ATmega162V	16	512	1K	35	1	2	-	-	Y	2	2	-	Y	S	1.8-5.5	0-8	40 PDIP, 44 TQFP, MLF	now
ATmega162	16	512	1K	35	1	2	-	-	Y	2	2	-	Y	S	2.7-5.5	0-16	40 PDIP, 44 TQFP, MLF	now
ATmega16L	16	512	1K	32	1	1	1	-	Y	2	1	8	Y	S	2.7-5.5	0-8	40 PDIP, 44 TQFP, MLF	now
ATmega16	16	512	1K	32	1	1	1	-	Y	2	1	8	Y	S	4.5-5.5	0-16	40 PDIP, 44 TQFP, MLF	now
ATmega169V	16	512	1K	54	1	1	-	1	Y	2	1	8	Y	S	1.8-3.6	0-1	64 TQFP, MLF	Q3,03
ATmega169L	16	512	1K	54	1	1	-	1	Y	2	1	8	Y	S	2.7-3.6	0-8	64 TQFP, MLF	Q3,03
ATmega169	16	512	1K	54	1	1	-	1	Y	2	1	8	Y	S	4.5-5.5	0-16	64 TQFP, MLF	Q3,03
ATmega32L	32	1K	2K	32	1	1	1	-	Y	2	1	8	Y	S	2.7-5.5	0-8	40 PDIP, 44 TQFP, MLF	now
ATmega32	32	1K	2K	32	1	1	1	-	Y	2	1	8	Y	S	4.5-5.5	0-16	40 PDIP, 44 TQFP, MLF	now
ATmega64L	64	2K	4K	53	1	2	1	-	Y	2	2	8	Y	S	2.7-5.5	0-8	64 TQFP, MLF	now
ATmega64	64	2K	4K	53	1	2	1	-	Y	2	2	8	Y	S	4.5-5.5	0-16	64 TQFP, MLF	now
ATmega128L	128	4K	4K	53	1	2	1	-	Y	2	2	8	Y	S	2.7-5.5	0-8	64 TQFP, MLF	now
ATmega128	128	4K	4K	53	1	2	1	-	Y	2	2	8	Y	S	4.5-5.5	0-16	64 TQFP, MLF	now

Student level

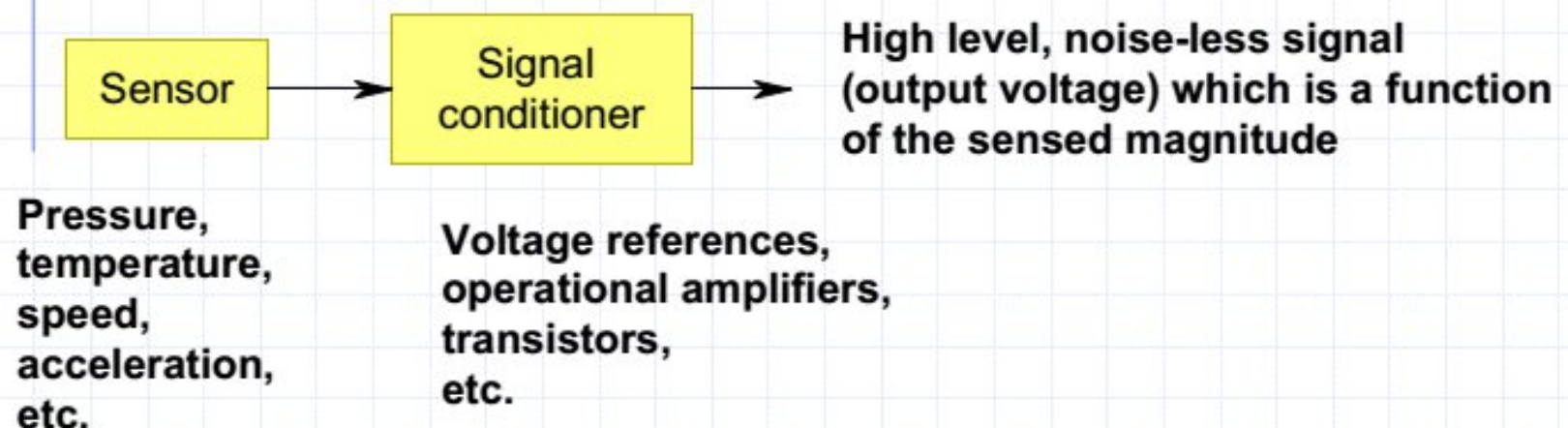
DIGITAL SYSTEM DESIGN PROCEDURES/METHODS

Project design tutorial (I)



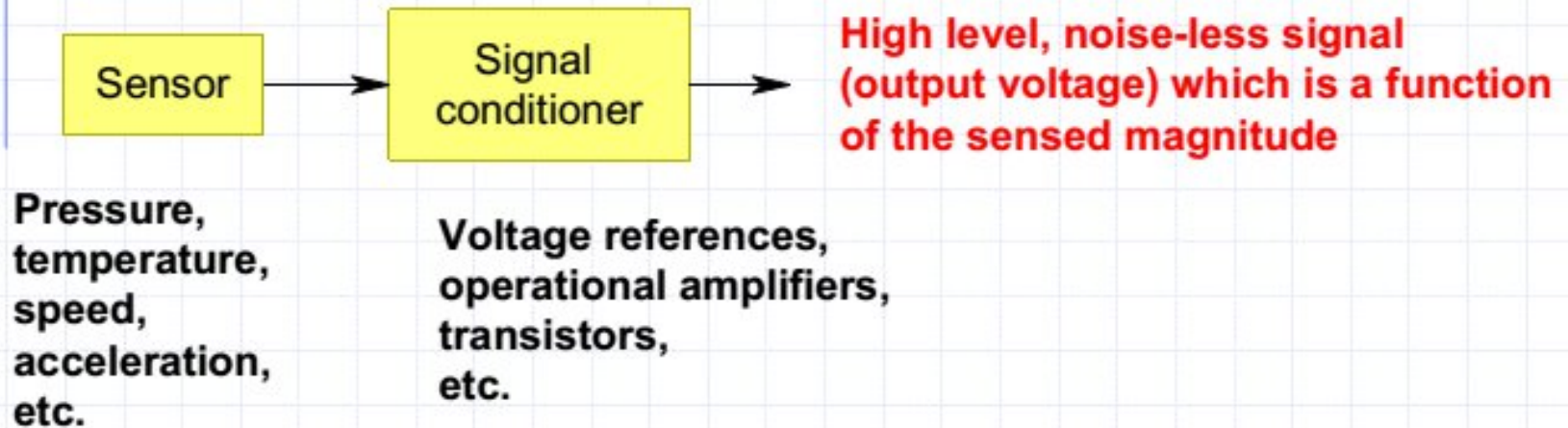
Project design tutorial (II)

Typical sensor analogue subsystem



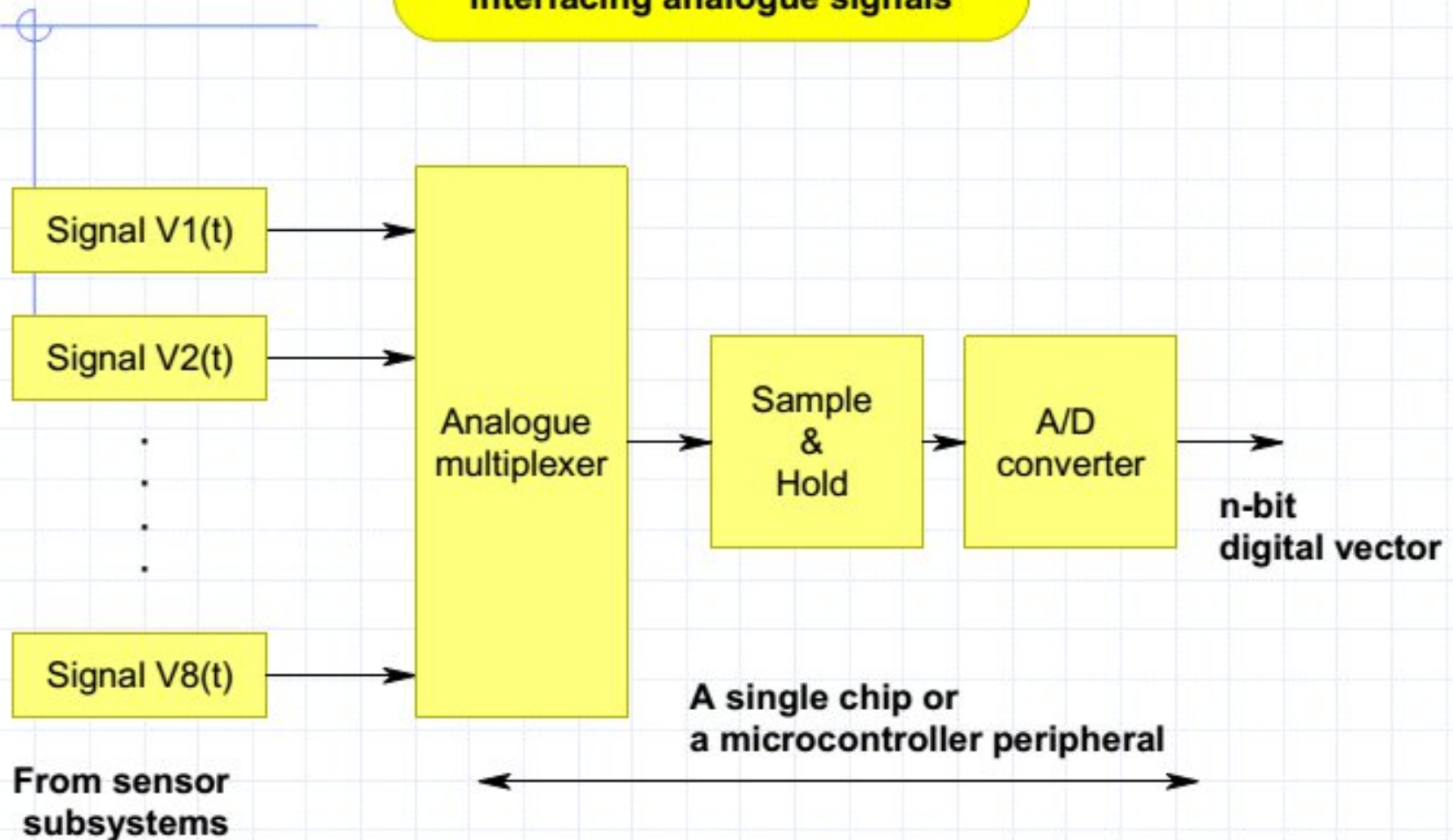
Project design tutorial (III)

Typical sensor analogue subsystem

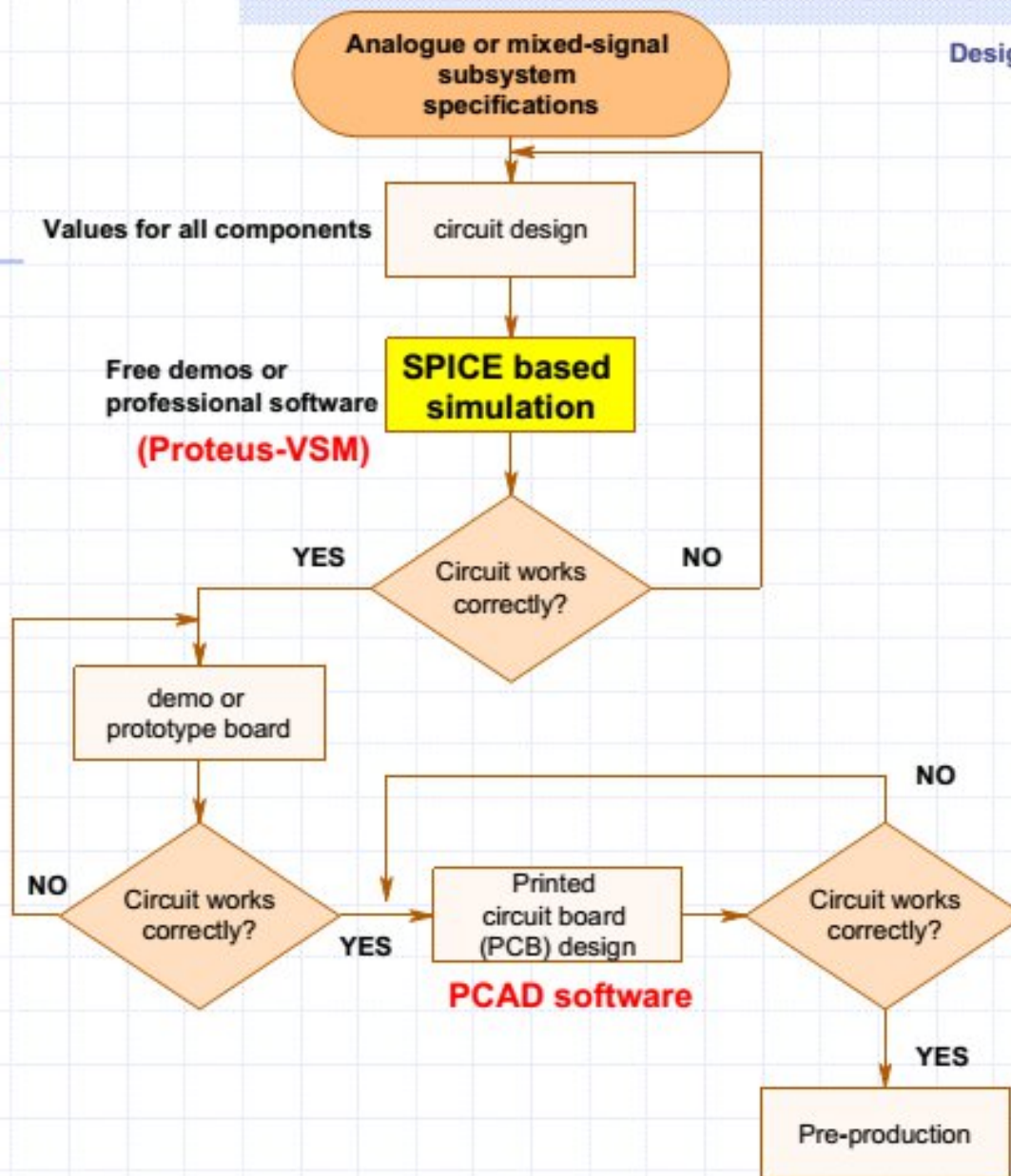


Project design tutorial (IV)

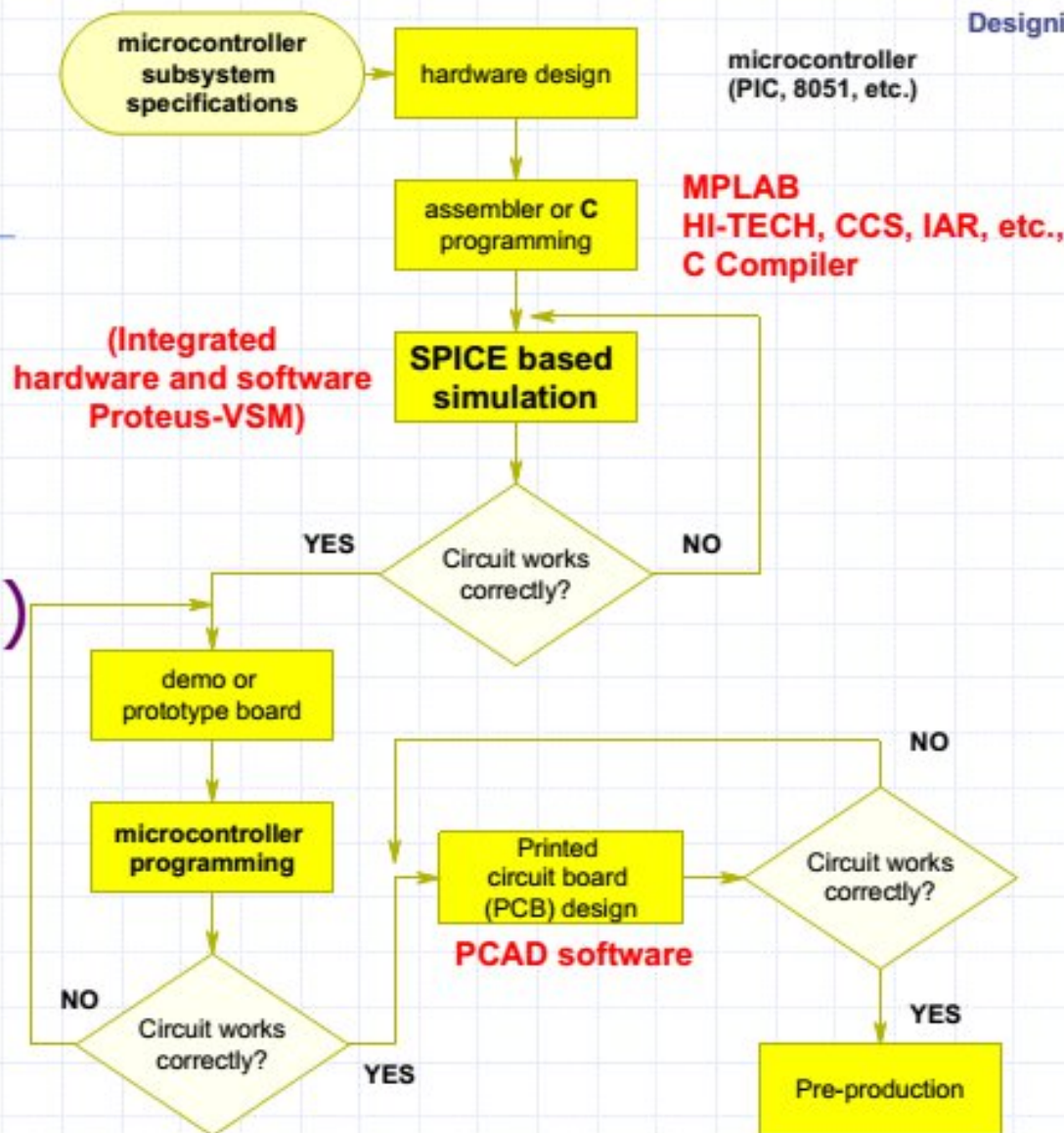
interfacing analogue signals



Project design tutorial (V)



Project design tutorial (VI)



microcontroller
(PIC, 8051, etc.)

**MPLAB
HI-TECH, CCS, IAR, etc.,
C Compiler**

KEIL™
Tools by ARM

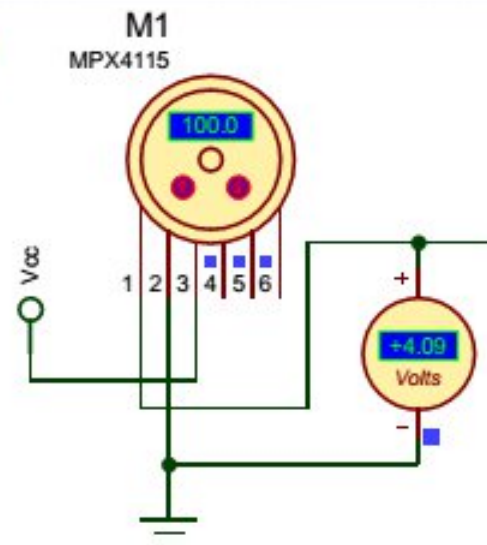


PCAD software

An example:

Initial design of an absolute pressure meter

- Specifications:
 - Pressure range from 900 hPa – 1100 hPa
 - 5 V power supply
- Data from the sensor datasheet
- Initial design of the sensor conditioner
- SPICE-based electrical simulation
- Microcontroller circuit and software design
- Prototyping

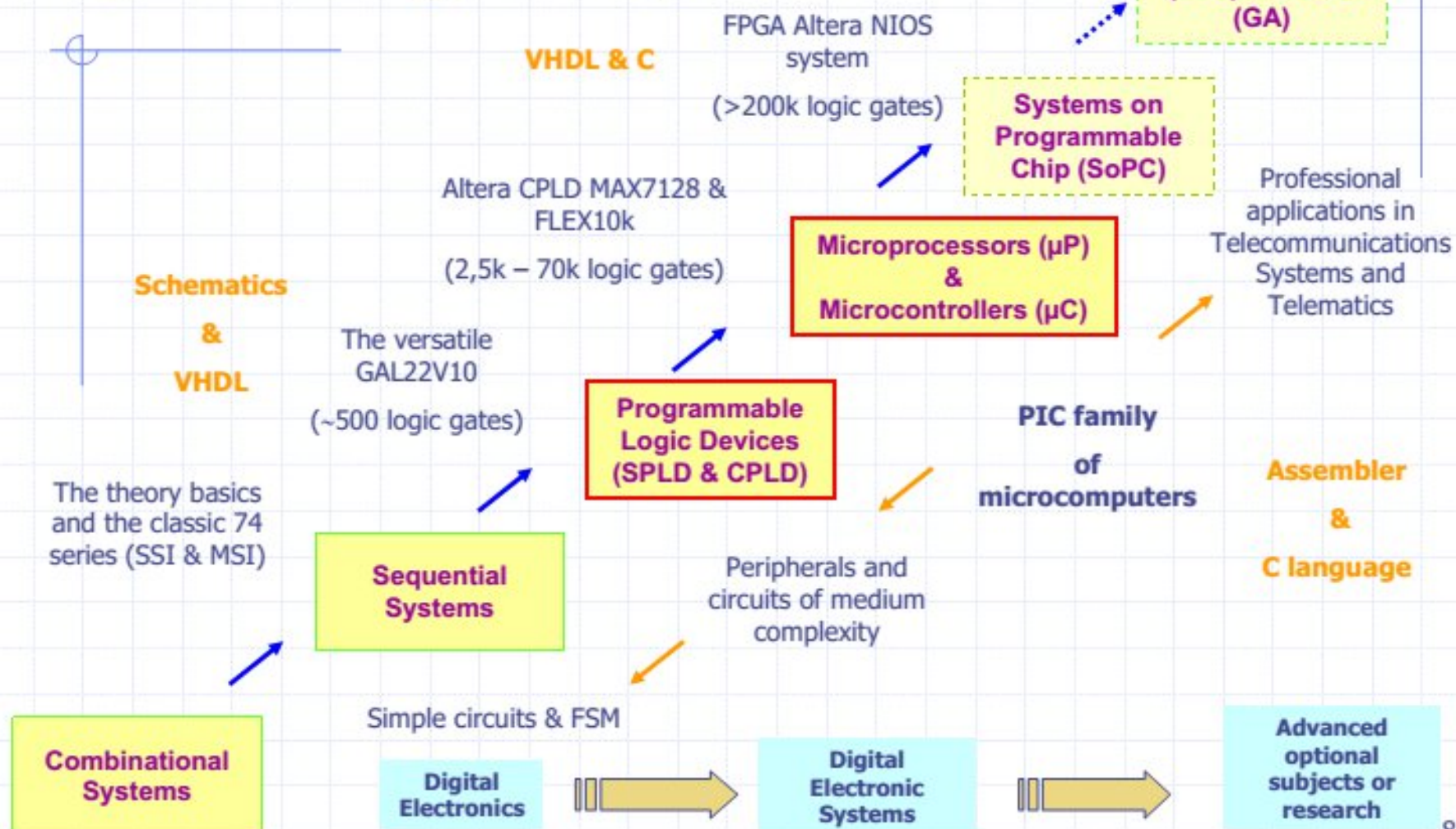


See the web page for details:

http://epsc.upc.edu/projectes/sed/unitats/unitat_1_1/Unitat_1_1.htm

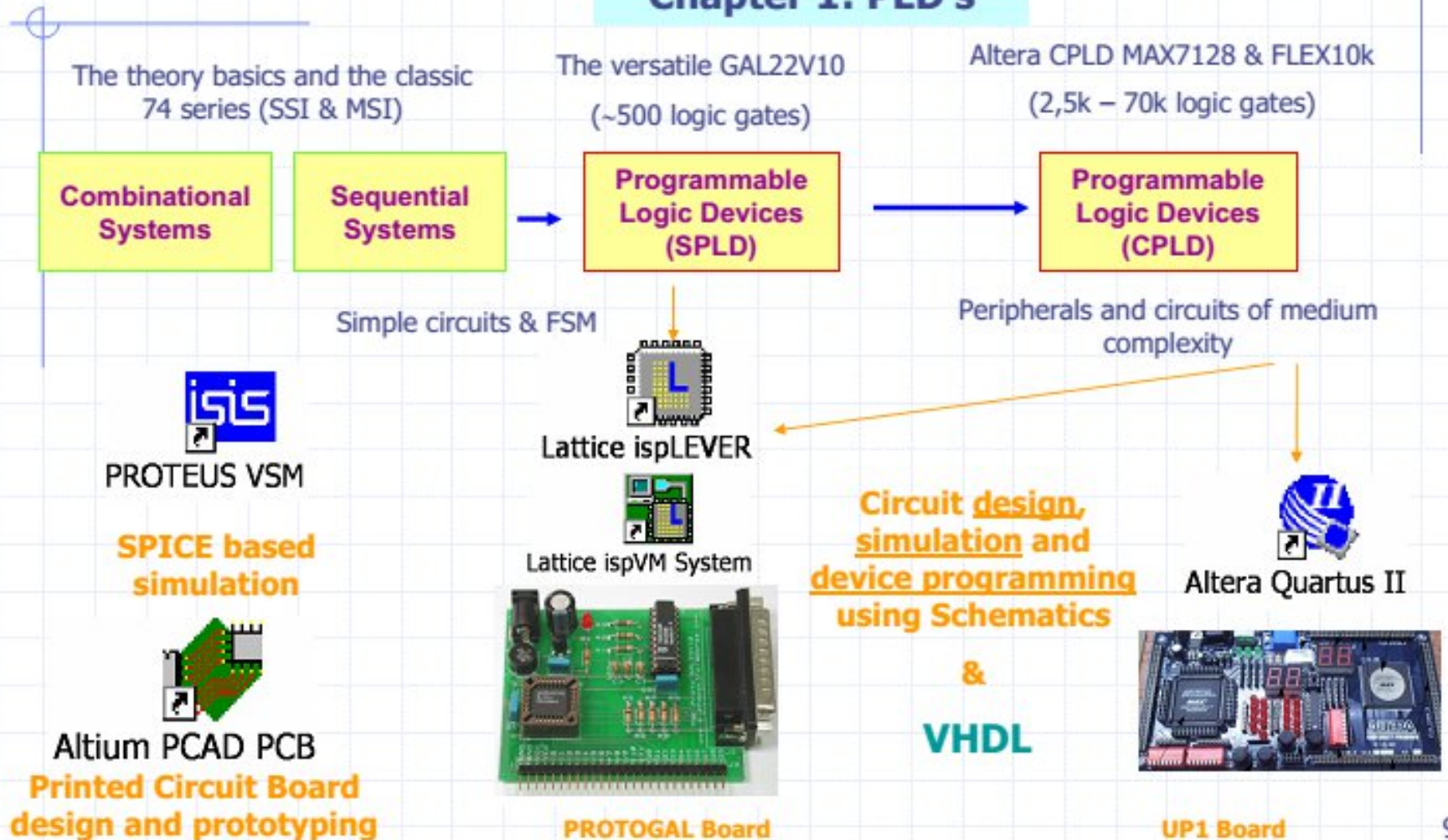
Some concepts involved: physical quantities, sensors, instrumentation, electronic circuits, simulation, analogue-to-digital interfaces, PIC microcontrollers, C code compiler

Digital technology roadmap



CAD tools for digital electronic design systems

Chapter 1: PLD's



UP1

Chapter 1: PLD's

LAB training for CPLD & FPGA systems

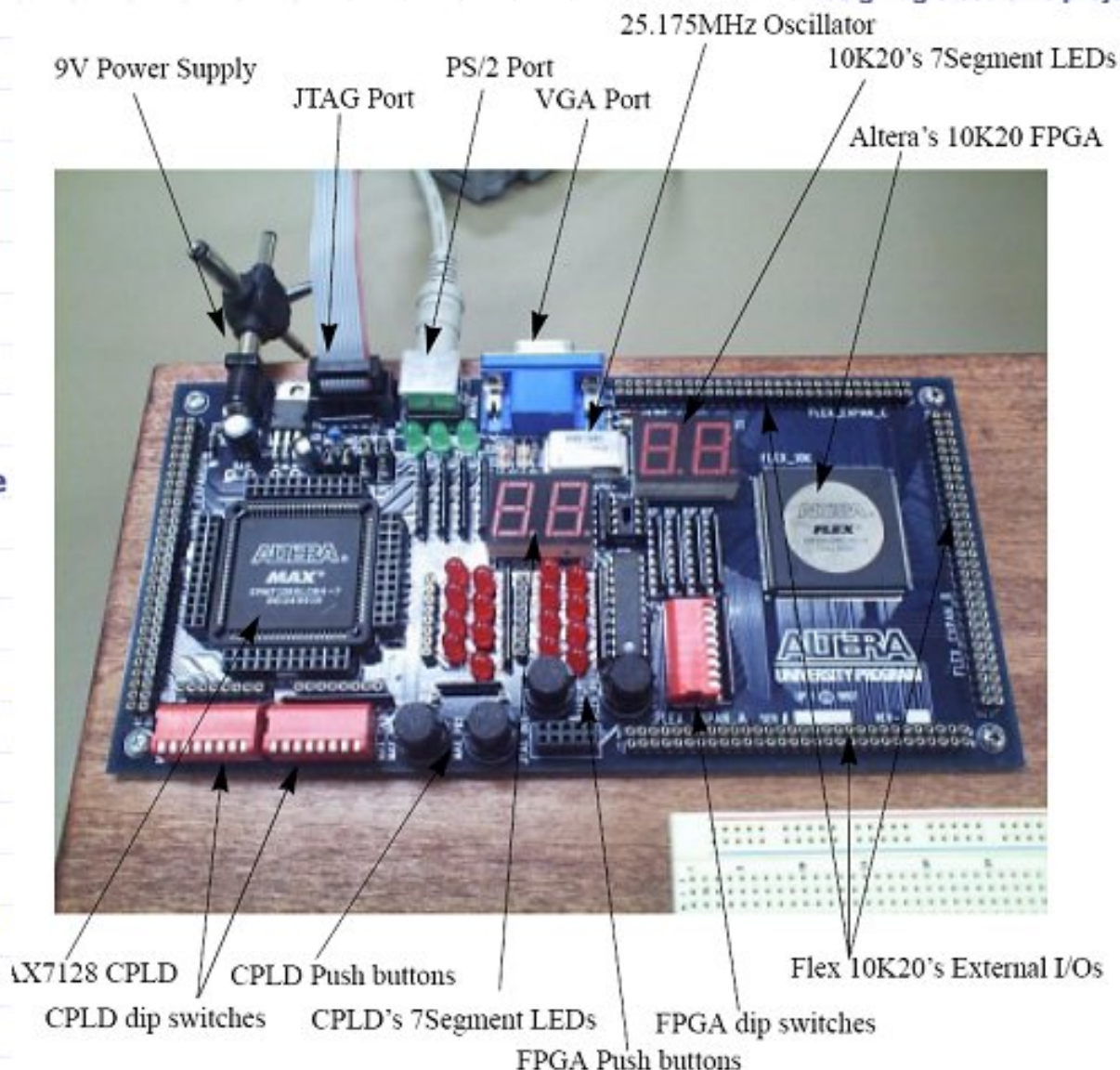
Foto from the internet, source:
**Altera's MAX+plus II and the
UP 1 Educational Board**

*A User's Guide
for*

**Advanced Logic Design,
CPE/EE 422/502**

B. Earl Wells, Sin Ming Loo
Department of Electrical and
Computer Engineering
The University of Alabama in
Huntsville
Huntsville, AL 35899
Version 1, September 14 2000

Designing electronic projects



CAD tools for digital electronic design systems

Chapter 2: μP & μC

Microchip PIC family of microcontrollers



Microchip MPLAB IDE

Circuit design, simulation
and device programming
using

Assembler & C

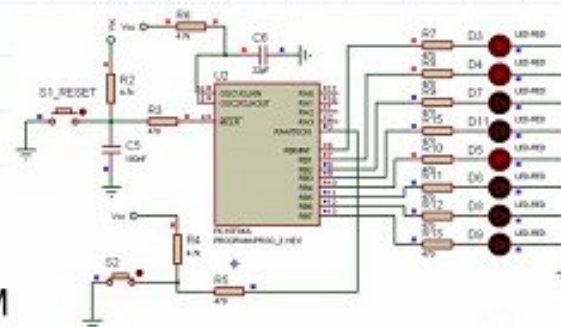


HI-TECH Lite C
Compiler



PROTEUS VSM

SPICE based
interactive simulation
of microcontroller
circuits



Altium PCAD PCB

Printed Circuit Board
design and prototyping



PIC boards

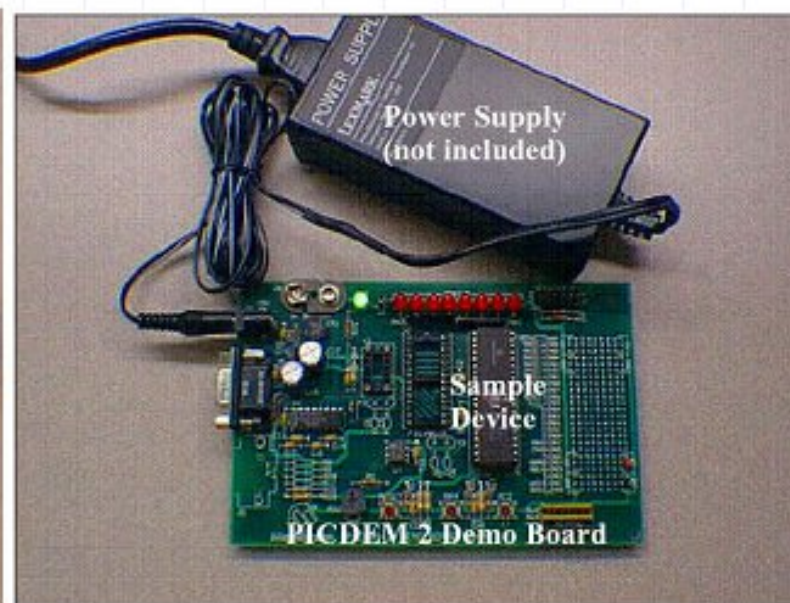
Chapter 2: μP & μC

LAB training for
microcontroller systems



PIC Millennium Board

(http://www.elgarelectronics.co.uk/el_prod.html)



PICDEM2 board

CAD tools for digital electronic design systems

Chapter 3: Systems on Programmable Chip (SoPC)

Professional applications in Telecommunications Systems and Telematics

PLD Altera NIOS system (>200k logic gates)



Altera Quartus II

SoPC Builder
and device
programming
using
Schematics
&
VHDL



Altera Nios SDK Shell

C compiler and
Shell for NIOS
processor
(serial port
interfaced)



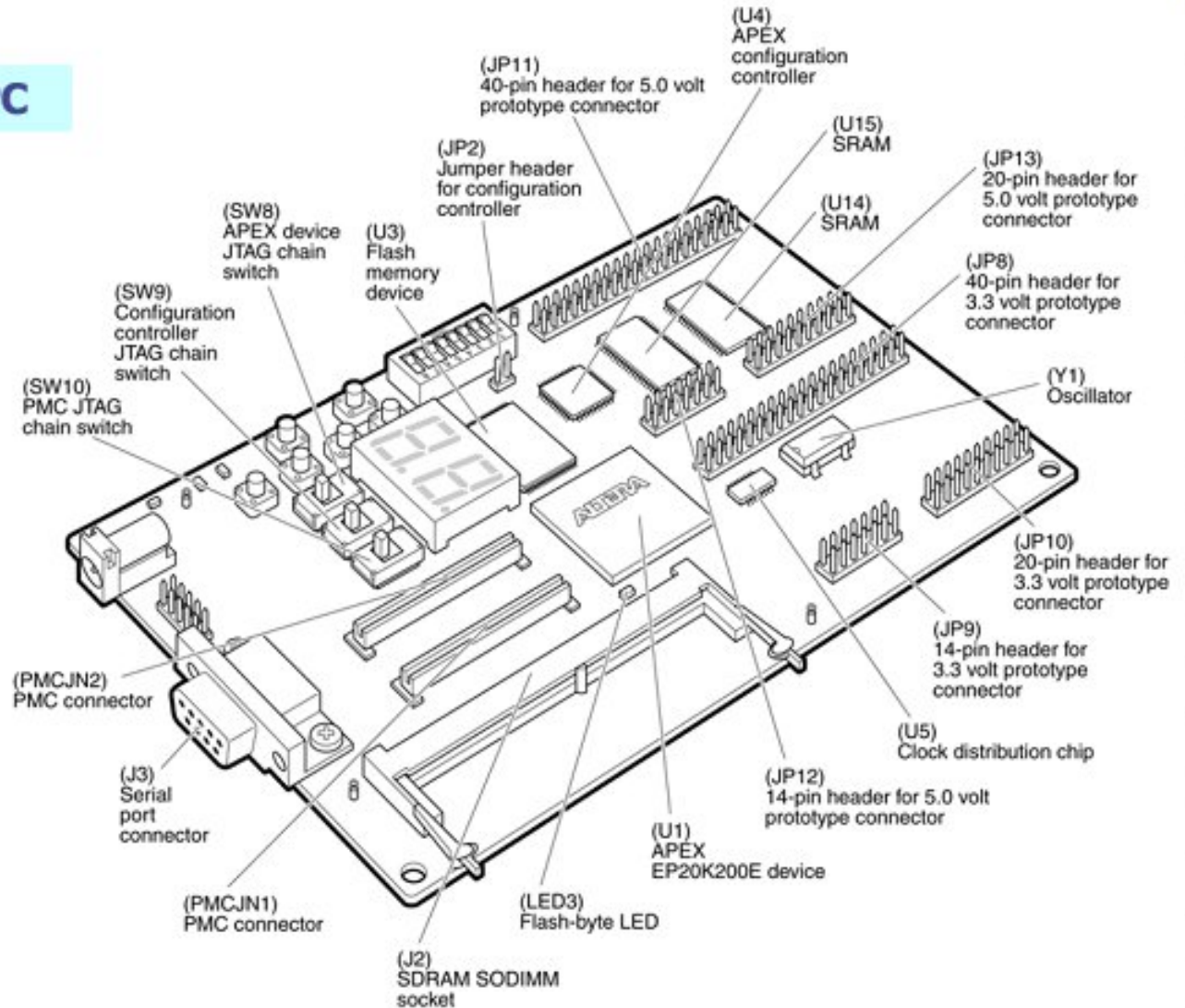
Altium PCAD PCB

Printed Circuit Board
design and prototyping

Chapter 3: SoPC

NIOS Development Board

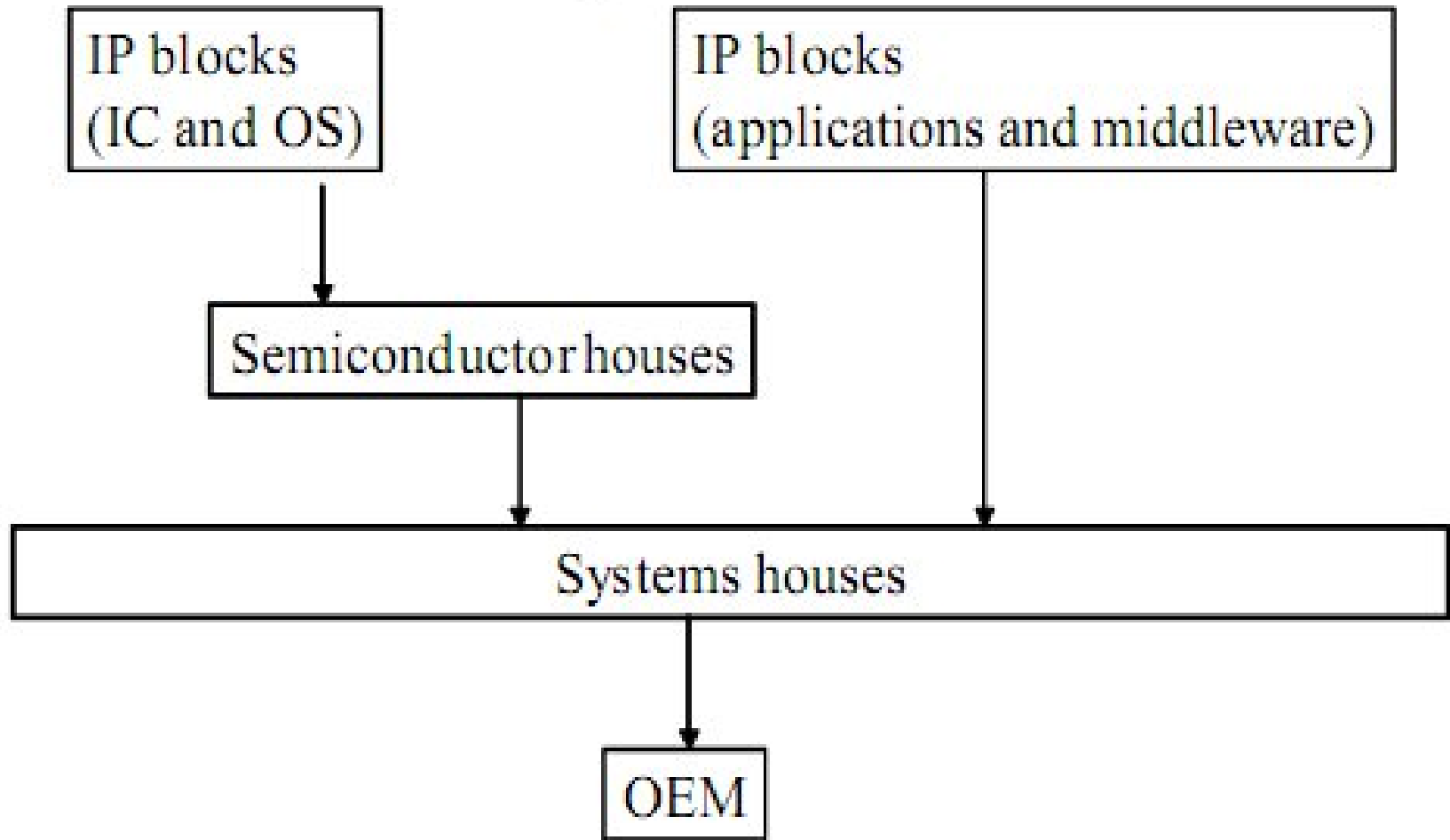
LAB training for
SoPC based in the
Altera APEX
EP20K200E FPGA
(484 pin)



Professional

DIGITAL SYSTEM DESIGN PROCEDURES/METHODS

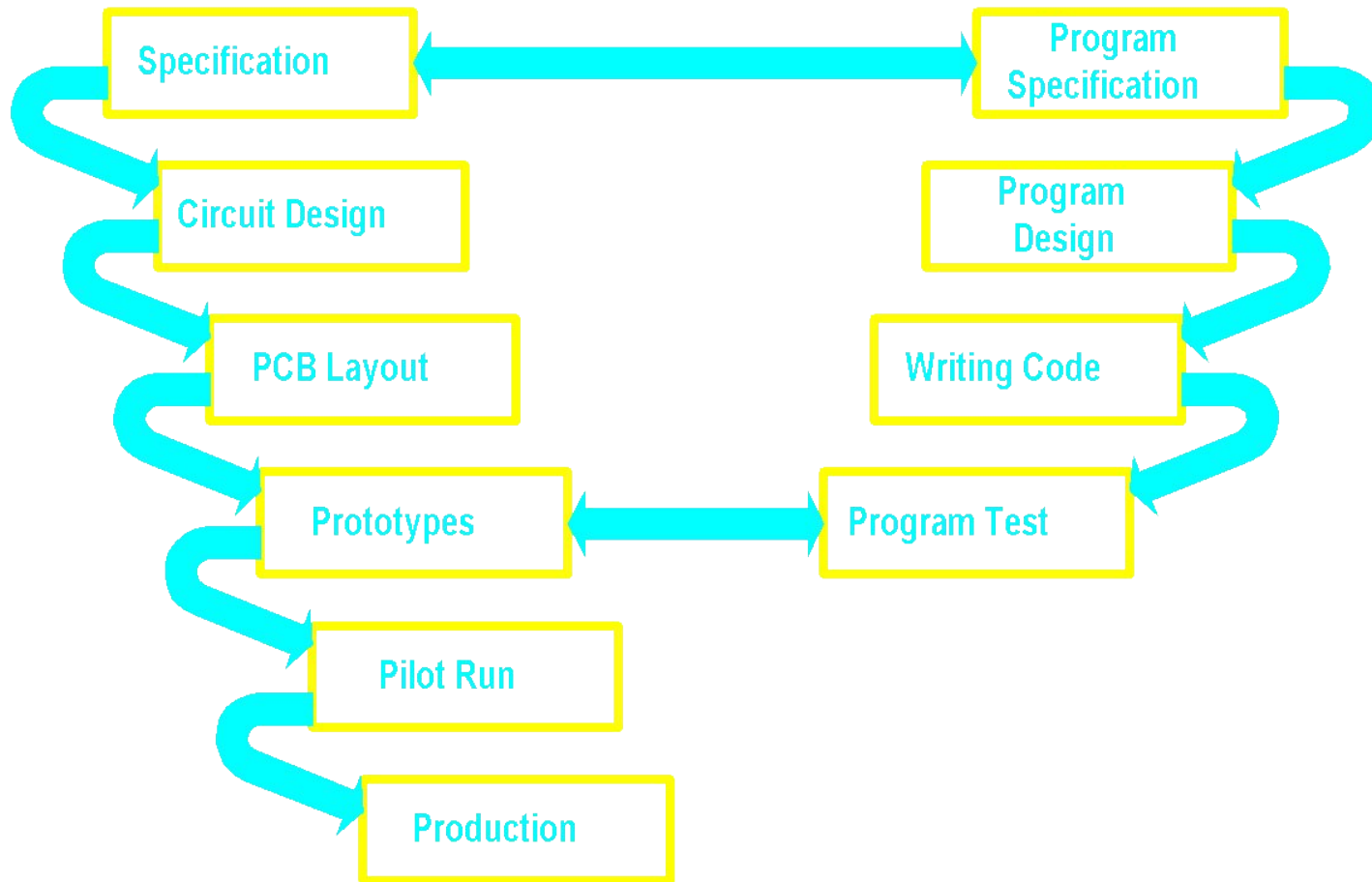
THE EMBEDDED SoC PROVIDER-INTEGRATOR CHAIN



BASIC DESIGN METHODOLOGIES

- Figure out flow of decision-making.
- Determine when bottom-up information is generated.
- Determine when top-down decisions are made.

Electronic Design Flowchart



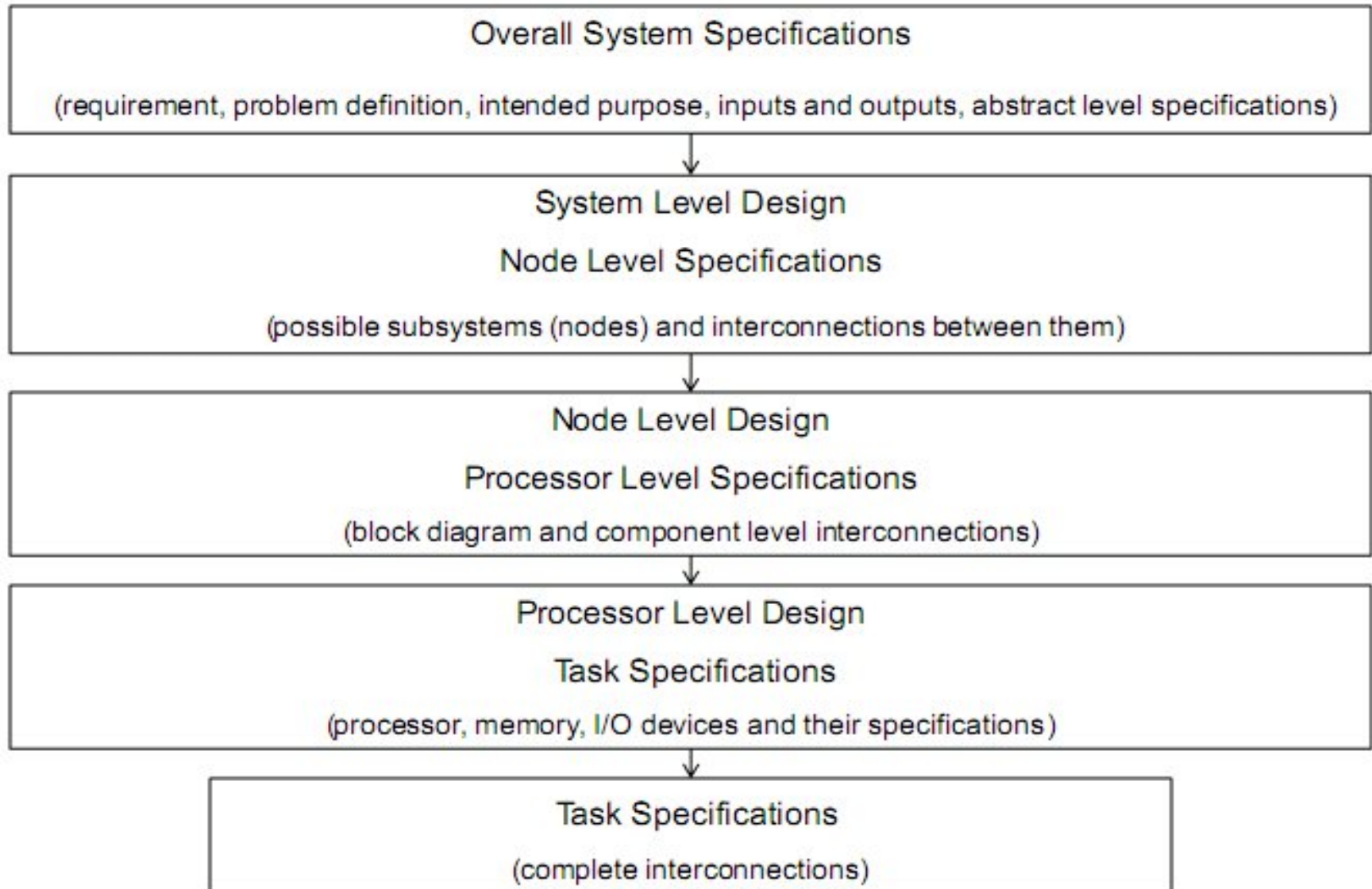
Source

DESIGN STEPS

- **Specification**
- **Circuit Design**
- **Printed Circuit Board Layout**
- **Software design**
- **System Integration and Testing**
- **Pilot Run**
- **Production**
- **Operation and Maintenance**

Source

SPECIFICATIONS DETERMINATION



DESIGN METRICS

- ❑ **Nonrecurring engineering (NRE) Cost:** One time design cost
- ❑ **Unit Cost:** monetary cost of manufacturing each copy of the system, excluding NRE cost.
- ❑ **Size:** Physical space required by the system. (s/w -> bytes, h/w -> transistors and gates)
- ❑ **Performance:** The execution time or throughput of the system.
- ❑ **Power Consumption:** amount of power consumed by the system (lifetime of a battery, cooling requirements of the IC i.e. more power -> more heat).
- ❑ **Flexibility:** ability to change the functionality of the system without incurring heavy NRE cost. (e.g. software)

DESIGN METRICS (CONT..)

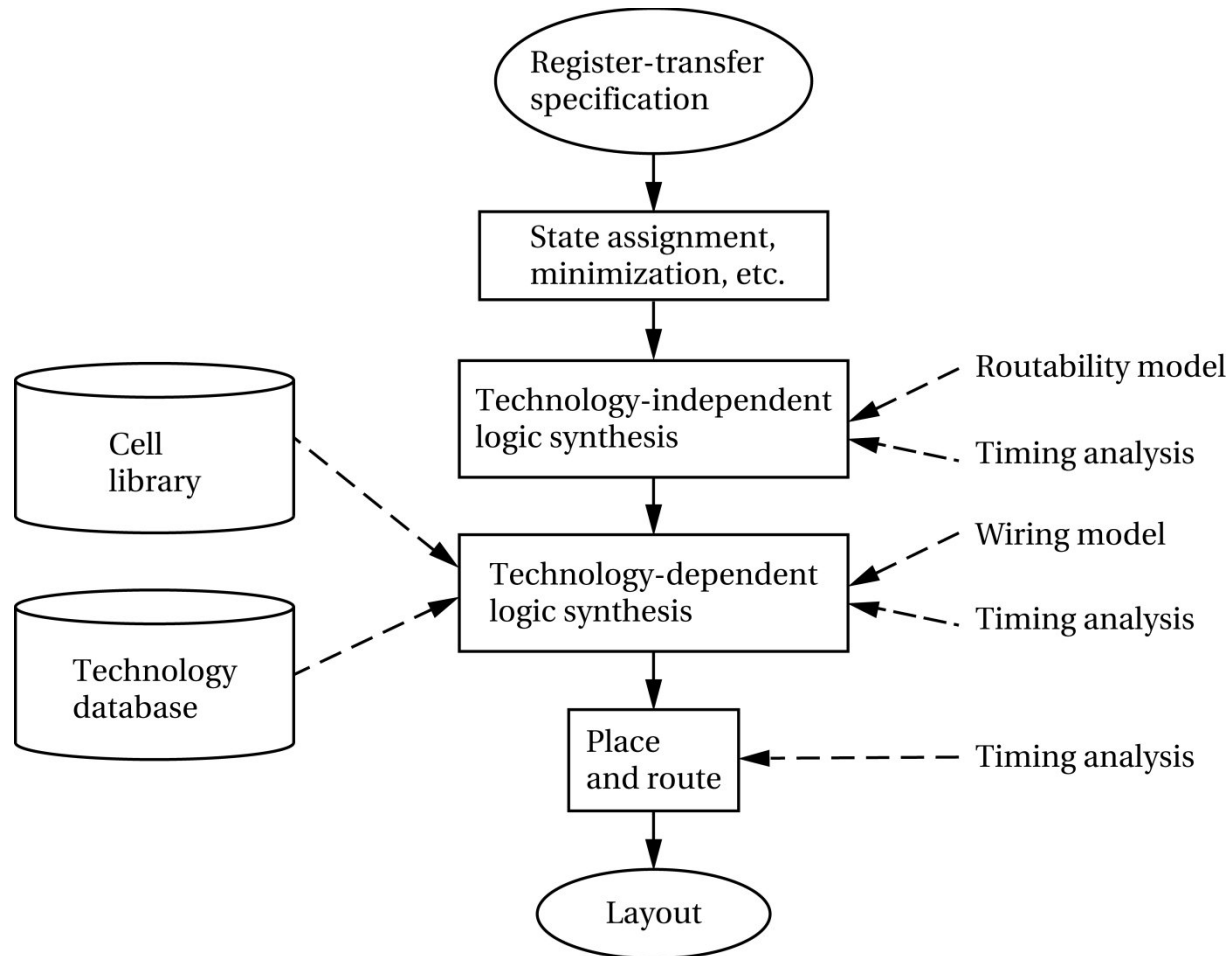
- ❑ **Time-to-prototype:** time needed to build a working version of a system
Works as a yardstick to verify the product's usefulness and correctness as well as to refine and fine tune the product.
- ❑ **Time-to-market:** time required to develop a system to the point that it can be released and sold to the customers. (design time + manufacturing time + testing time = time-to-market)
- ❑ **Maintainability:** the ability to modify the system after its initial release.
- ❑ **Correctness:** measure of the confidence that the built system functions correctly (throughout the design process using test circuitry).
- ❑ **The Performance Design Metric:** measure of how long the system takes to execute our desired tasks
 - **Latency or Response Time:** time between the start of the task's execution and end (e.g time to process an image = 0.20 sec).
 - **Throughput:** number of tasks processed per unit time (e.g a camera may process 4 images/sec)..

DESIGN STEPS

- Specification
- **Circuit Design**
- Printed Circuit Board Layout
- Software design
- Integration and testing
- Pilot Run
- Production
- Operation and Maintenance

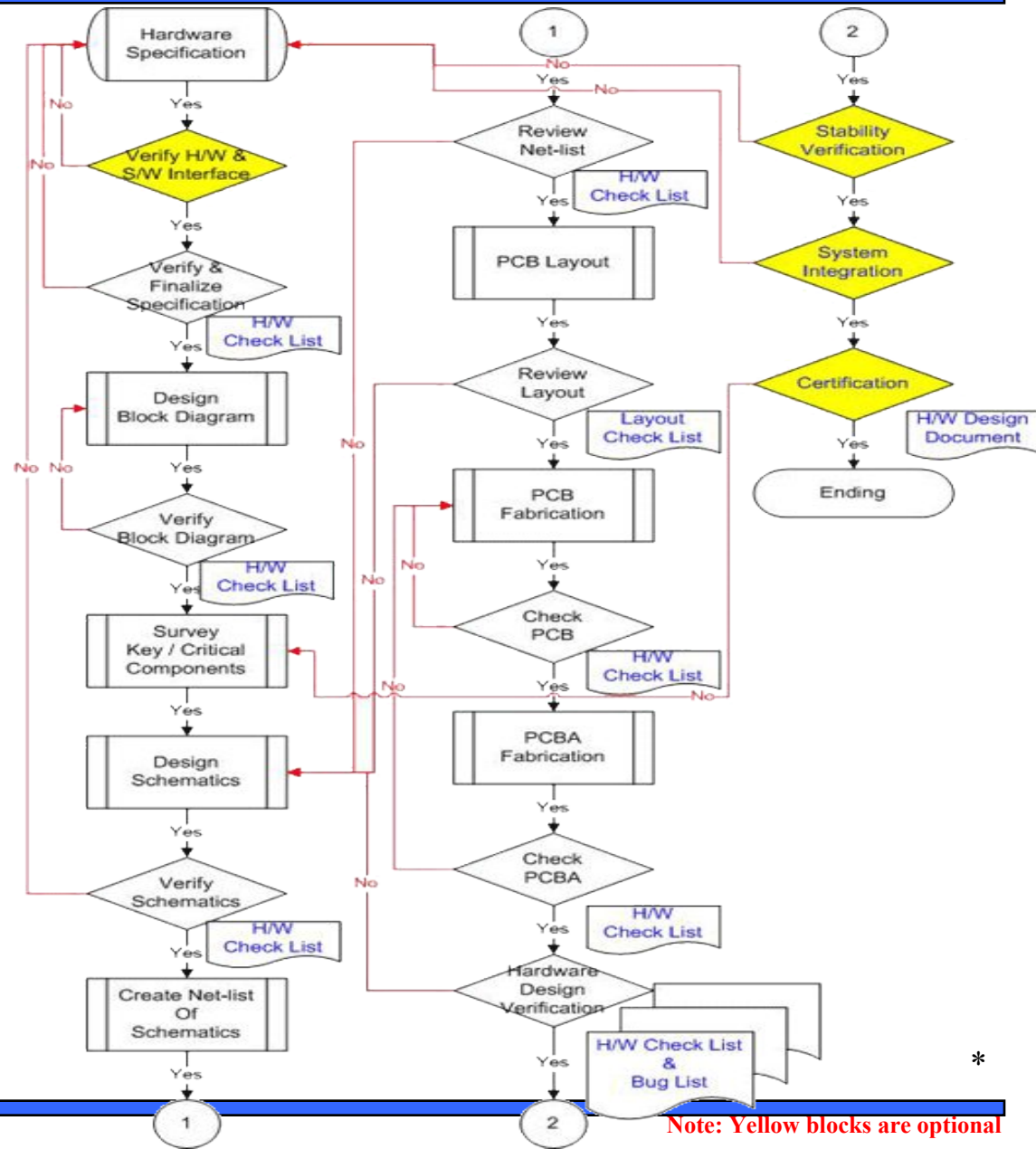
Source

HARDWARE DESIGN FLOW



HARDWARE/CIRCUIT DESIGN FLOW

•Note: Some blocks of this flow provide more detailed information (e.g., specs) compared to that of the blocks of same names which may belong to other phases (e.g., Hardware Spec, Verify and Finalize Spec, System Integration). Thus notice the detailed differences among them.



Note: Yellow blocks are optional

HARDWARE/CIRCUIT DESIGN FLOW

- Hardware Spec block
 - Should be clear and doable as possible.
 - Include software features, schedule, PCB dimensions, stack up, drill location, HW/SW interface, certification, BOM cost requirements.
- Verify HW/SW Interface block (Optional)
 - Define and confirm the interface between HW and SW (computation power, definition of I/O ports, memory size, system expansion/flexibility capability, scheduling of BSP porting and integration, technical contact window and integration density, etc.
- Verify and Finalize Spec block
 - Review and finalize specs, preceded by HW check list
- Design Block Diagram
 - Create block diagram of the HW platform/circuit board
- Verify Block Diagram
 - Review and confirm the block diagram

HARDWARE/CIRCUIT DESIGN FLOW

- Survey Key/Critical Components
 - Check leadtime, price and performance, ... of key/critical components
- Design schematics
 - Design circuitry based on client's final specs
- Verify Schematics
 - Review schematics for flaws
- Create Net-list of schematics
 - Overture of PCB layout
- Review Net-List
 - Ensure that the schematics is consistent to the net-lists and all flaws are corrected
- PCB layout (Details are in the next phase)
 - Placement -> Trace routing -> Test points -> Optical reference spots -> PCB stack -> Impedance Control -> Reserve EMI/EMC solution as possibly - > Requested by the client

HARDWARE/CIRCUIT DESIGN FLOW

- Review Layout (Details are in the next phase)
 - Review the layout according to your/client's requirements, standards, check list
- PCB Fab (Details are in the next phase)
 - Make PCB
- Check PCB (Details are in the next phase)
 - Check PCB according to your/client's requirements, standards, check list
- PCBA Fab (Details are in the next phase)
 - Make PCBA
- Check PCBA (Details are in the next phase)
 - Check PCBA according to your/client's requirements, standards, check list
- HW Design Verification
 - Functionality debug/Make and perform the check list based on the your/client's requirements, concerns and standards.

HARDWARE/CIRCUIT DESIGN FLOW

- Stability Verification (Optional)
 - Based on client's requirements
- System Integration (Optional and details are in the next phase)
- Certification (Optional)
- Ending
 - Provide HW design documentation, paperwork (acceptance certificate, CO and CQ, ...)
 - Close the project

HARDWARE TOOLS

Development boards, EVB

Platforms

Bread boards

Hardware debuggers, analyzers

Emulators

Etc...

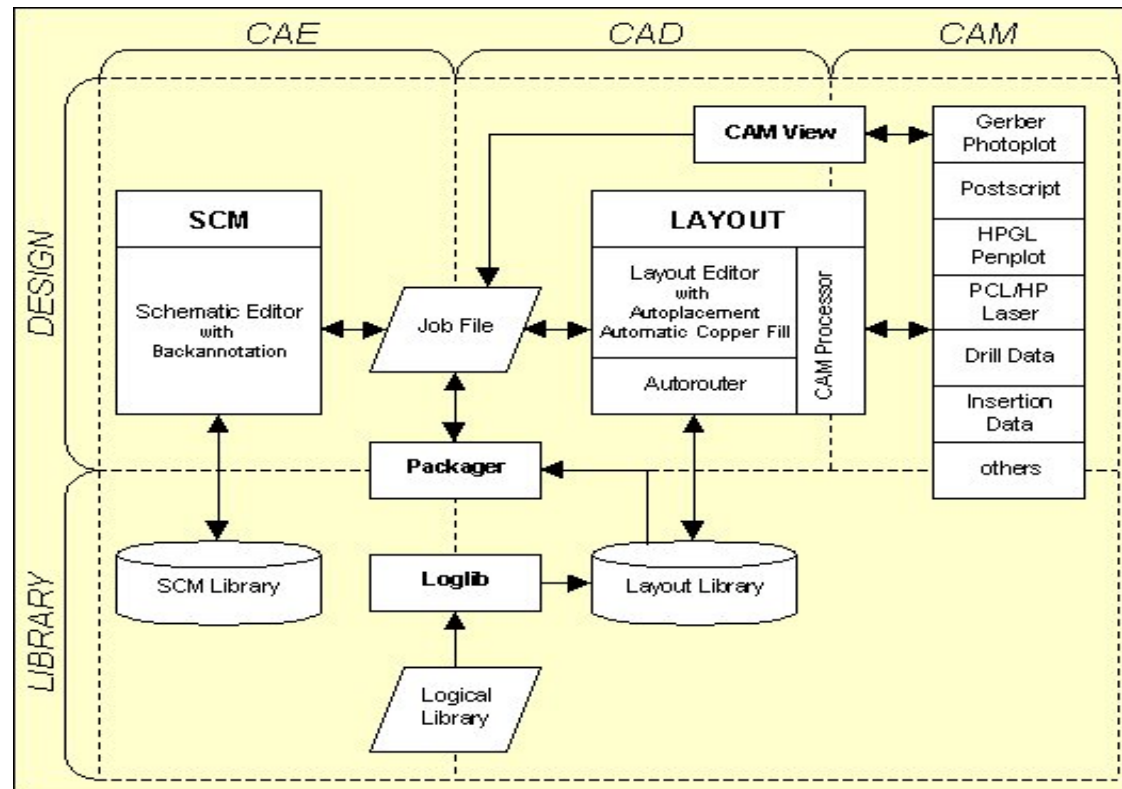
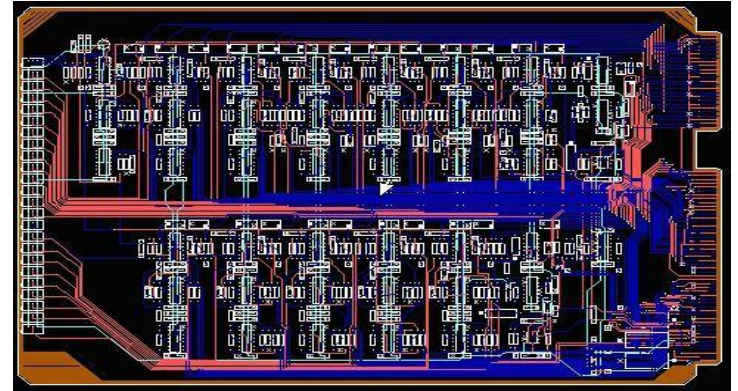
DESIGN STEPS

- Specification
- Circuit Design
- Printed Circuit Board Layout**
- Software design
- Integration and testing
- Pilot Run
- Production
- Operation and Maintenance

Source

Printed Circuit Board Layout

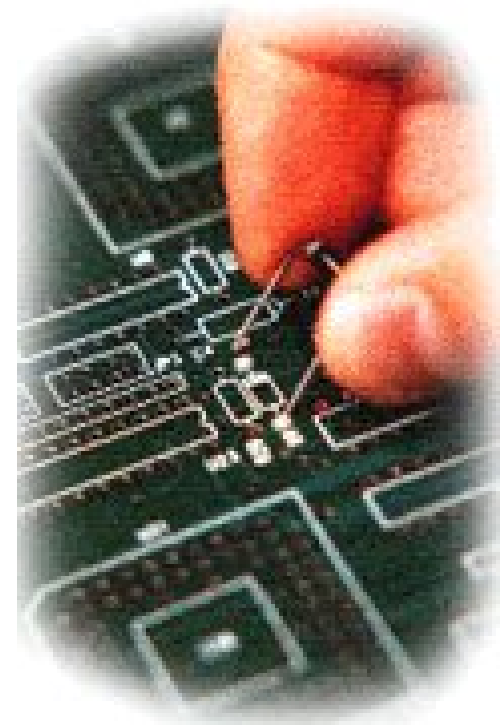
- Connections on the PCB should be identical to the circuit diagram
 - Circuit diagram is arranged to be readable
 - PCB layout is arranged to be functional
- PCB layout can be performed:
 - manually (using CAD)
 - in combination with an Autorouter



Source

Methods of PCB construction

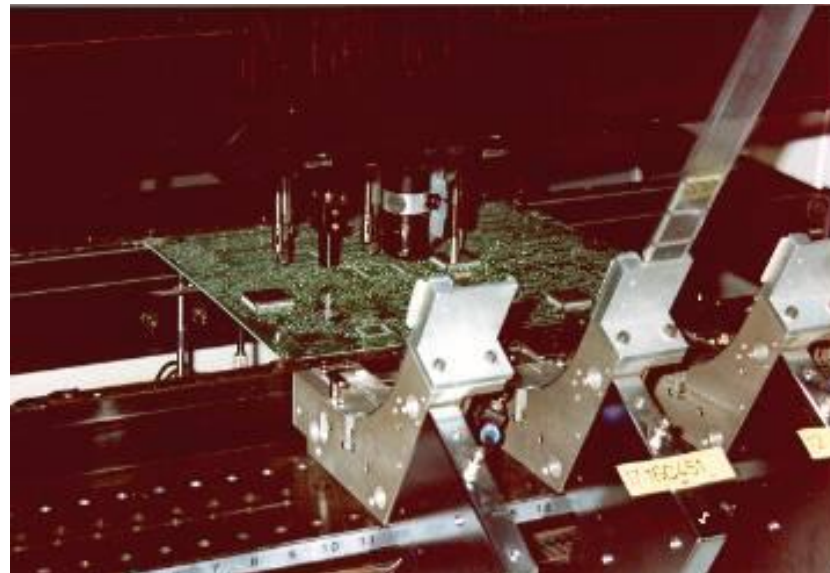
- Conventional
 - Rigid PCB of thickness 1.6mm
 - Wire-leaded components mounted on only one side of the PCB
 - All the leads through holes, soldered and clipped.
 - Easier to debug and repair than Surface mount



Source

Methods of PCB construction

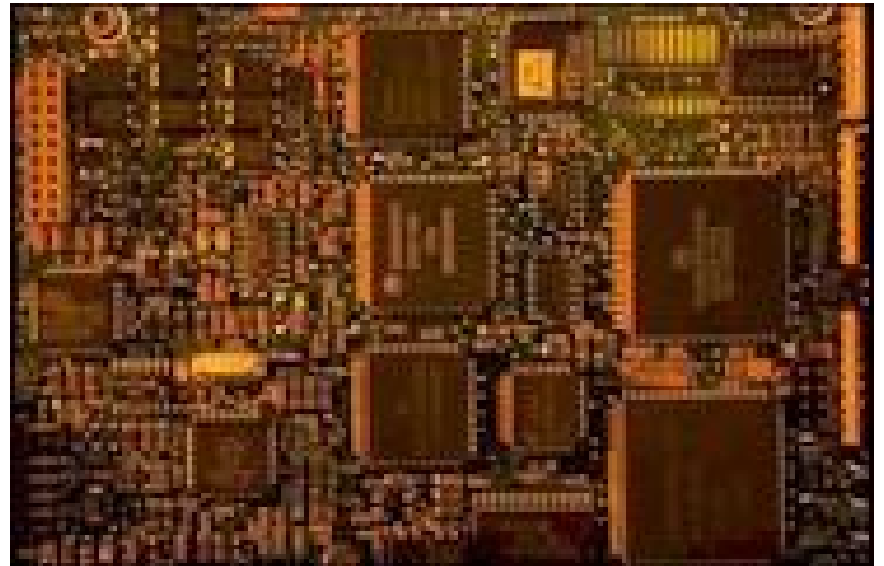
- Surface Mount Technology (SMT) devices (SMD)
 - PCB with tag-leaded components soldered flush to PCB pads
 - Holes are still needed on the PCB, not where the component leads are attached
 - Generally smaller than conventional
 - Generally more suited to automated assembly than conventional



Source

Methods of PCB construction

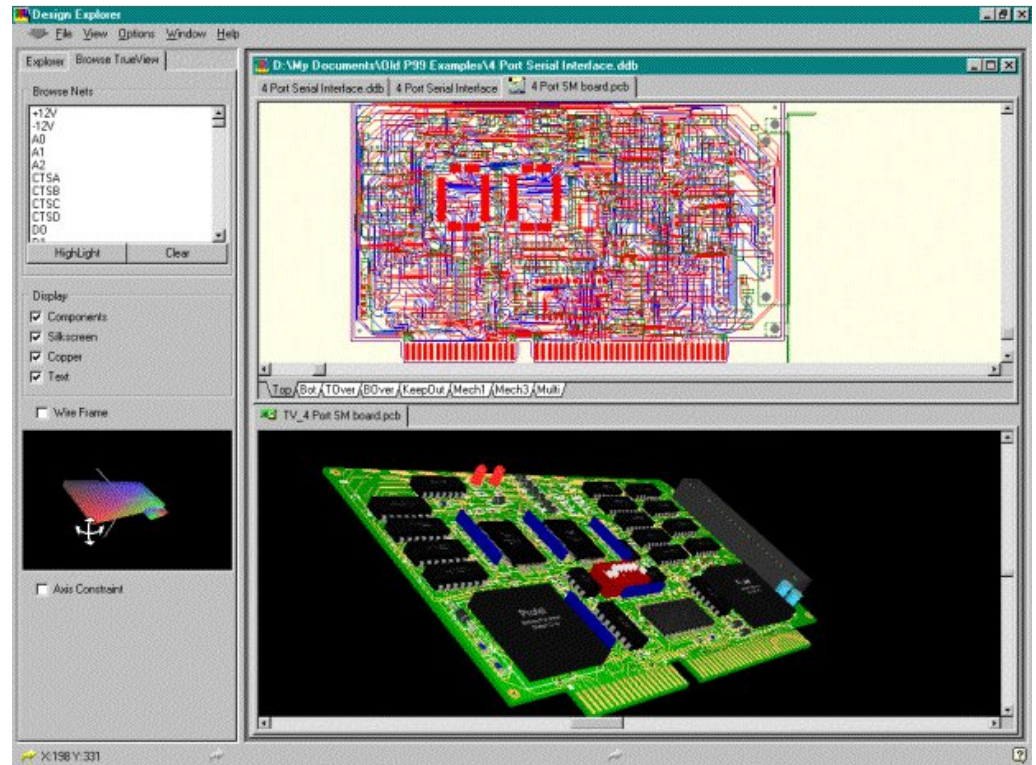
- Surface mount and conventional mix
 - Most boards are a mix of surface mount and conventional components
 - Disadvantages because the two technologies require different methods of insertion and soldering



Source

Printed Circuit Board Etching

- CAD File processing
 - PCB CAD files are sent to the PCB Manufacturer
 - PCB manufacturer inspects the files, making a drill list and adding identification
 - CAD files are processed and sent to a photoplotter to turn into film artwork



Source

PCB DOCUMENTATION

- Front Cover
 - Title, date, version number, customer details, project features
- Schematic
- ECO Sheet
 - Details of any circuit modifications
- Bill of material
 - The parts list
- Parts key
 - A glossary of the part number abbreviations, with package sizes, lead spacing, tolerance notes and preferred types

Source

PCB DOCUMENTATION

- Front panel artwork
- Manufacturing notes
 - Contains the notes relating to previous production runs
 - for instance problems encountered, methods of testing
- Drilling diagram
 - Showing the positioning and size of every hole on the PCB
- Actual size PCB overlay
 - Showing the positioning and identification of the PCB components
 - The plan is printed actual size to allow components to be placed against it to check for fit

Source

DESIGN STEPS

- **Specification**
- **Circuit Design**
- **Printed Circuit Board Layout**
- **Software design**
- **Integration and testing**
- **Pilot Run**
- **Production**
- **Operation and Maintenance**

Source

SOFTWARE DESIGN

- SW design is a step in which design is translated into machine-readable form.
- If SW design is done in sufficient detail then coding can be done effectively.
 - Programs are created in this phase.
- In this phase all software divided into small module then after doing coding for that small module rather than do coding whole software.
- According to design programmers do code and make class and structure of whole software.



SOFTWARE DESIGN

- The major steps in software (in general) / firmware (in particular) design are:
 - Program Specification
 - Program Design
 - Writing code
 - Program Test

Source

PROGRAM SPECIFICATION

- The specification for the Electronic Product being designed will usually also be the specification for the programming required
- The program specification will required the writer to go into substantial detail about how the product actually operates, and how it is used
- A thorough program specification leads straight in to Flow charts and timing diagrams, which are components of Program Design

PROGRAM DESIGN

- The program design stage lays out the structure and algorithms of the software
- The structure and algorithms may be laid out as:
 - flowchart
 - timing diagram
 - description of a protocol
 - memory map; or
 - equation

WRITING CODE

- If the program is well specified and the algorithm design stage has been thorough, the actual code writing stage can become almost mechanical
- By defining the software at the outset, before code is written, a much more defined, integrated, set of code can be produced
- The extra space occupied by comments costs nothing, and if the comments are well laid out there is no possibility that they can detract from understanding the code

PROGRAM TEST

- Divide the testing of the design into small, autonomous units
 - It is easier to detect faults before they are compounded by other factors
- Program testing requires getting diagnostic data out of the target, for analysis
 - By emulation tools, or through the hardware itself (for instance a serial port)

SOFTWARE ENGINEERING FOR EMBEDDED SYSTEMS

- Design for Composability
 - Configurable
 - Adaptable
 - Software reuse
 - Porting of Software
 - The ported software needs extensive checking, modification and re-validation
- Systematic validation
 - Co-design
 - Simulation

SOFTWARE TYPES – APPLICATION AND SYSTEM

- Software/Program:

- Software is an umbrella term that refers to anything data
Software is an umbrella term that refers to anything data stored in hardware, in general practice, it usually refers to programs that you run on a device like a computer or PDA
- Usage of the two words together is redundant, but very common since they are of the same meaning.
- A program/software often refers to a file/a package of files to execute a function(s)/task(s)

- Application software/program

- Software designed to help the user to perform specific tasks (e.g., process data for the user).
- Examples: Word processors, email clients, media players, ...

SOFTWARE TYPES – APPLICATION AND SYSTEM

- System software/program

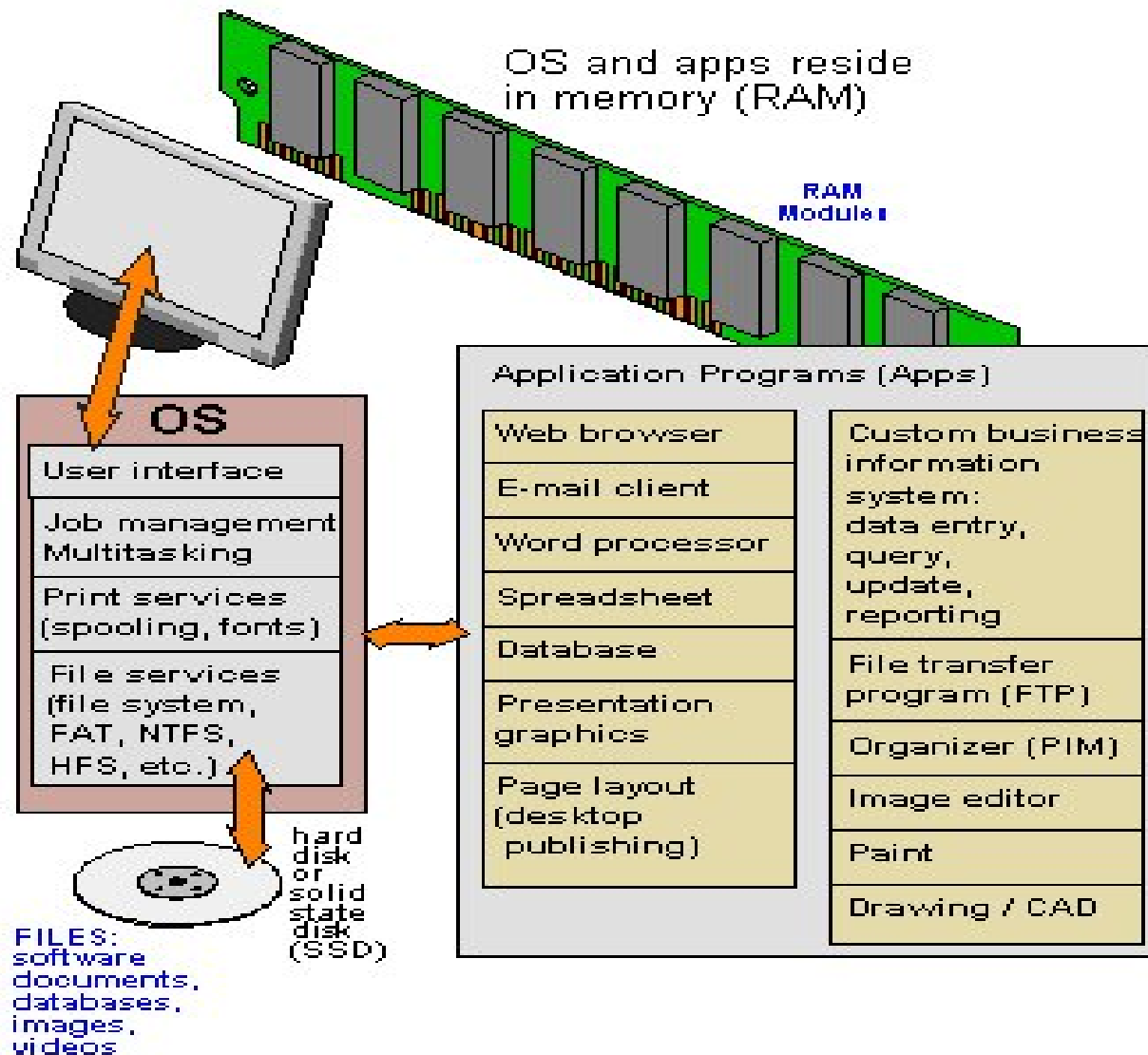
- Designed to operate and control the hardware Designed to operate and control the hardware devices (e.g., PC) and to provide a platform for developing and running application software Designed to operate and control the hardware devices (e.g., PC) and to provide a platform for developing and running application software. It includes programs such as OS's, database managers, compilers, Web servers, router OS's and control programs for other network devices (in contrast with application program).

- Provides the infrastructure in the computer (operating system, utilities and related components)

- Typically system software does not directly apply in the performance of tasks that benefit the user. It serves the application, which in turn serves the user.

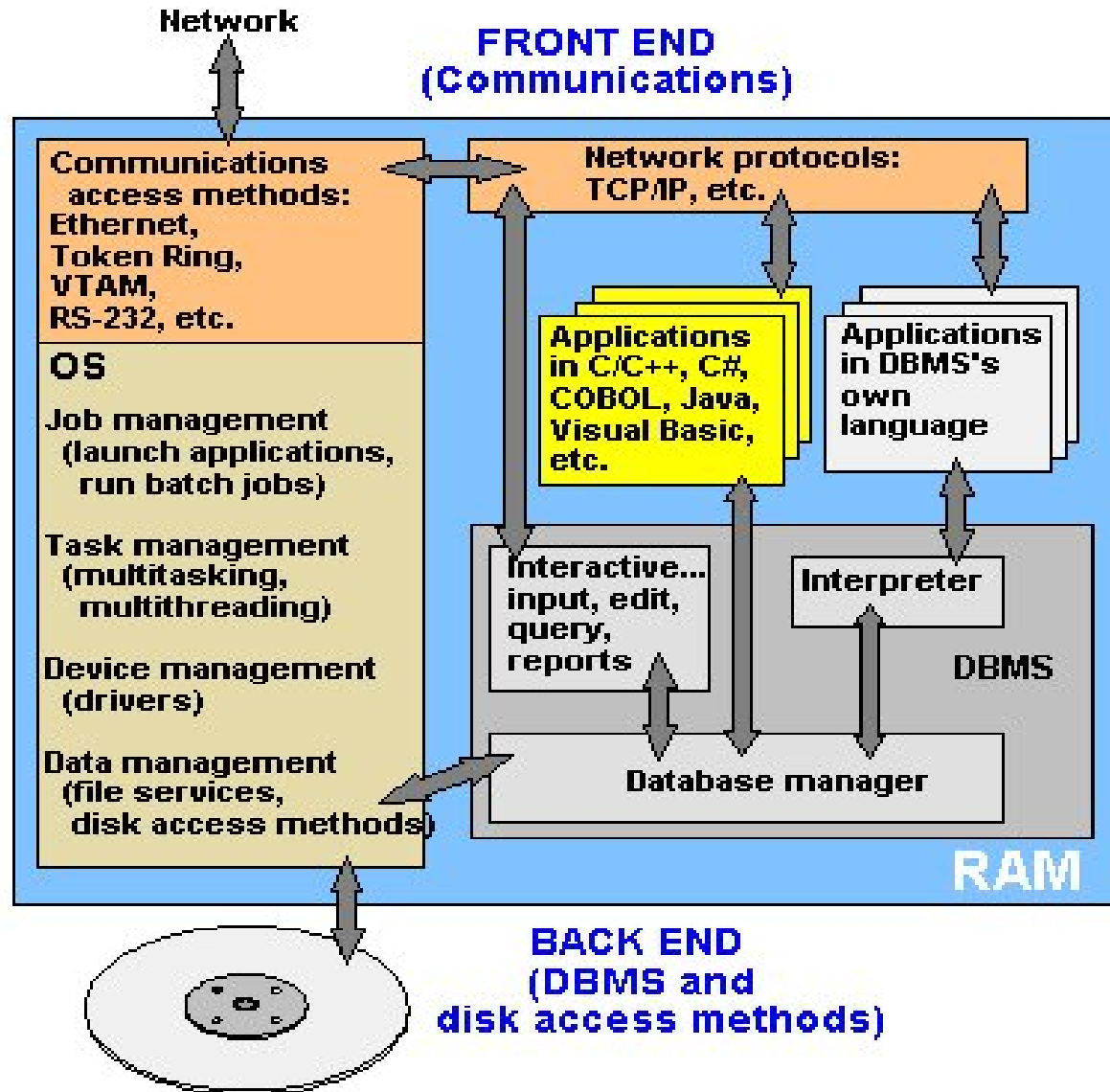
*

- Examples: OS, drivers, BIOS, communication & messaging



SYSTEM SOFTWARE AND APPLICATION

INTERACTION



APPLICATION SOFTWARE

APPLICATION SOFTWARE

These four programs may be stand-alone or part of an application suite.

WORD PROCESSING

Create and edit text files. Replaces all typewriter functions. Some programs provide rudimentary desktop publishing.

DATABASE

Create and edit data records. Interactive editing of data. Ask questions, summarize, sort and print reports.

SPREADSHEET

Create and edit rows and columns of numbers for budgets and financials and "what if" analysis. Multidimensional spreadsheets provide different slices, or views, of data quickly. Advanced financial planning systems provide goal seeking as well as statistical calculations.

PRESENTATION GRAPHICS

Create slide shows, do freehand drawing and turn numbers into 2-D and 3-D business graphics.

APPLICATION SOFTWARE

WEB BROWSER

"Surf the Web." Access the largest body of information in the world, shop online and be entertained.

E-MAIL CLIENT

Send and receive e-mail messages through a separate client program that typically has more features than e-mail via the Web browser.

MEDIA PLAYER

Play back MP3, WMA, AAC and other audio formats. Play back WMV, MPEG and other video formats.

PIM (Personal Information Manager)

Organize random information for fast retrieval. Includes such features as a telephone list with automatic dialing, calendar, scheduler and tickler.

PROJECT MANAGEMENT

Keep track of a project and determine the impact of changes. The "critical path" is computed, which monitors all tasks that will slow down the entire project if delayed.

CAD (Vector graphics)

Create drawings for illustration and industrial design.

IMAGING (Raster graphics)

Scan documents and paint pictures into TV-like images.

DIAGRAMMING PROGRAM

Create drawings of interconnected symbols, such as network diagrams and organization charts. When symbols are moved, the lines stay connected.

CONTACT MANAGER

Keep track of prospects, names, addresses, appointments. Similar to a PIM, but specialized for sales activities.

MATHEMATICAL

SYSTEM SOFTWARE

SYSTEM SOFTWARE

OPERATING SYSTEM

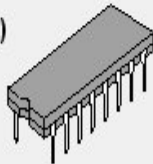
Manages the computer system. Provides file, task and job management. All application programs "talk to" the operating system. Examples are Windows, Mac OS X, Unix and Linux.

DRIVER

Software that supports a peripheral device, such as a display adapter or DVD drive. The driver contains the detailed machine language necessary to activate all functions in the device. The operating system commands the driver, which in turn commands the hardware device.

BIOS (Basic Input/Output System)

In a PC, a set of software routines built into a chip that boots the machine and serves as an interface between the drivers and the peripheral devices.



SYSTEM SOFTWARE

TP MONITOR

Mainframe/midrange program that distributes input from multiple terminals to the appropriate application. This function is also provided in LAN operating systems.

CICS is widely used in IBM mainframes, and Tuxedo and Encina are widely used in Unix systems.

NETWORK OPERATING SYSTEM

Manage traffic and security between clients and servers in a network. Examples are Windows, Unix, Linux and NetWare.

COMMUNICATIONS PROTOCOL

Set of rules, formats and functions for sending data across the network. There are many protocol layers starting at the top application layer to the bottom physical layer

The most popular transport protocol, which manages the delivery of packets is TCP/IP. The most popular data link protocol (access method), which transmits packets from point to point is Ethernet.

MESSAGING PROTOCOL

Set of rules, formats and functions for sending, storing and forwarding e-mail in a network. The major messaging protocol is the Internet's SMTP. Earlier protocols were SNADS, MHS, X.400, cc:Mail and Microsoft Mail.

SYSTEM SOFTWARE

DATABASE MANAGEMENT SYSTEM (DBMS)

Manages the storage, retrieval, security and integrity of the database. A DBMS may provide interactive data entry, updating, query and reporting or rely entirely on the application program for such functions. The DBMS may reside in a mainframe or in a file server in a client/server architecture. Most DBMSs support the SQL language, and many include a complete programming language for application development.

Examples of popular mainframe and client/server DBMSs are DB2, Oracle, SQL Server and Informix. MySQL and PostgreSQL are open source DBMSs for Unix/Linux.

PROGRAMMING LANGUAGE

Translate source language into machine language using assemblers, compilers, interpreters and application generators. All system software and application software must be programmed in a programming language and turned into machine language for execution.

Examples of programming languages are assembly language, C, C++, Visual Basic, Java, Perl, Python, FORTRAN and COBOL.

SOFTWARE VS. FIRMWARE

- Software is an umbrella term that refers to anything data Software is an umbrella term that refers to anything data stored in hardware, in general practice, it usually refers to programs that you run on a device like a computer or PDA

- Software tends to be big applications to be quite useful, ranging from a few hundred kilobytes to a few gigabytes in size (generally refers to high level programs).
- Often stored in memory that is easily accessible and even replaceable by the user (e.g., hard-disk). Also easy to install on different hardware platforms
- Software is changed constantly

- Firmware is basically a type of software

- It is programmed information that is responsible for the operation of certain hardware like traffic lights, modems, consumer appliances (firmware is used to microcode embedded in most hardware).
- Firmwares are very small and would usually be just a few kilobytes in size.
- Memory to store firmwares is often embedded to the device itself and is not replaceable by the user, e.g., ROM Memory to store firmwares is often embedded to the device itself and is not replaceable by the user, e.g., ROM, EPROM Memory to store firmwares is often embedded to the device itself and is not replaceable by the user, e.g., ROM, EPROM, or Flash (firmware is critical for the device to run and tampering or removing it can cause serious consequences) => firmware is very rarely changed *

Firmware

- The major steps in Firmware design are:
 - Program Specification
 - Program Design
 - Writing code
 - Program Test

SOFTWARE COMPONENTS

- For products

- Applications

- Middleware

- Provides services to software applications beyond those available from the OS, thus can be described as "software glue".

- OS kernels

- A program with functions for memory (de)allocation, task scheduling, inter-process communication, management of exception handling signals, mailboxes, interrupt control, device drivers and management

- Drivers

- Control/operate a particular type of hardware device. When a calling program invokes a routine in the driver, the driver issues commands to the device. Once the device sends data back to the driver, the driver may invoke routines in the original calling program.

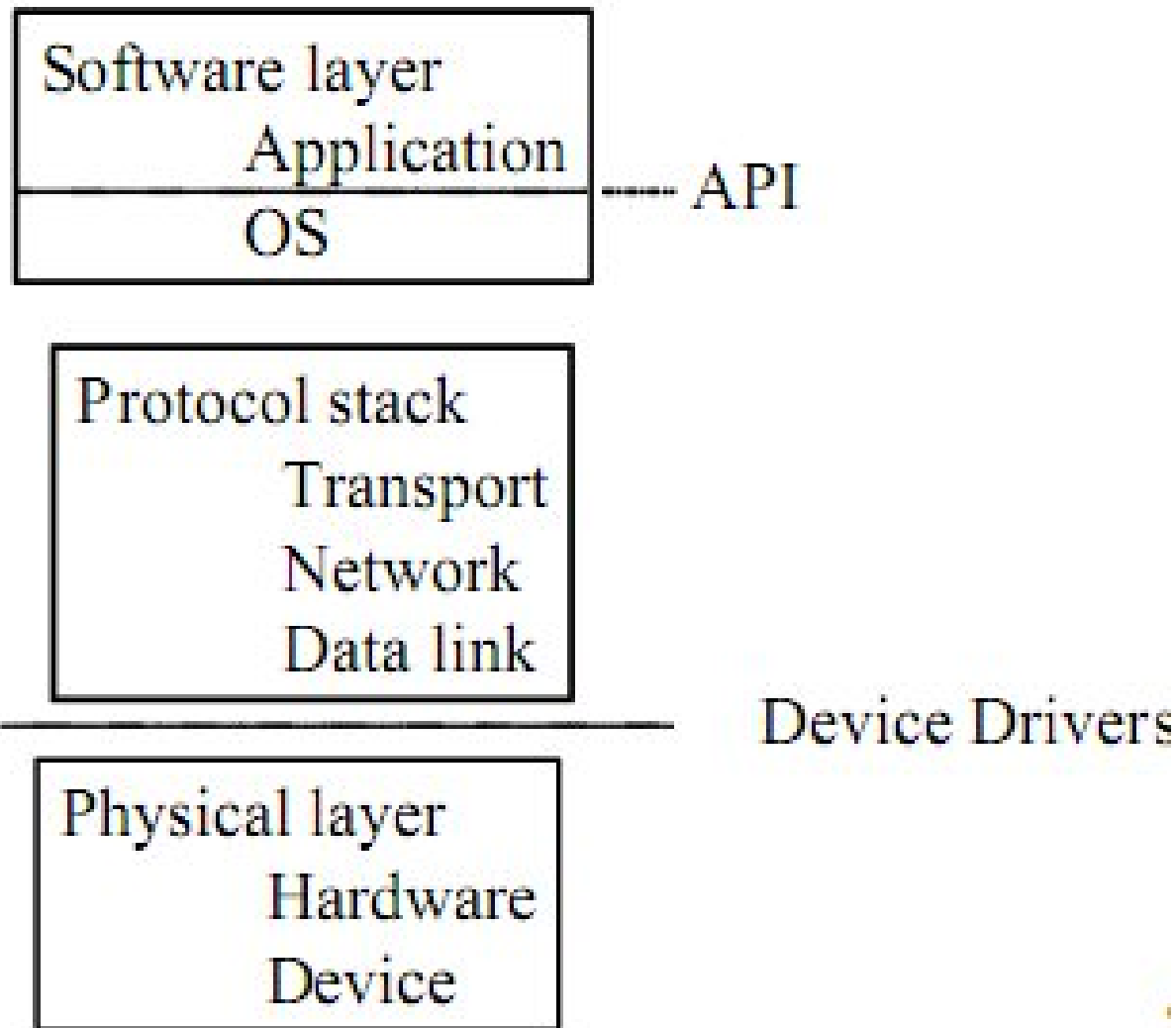
- Software for controlling, receiving and sending a byte or a stream of bytes from/to a device. Drivers are hardware-dependent and operating-systemSoftware for controlling, receiving and sending a byte or a stream of bytes from/to a device. Drivers are hardware-dependent and operating-system-specific. They usually provide the interrupt handling required for any necessary asynchronous time-dependent hardware

- Acting as translator between a hardware device and the applications or OS's that use it.

- For development

- Software tools/Libraries

LAYERED SOFTWARE ARCHITECTURE

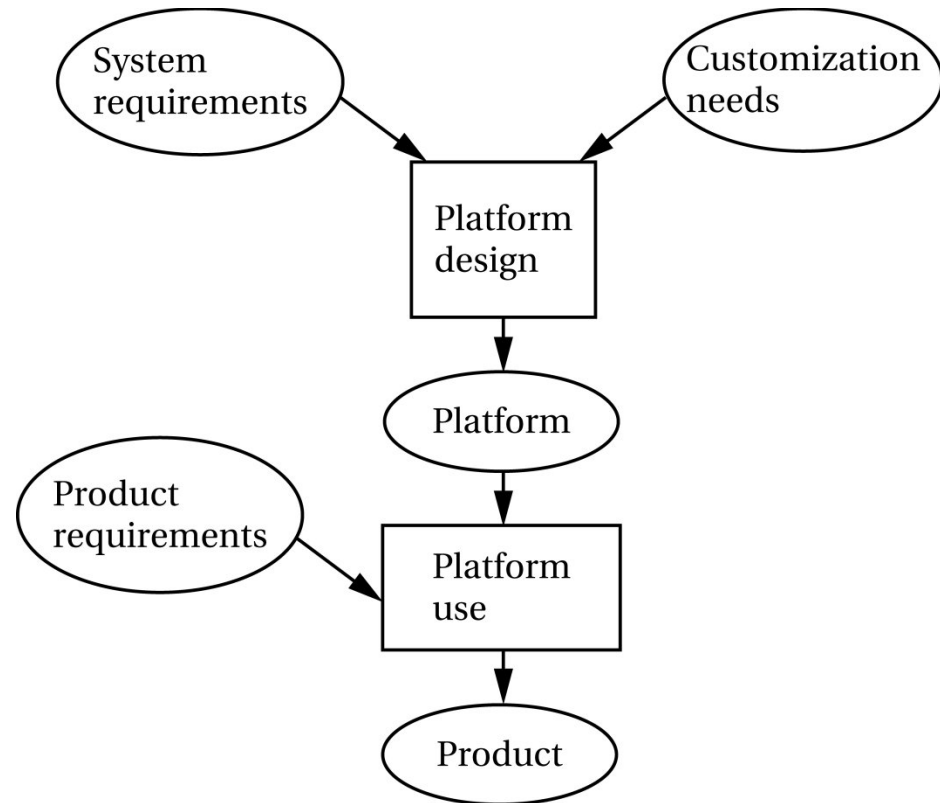


SOLUTIONS TO EMBEDDED SOFTWARE

- Platform-based design
 - Architecture dependent, rapid platform derivative design
- Function-architecture design
 - Implementation independent
- Software as Components
 - Concept of design patterns
- API standardization
 - RTOS
 - Protocol stacks
 - Debugger API, Simulation Control API, Peer API

Platform-based design

- **Platform includes hardware, supporting software.**
- **Two stage process:**
 - **Design the platform.**
 - **Use the platform.**
- **Platform can be reused to host many different systems.**



PLATFORM DESIGN

- Turn system requirements and software models into detailed requirements.
 - Use profiling and analysis tools to measure existing executable specifications.
- Explore the design space manually or automatically.
- Optimize the system architecture based on the results of simulation and other steps.
- Develop hardware abstraction layers and other software.

SOLUTIONS TO EMBEDDED SOFTWARE

- System level design language
 - ADL
 - Open SystemC initiative
 - HDL
 - VSIA (Virtual Socket Interface Alliance)
System Level Design
 - More abstraction level: UML, UML-RT
- Toolkit
 - Compilers
 - Assemblers
 - Profilers
 - Debuggers
 - Simulators

COMPONENTS OF EMBEDDED SYSTEMS

(SOFTWARE)

- Main application software:

- performs series of tasks or multiple tasks concurrently.
- constrained due to low memory, low processing power requirement etc.

- Real Time Operating System (RTOS):

- supervises the application software.
- provides a mechanism to let the processor run a process as per scheduling.
- performs context-switching between various processes (tasks).
- organizes access to a resource in sequence of the series of tasks of the system.
- schedules their working and execution by following a plan to control the latencies and to meet the deadlines.
- sets the rules during the execution of the application software.

SOFTWARE FOR EMBEDDED SYSTEMS

- ❑ Specific to a given application.
- ❑ Processor processes instructions and data and in the final stage these are placed in the memory (ROM) for all the tasks that have to be executed.
- ❑ Final stage software is called ROM image.
- ❑ Bytes at each address must be defined for creating the ROM image. By changing the image the same hardware platform will work differently and can be used for entirely different applications or for new upgrades of the same system.
- ❑ The ROM image consists of the boot-up program, stack(s) address pointer(s), program counter address pointer(s), application tasks, Interrupt Service Routines (ISRs), input data and vector addresses.
- ❑ Assembly Language: specific to a processor or its family. A thorough understanding of processor's instruction set is required and is cumbersome when large programs are to be written.

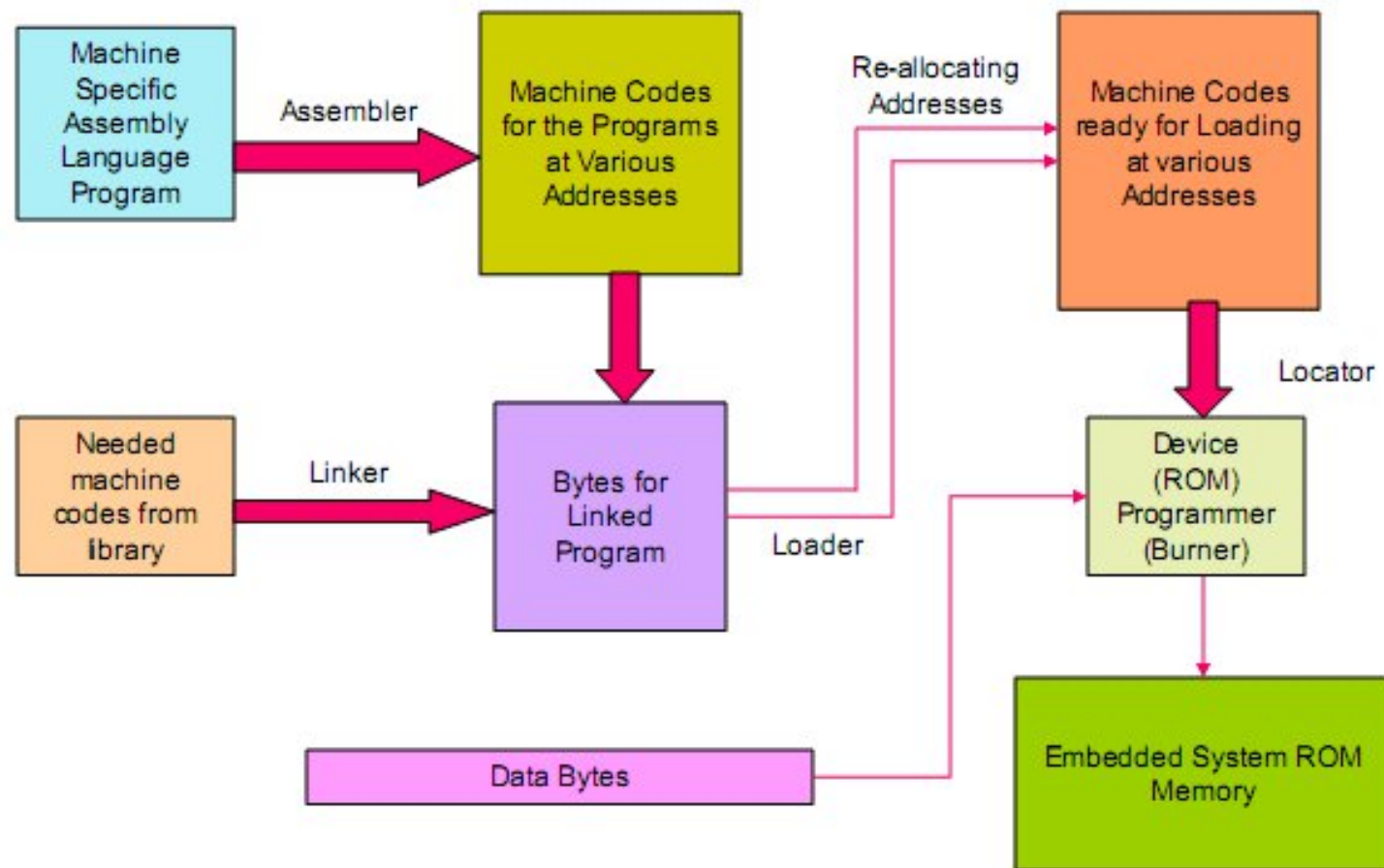
SOFTWARE FOR EMBEDDED SYSTEMS

- ❑ **Instructions:** A binary pattern designed inside a microprocessor to perform a specific function.
- ❑ **Instruction Set:** An entire group of instructions.
- ❑ **Mnemonic:** A combination of letters to suggest the operation of an instructions (Mnemonic = mindful)
- ❑ **Assembly Language Program:** A set of instructions written in mnemonics of a given microprocessor / microcontroller or its family such as 8085, 8086 etc.
- ❑ **Machine Language:** The binary medium of communication with a computer through a designed set of instructions.

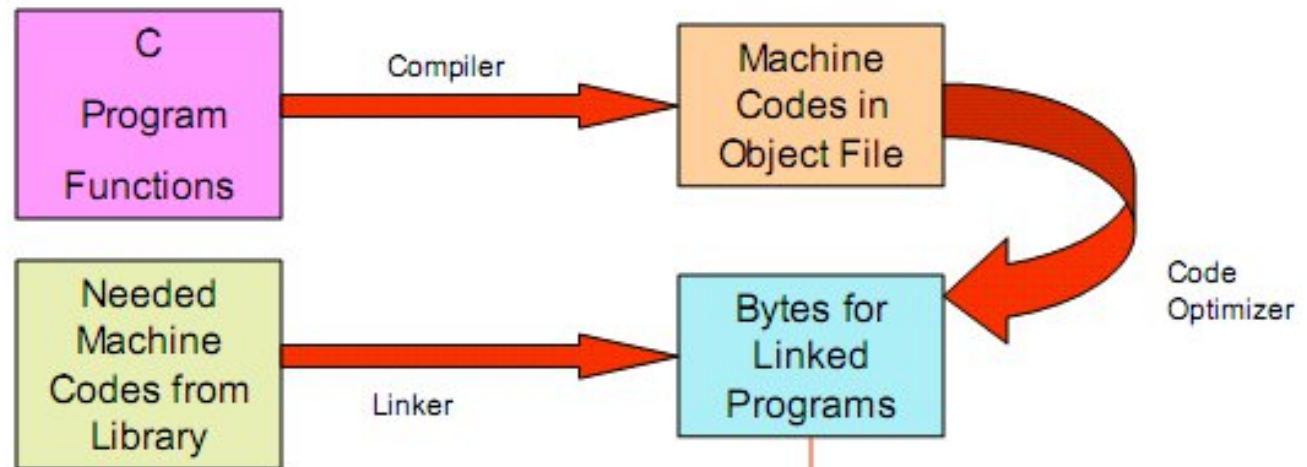
SOFTWARE FOR EMBEDDED SYSTEMS

- ❑ **Low-Level Language:** A medium of communication with a computer in which programs are written in mnemonics. An assembly language is specific to a given computer.
- ❑ **High-Level Language:** A medium of communication that is independent of a given computer. Programs are written in English-like words and they can be executed on machine using a translator (a compiler or Interpreter).
- ❑ **Compiler:** This is a program that translates English-like words, of a high-level language in the machine language of a computer.
- ❑ **Interpreter:** An interpreter is a program that translates the English-like statements of a high-level language into the machine language of a computer.
- ❑ **Assembler:** A computer program that translates an assembly language program from mnemonics to the binary machine code of a computer.
- ❑ **Linker:** links the machine codes generated by assembler with the other required assembled codes. Generates a file known as linked file or executable '.exe' file to run on a computer.
- ❑ **Loader:** reallocates the codes after finding the physical RAM addresses available at a given instant. It is a part of the OS and places codes into the memory after reading the '.exe' file.
- ❑ **Locator:** reallocates the linked file and creates a file for permanent location of codes.

From ALP to ROM Image



Software in High Level Language



Different Program Layers in the Embedded Software (e.g. 'C')

Preprocessor Commands

Main Function

Interrupt Service Routines

Tasks 1.....N

Kernel and Scheduler

Standard Library Functions Including Network Protocol Functions for Sending Stack and Receiving Stack

Machine codes ready for loading at various addresses + ROM Programmer

Embedded System ROM Memory

MIDDLEWARE

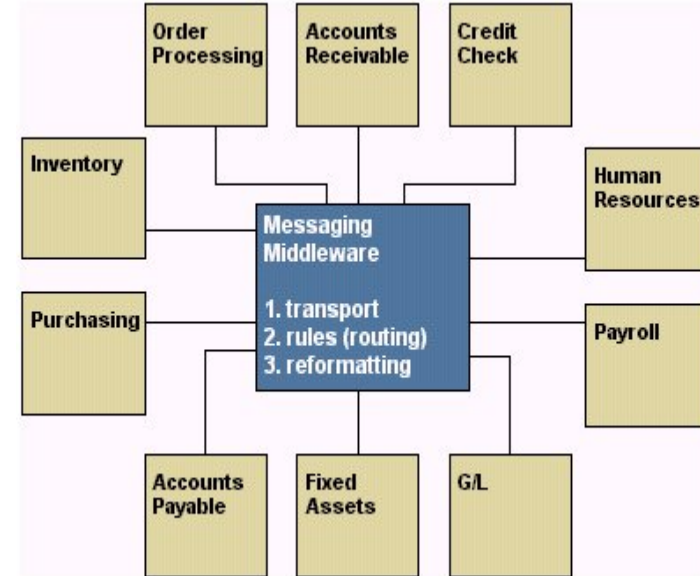
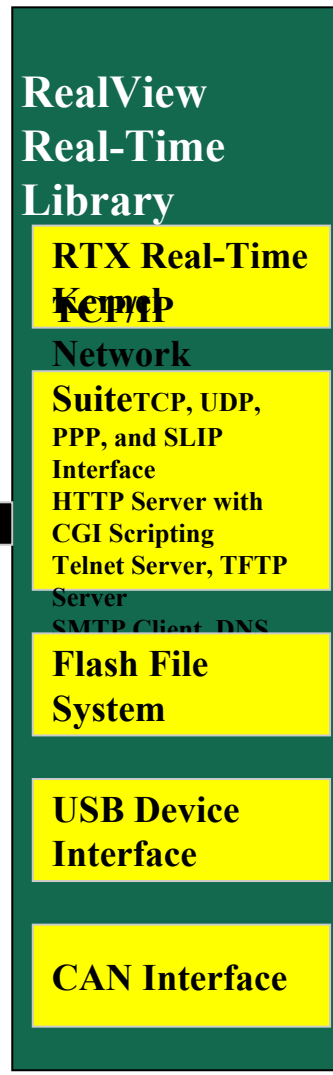
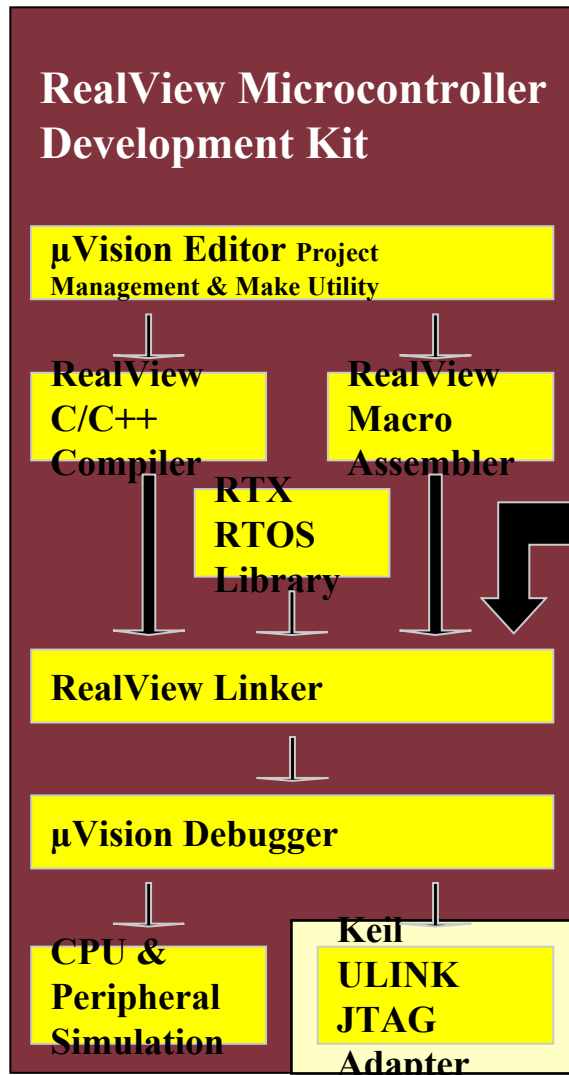
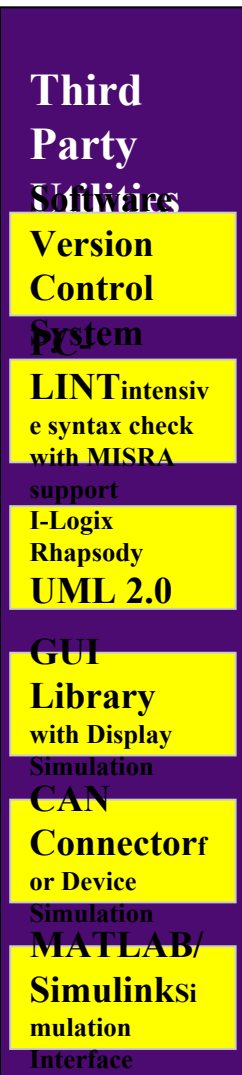
- Provides services to software applications beyond those available from the OS, thus can be described as "software glue".

- Commonly enables two separate programs to communicate and share data (e.g., software on a Web server Commonly enables two separate programs to communicate and share data (e.g., software on a Web server that enables the HTTP Commonly enables two separate programs to communicate and share data (e.g., software on a Web server that enables the HTTP server to interact with scripting engines like PHP Commonly enables two separate programs to communicate and share data (e.g., software on a Web server that enables the HTTP server to interact with scripting engines like PHP or ASP). It runs quietly in the background, but serves as an important "glue" between the server applications.

- Also helps different applications communicate over a network Also helps different applications communicate over a network. It enables different protocols Also helps different applications communicate over a network. It enables different protocols to work together by translating the information passed from one system to another. This type of middleware may be installed as a "Services-Oriented Architecture" (SOA Also helps different applications communicate over a network. It enables different protocols to work together by translating the information passed from one system to another. This type of middleware may be installed as a "Services-Oriented Architecture" (SOA) component on each system on the network. When data is sent *

between these systems, it is first processed by the middleware component, then

MIDDLEWARE



Messaging Middleware

Messaging middleware provides a common interface and transport between applications. If the target machine is down or overloaded, it stores the data in a message queue until it becomes available. The messaging system may contain business logic that routes messages to the appropriate destinations and reformats the data as well. Messaging middleware is

Stellarisware as middleware for TI ARM

Free license and royalty-free source code:

- Peripheral Driver Library
- Graphics Library
- USB Library
- Boot Loader
- IEC 60730 Library

Enabling our customers with the ability to rapidly develop and deploy their products at competitive costs yielding a higher overall value for the Stellaris solution!

KERNEL, OS AND RTOS

- ❑ **Operating System (OS)** allocate resources in the computing system among programs that request them. It also schedules processes to meet performance requirements and provide services beyond CPU scheduling.
- ❑ **Multitasking:** Processing codes for the different tasks as directed by the scheduler.
- ❑ **Kernel:** A program with functions for memory allocation and de-allocation, task scheduling, inter-process communication, effective management of shared memory access by using the signals, exception (error) handling signals, queues, mailboxes, pipes and sockets, I/O management, interrupts control, device drivers and device management.
- ❑ **Real Time Operating Systems (RTOS):** An OS for real-time programming and scheduling, process and memory manager, device drivers, device management and multitasking.
- ❑ RTOS has to be **scalable** in embedded systems.
- ❑ **Scalable OS** is one in which memory is optimized by having only that part of features that are needed and is associated with the final software.

SOFTWARE DESIGN USING RTOS

Software Design using RTOS

- ☐ Each Task has a distinct memory location
- ☐ Tasks are made up of one or more procedures
- ☐ Shared Memory Architecture (Graphic chips doesn't have own dedicated memory, instead shares main system RAM with CPU and other components)
- ☐ Distributed memory Architecture – UMA/ NUMA
- ☐ Scheduling done by OS - takes care of constraints

- ☐ RTOS kernel schedules transitions of task
 - Ready state -> Running State
 - Block state -> Running State (pre-emptive scheduling)
 - Tasks can enter into ready state at any time
 - At a time only one task will be in running state

- ☐ RTOS also controls inter-process communication

DEVICE DRIVERS

- Control/operate a particular type of hardware device.
 - When a calling program invokes a routine in the driver, the driver issues commands to the device. Once the device sends data back to the driver, the driver may invoke routines in the original calling program.
 - Software for controlling, receiving and sending a byte or a stream of bytes from/to a device.
- Drivers are hardware-dependent and OS-specific. They usually provide the interrupt handling required for any necessary asynchronous time-dependent hardware.
- Acting as translator between a hardware device and the applications or OS's that use it.

DEVICE DRIVER CONCEPTS

Device Driver: a software for *controlling, receiving and sending* a byte or a stream of bytes from or to a device. Drivers control three functions

- (i) **Initializing** that is activated by placing appropriate bits at the control register or word.
- (ii) **Calling** an ISR on interrupt or on setting a status flag in the status register and run (drive) the ISR.
- (iii) **Resetting** the status flag after interrupt service. (e.g. device drivers for accessing a serial port, keyboard, mouse etc.)

Device Driver ISR: Consider a device driver for handling port inputs. It does the following:

- (i) Resets the device-buffer-full flag (in status register) and thus prepares the device for the next read. [The flag must be reset when the corresponding ISR starts execution.]
- (ii) Reads the input buffer (s) by emptying the buffer and storing the byte (s) in memory or using the bytes received as per system requirement.

Before driving the device must be configured. A device driver routine differs from other ISRs.

Timer Device Driver: Following steps are needed

- (i) Write in a register that holds the timer reset value, the number of count-inputs, *numTicks* for RTC
- (ii) Write in status register the timer status flag (s) = reset (in case the device does reset automatically this flag (s) on a read of the status flag (s)).
- (iii) Write each bit present in the control register (s). Write interrupt secondary and primary level enable bits = true in control register, write other bits according to their uses.

API - APPLICATION PROGRAMMING INTERFACE

• API is a set of routines API is a set of routines (also data structures API is a set of routines (also data structures, object classes API is a set of routines (also data structures, object classes, variables, library), protocols API is a set of routines (also data structures, object classes, variables, library), protocols, and tools for building software applications API is a set of routines (also data structures, object classes, variables, library), protocols, and tools for building software applications. A good API makes it easier to develop a program API is a set of routines (also data structures, object classes, variables, library), protocols, and tools for building software applications. A good API makes it easier to develop a program by providing all the building blocks. A programmer then puts the blocks together.

• Most OS's, such as MS-Windows Most OS's, such as MS-Windows, provide an API so that programmers can write applications consistent with the operating environment. Although APIs are designed for programmers, they are ultimately good for users because they guarantee that all ^{*} programs using a common API will have similar interfaces. This makes it easier for users to learn

API - APPLICATION PROGRAMMING

INTERFACE

application



User Interface

The user talks to the computer via the commands, menus and buttons on the user interface.



Application

Application Programming Interface

The application talks to the operating system via the API, which defines the parameters that are passed between them.

Operating System

Application

Application Binary Interface

The application talks to the computer via the operating system APIs and by being in the machine language of the computer it is running in. The combination of OS and machine is the ABI.

Operating System

Machine Language

E-mail, fax, client/server

Protocol

Applications use protocols to connect to a network. The network software uses additional protocols, all of which make up the communications interface. See OSI model

E-mail, fax, client/server

	A	B	C	D	E	F	G
1	50						
2	100						
3	50						
4	100	1000					
5	50						
6	190						
7	500						
8							
9							
10							
11							
12							
13							
14							
15							
16							
17							

ABI

Machine language

The repertoire of machine language instructions that a computer can follow. It is a major architectural component and is either built into the CPU or into microcode.

CPU Examples

x86 (PC)
PowerPC (Mac)
IBM AS/400
IBM S/390
Sun SPARC
Alpha
MIPS

+

API

A language and message format used by an application program to communicate with a program that provides services for it.

OS Examples

Windows
Mac OS
DOS
OS/2
UNIX
Linux

SOFTWARE TOOLS

Editor

Interpreter / Compiler

Assembler / Cross-assembler

Simulator

RTOS

Logic Analyzer

Prototyper

Locator

EDA packages like Altium, Orcad, CAE, CAD, CAM

IDE like KeilC, AVR Studio,

Etc...

PROGRAMMER'S VIEW

- Programmer doesn't need detailed understanding of architecture
 - Instead, needs to know what instructions can be executed
- Two levels of instructions:
 - Assembly level
 - Structured languages (C, C++, Java, etc.)
- Most development today done using structured languages
 - But, some assembly level programming may still be necessary
 - Drivers: portion of program that communicates with and/or controls (drives) another device
 - Often have detailed timing considerations, extensive bit manipulation
 - Assembly level may be best for these

ISA (INSTRUCTION SOFTWARE ARC) AND IMPLEMENTATION

- ISA is the hardware/software interface
 - Defines set of programmer visible state
 - Defines instruction format (bit encoding) and instruction semantics
 - Examples: *MIPS*, *x86*, *IBM 360*, *JVM*
- Many possible implementations of one ISA
 - 360 implementations: *model 30* (c. 1964), *z990* (c. 2004)
 - x86 implementations: *8086* (c. 1978), *80186*, *286*, *386*, *486*, *Pentium*, *Pentium Pro*, *Pentium-4* (c. 2000), *AMD Athlon*, *Transmeta Crusoe*, *SoftPC*
 - MIPS implementations: *R2000*, *R4000*, *R10000*, ...
 - JVM: *HotSpot*, *PicoJava*, *ARM Jazelle*, ...

OPERATING SYSTEM

- Optional software layer providing low-level services to a program (application).
 - File management, disk access
 - Keyboard/display interfacing
 - Scheduling multiple programs for execution
 - Or even just multiple threads from one program
 - Program makes system calls to the OS

```
DB file_name "out.txt" -- store file name

MOV R0, 1324           -- system call "open" id
MOV R1, file_name      -- address of file-name
INT 34                 -- cause a system call
JZ  R0, L1              -- if zero -> error

    . . . read the file
JMP L2                  -- bypass error cond.
L1:
    . . . handle the error

L2:
```

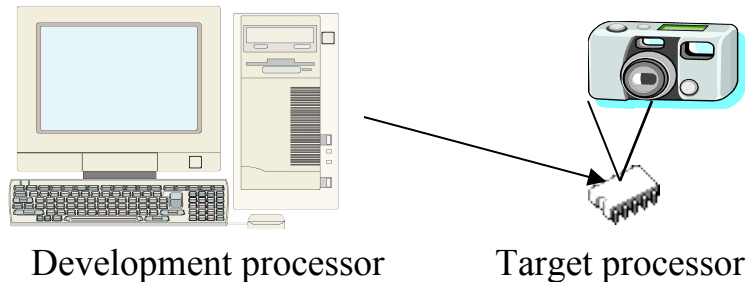
DEVELOPMENT ENVIRONMENT

- Development processor

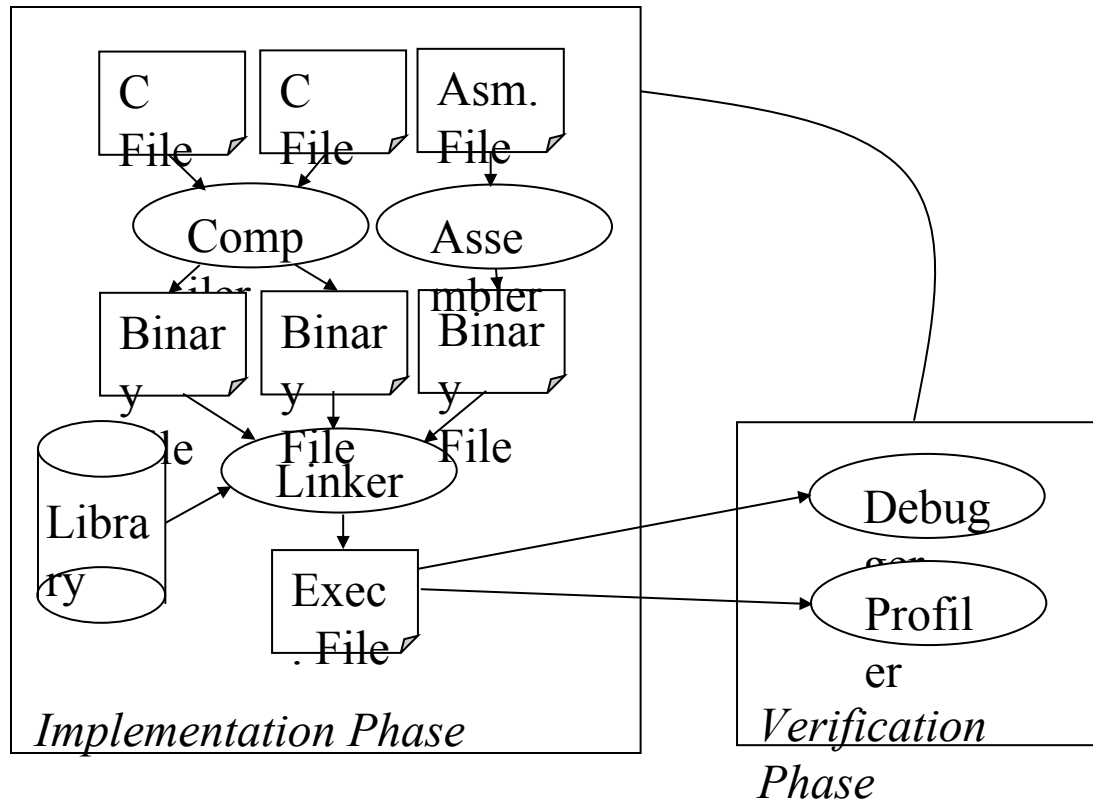
- The processor on which we write and debug our programs
 - Usually a PC

- *Target processor*

- The processor that the program will run on in our embedded system
 - Often different from the development processor



SOFTWARE DEVELOPMENT PROCESS



- Compilers

- Cross compiler

- Runs on one processor, but generates code for another

- Assemblers

- Linkers

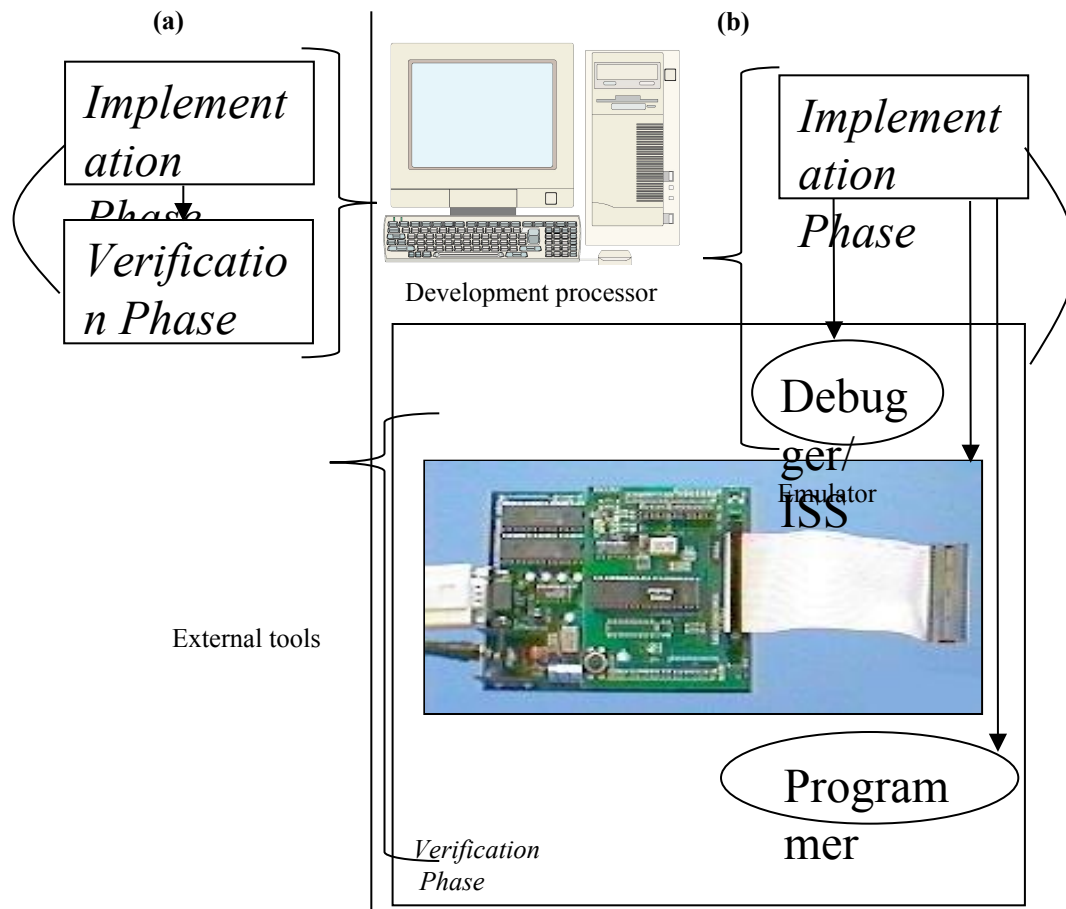
- Debuggers

- Profilers

RUNNING A PROGRAM

- If development processor is different than target, how can we run our compiled code? Two options:
 - Download to target processor
 - Simulate
- Simulation
 - One method: Hardware description language
 - But slow, not always available
 - Another method: *Instruction set simulator (ISS)*
 - Runs on development processor, but executes instructions of target processor

TESTING AND DEBUGGING



•ISS

- Gives us control over time – set breakpoints, look at register values, set values, step-by-step execution, ...
- But, doesn't interact with real environment
- Download to board
 - Use device programmer
 - Runs in real environment, but not controllable
- Compromise: emulator
 - Runs in real environment, at speed or near
 - Supports some controllability from the PC

PROGRAMMER CONSIDERATIONS

- Program and data memory space
 - Embedded processors often very limited
 - e.g., 64 Kbytes program, 256 bytes of RAM (expandable)
- Registers: How many are there?
 - Only a direct concern for assembly-level programmers
- I/O
 - How communicate with external signals?
- Interrupts

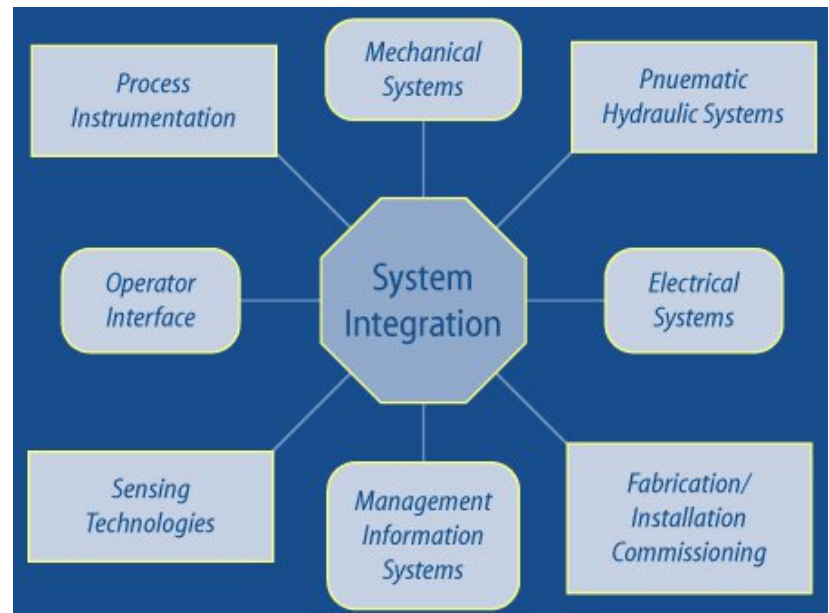
DESIGN STEPS

- Specification
- Circuit Design
- Printed Circuit Board Layout
- Software design
- Integration and testing
- Pilot Run
- Production
- Operation and Maintenance

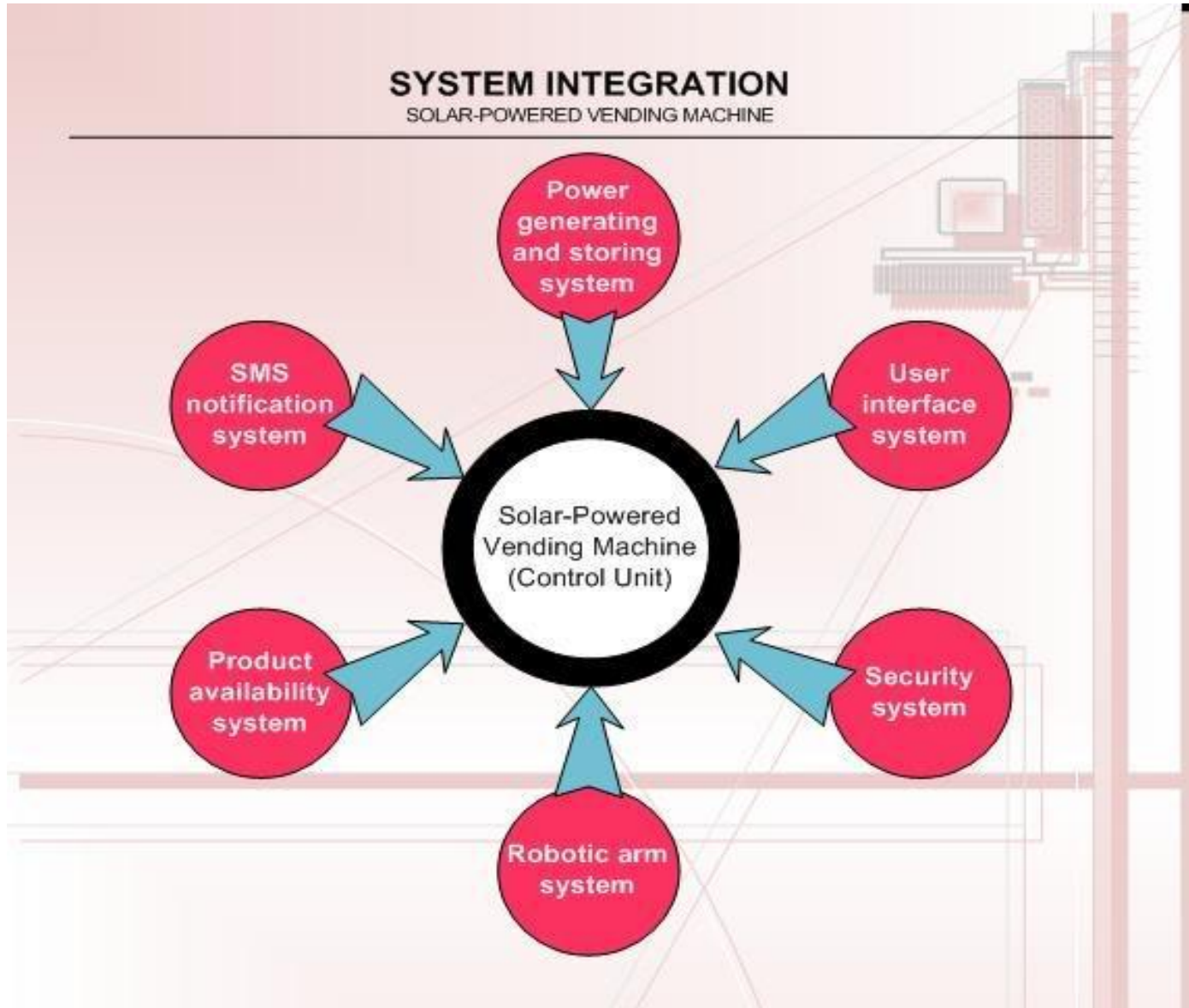
Source

SYSTEM INTEGRATION - SI

- **System integration** is the bringing together of the component subsystems into one system and ensuring that the subsystems function together as a system.
- In IT field, **systems integration** is the process of linking together different computing systems and software applications physically or functionally, to act as a coordinated whole.

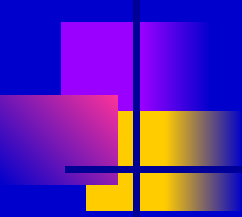


SYSTEM INTEGRATION - EXAMPLE



SYSTEM INTEGRATION AND TESTING

- **System testing** of SW/HW is testing conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements.
 - System testing falls within the scope of black box testing (White box testing is the testing of the internal workings or code of a software application. In contrast, black box or system testing is the opposite), and as such, should require no knowledge of the inner design of the code or logic.
 - Involves integrating and testing all the components/modules/subsystems developed in the previous phase as a complete system.
 - System testing is performed on the entire system in the context of a Functional Requirement Specification (FRS) and/or a System Requirement Specification (SRS). It tests not only the design, but also the behaviour and even the believed expectations of the customer. It is also intended to test up to and beyond the bounds defined in SW/HW requirements specification(s)



Benefits

- Less components
 - Component costs
 - Board size and cost
 - Assembly and testing costs
- Less inter-chip interconnects
 - Reliability
 - Power consumption
 - Board design, fabrication and assembly costs
- Smaller system volume (in cm²) and weight
 - Higher integration rate
 - Smaller case costs
 - Smaller transport costs
- In high volumes (in pcs), also lower circuit costs

DESIGN STEPS

- **Specification**
- **Circuit Design**
- **Printed Circuit Board Layout**
- **Software design**
- **Integration and testing**
- **Pilot Run**
- **Production**
- **Operation and Maintenance**

Source

PILOT RUN

- To test the product further, a Pilot run normally follows the prototyping stage
 - A **prototype** is an early sample or model built to test a concept/process/product or to act as a thing to be replicated or learned from.
 - A prototype is designed to test and trial a new design to enhance precision by system analysts and users.
 - Prototyping serves to provide/verify specifications for a real, working system rather than a theoretical one.
 - Prototype circuit boards, commonly referred to as PCBs, are a crucial link in the process from design conception to finished product. They allow for the user to test if a given design really works and to fine tune improvements before full scale production.
 - Without prototype circuit boards, there would be unacceptable risks and inefficiencies involved with the mass production of electronic circuit boards.
- Small quantity of units are field trialed in a beta test
- Opportunity to assess the manufacturability of the design, and the usability of the documentation

Source

DESIGN STEPS

- **Specification**
- **Circuit Design**
- **Printed Circuit Board Layout**
- **Software design**
- **Integration and testing**
- **Pilot Run**
- **Production**
- **Operation and Maintenance**

Source

PRODUCTION

- Following the pilot run there will likely be changes to the firmware, and possibly the circuit design, as the unit develops into a stable, final product
- This process is controlled by ECO's and version numbers
- The cost, style of design of the final production, is heavily influenced by the number of units manufactured

Source

DESIGN STEPS

- **Specification**
- **Circuit Design**
- **Printed Circuit Board Layout**
- **Software design**
- **Integration and testing**
- **Pilot Run**
- **Production**
- **Operation and Maintenance**

Source

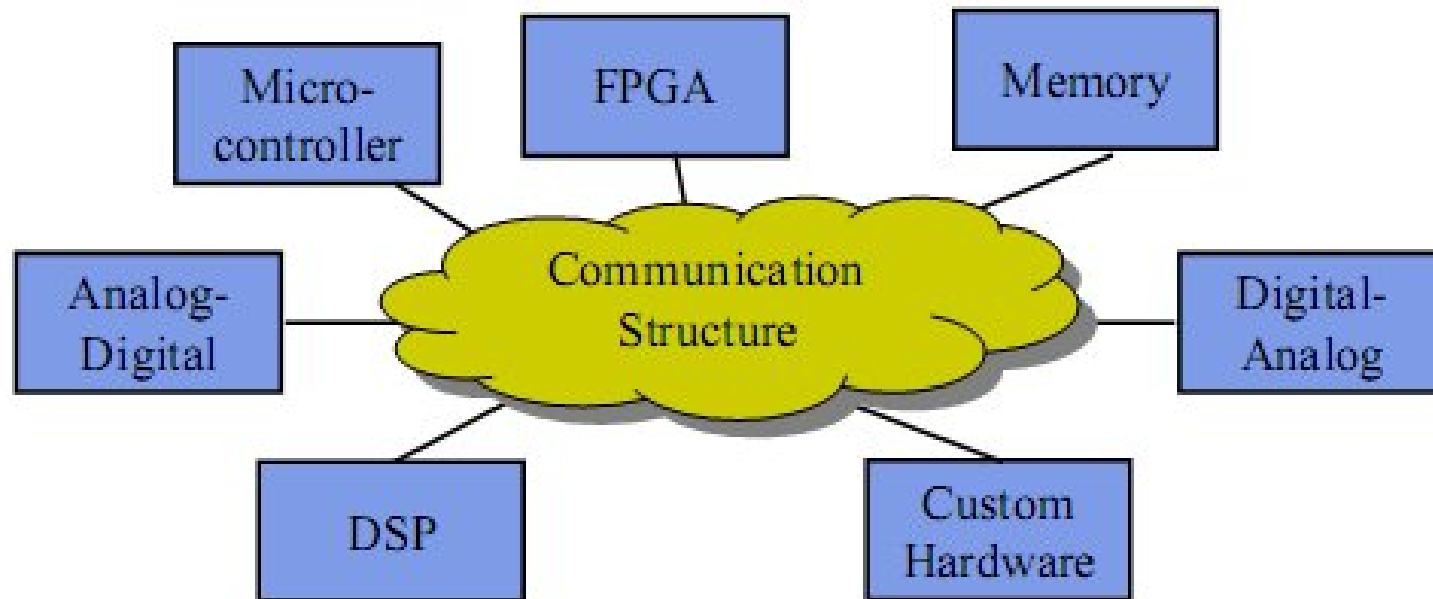
OPERATION AND MAINTENANCE

- This is often the final phase, in which the completed product is handed over to the client after alpha, beta testing.
- After the product has been deployed on the client site, it is the duty of the development team to undertake routine maintenance activities by visiting the client site.
 - If the customer suggests changes or enhancements, the development team take action until the contract/customer's satisfaction
 - Meet the changing customer needs
 - Adapted to accommodate changes in the external environment
 - Correct errors and oversights previously undetected in the testing phases
 - Enhancing the efficiency of the product/SW
- **Maintenance, repair, and operations (MRO)** involves fixing any sort of mechanical) involves fixing any sort of mechanical, plumbing) involves fixing any sort of mechanical, plumbing or electrical) involves fixing any sort of mechanical, plumbing or electrical device) involves fixing any sort of mechanical, plumbing or electrical device should it become out of order or broken (known as repair/unscheduled maintenance). It also includes performing routine actions which keep the device in working order (known as scheduled maintenance) involves fixing any sort of mechanical, plumbing or electrical device should it become out of order or broken (known as repair/unscheduled maintenance). It also includes

Introduction to SoC Architecture

System-on-Chip Architectures

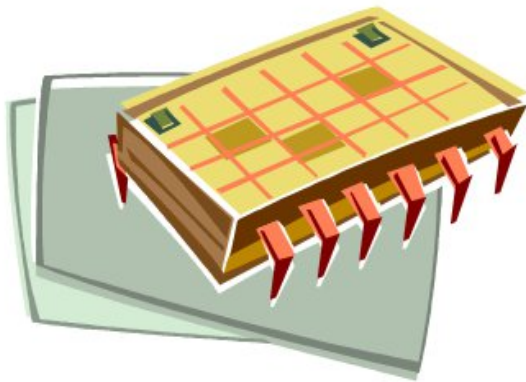
- A *system-on-chip architecture* integrates several heterogeneous components on a single chip



- A key challenge is to design the communication between the different entities of a SoC in order to minimize the communication overhead

System On Chip

SoC = A Complete System on a Single Chip Die



Microprocessor

On-Chip Memory

ASIC Core

I/O Devices

Analog/RF Devices

Small Volume

Low Power

Performance

Functionality

Introduction to SoC

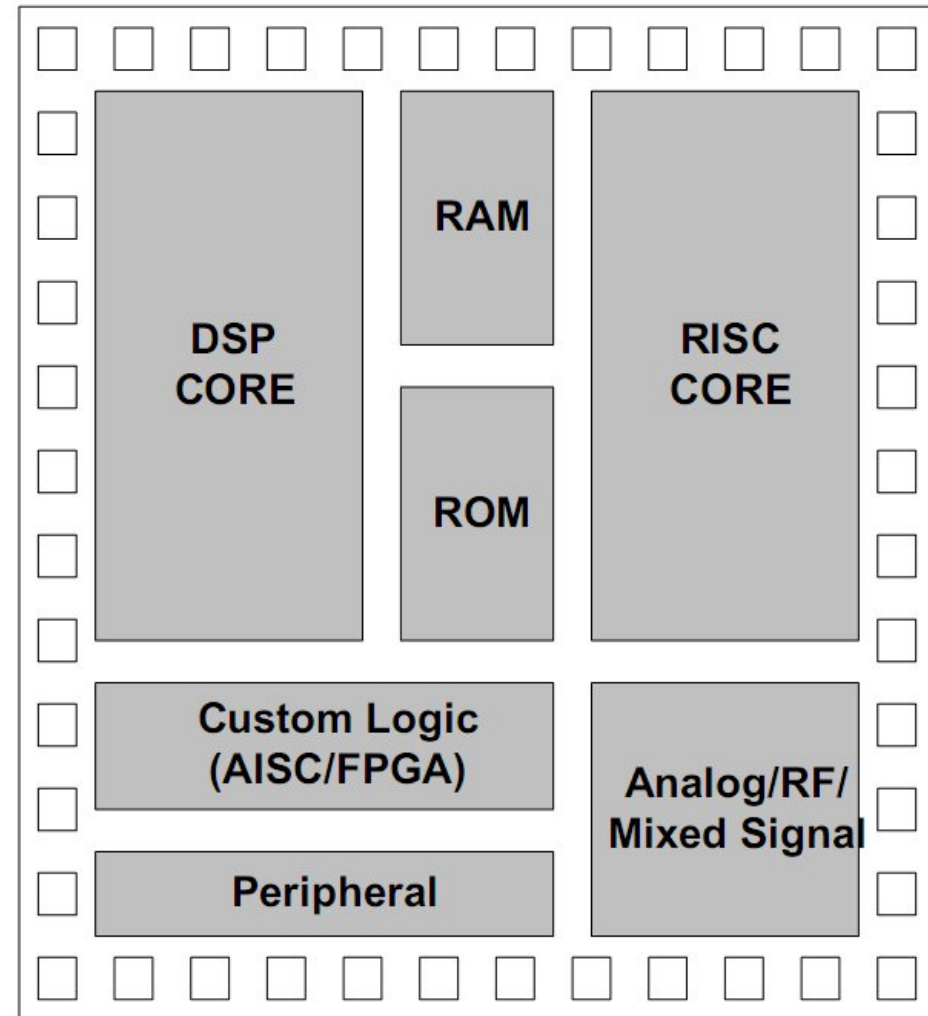
- System
 - Collection of all kinds of components and/or subsystems appropriately interconnected to perform the specified functions for end users.
- System-On-a-Chip (SoC)
 - A complex IC that integrates the major functional elements of a *complete end-product* into a *single chip*.
- General SoC Organization:
 - Hardware
 - at least one *programmable processor*
 - *on-chip memory*
 - *accelerating function* units implemented in hardware.
 - interfaces to *peripheral* devices
 - interfaces to *real world (analog & O/MEMS components)*
 - *Software:*
 - Real time OS
 - Device Drivers

Introduction to SoC

- System Viewpoint
 - A collection of all kinds of components and/or subsystems that are appropriately interconnected to perform the specified functions for end users.
- A SoC design is a “product creation process”
 - Starts at identifying the end-user needs
 - Ends at delivering a product with enough functional satisfaction to overcome the payment from the end-user

Why SoC?

- Cost
 - Low cost
- Compactness
 - Portable, hand-held
- Increase Performance
 - Quality
- Low Power Consumption
- Design reusable

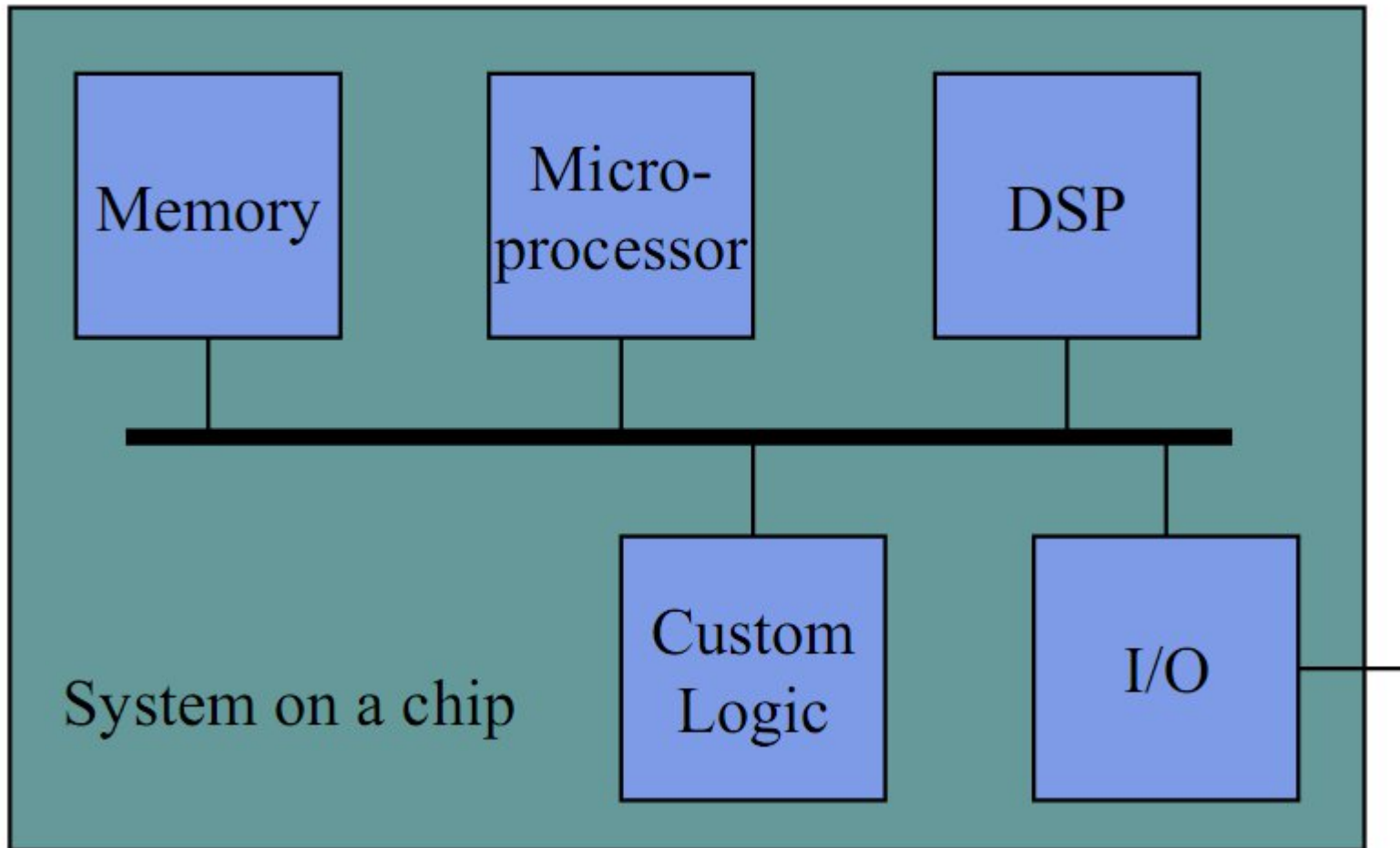


Benefits of Using SoC

- Reduced size
- Reduced overall system cost
- Lower power consumption
- Increased performance

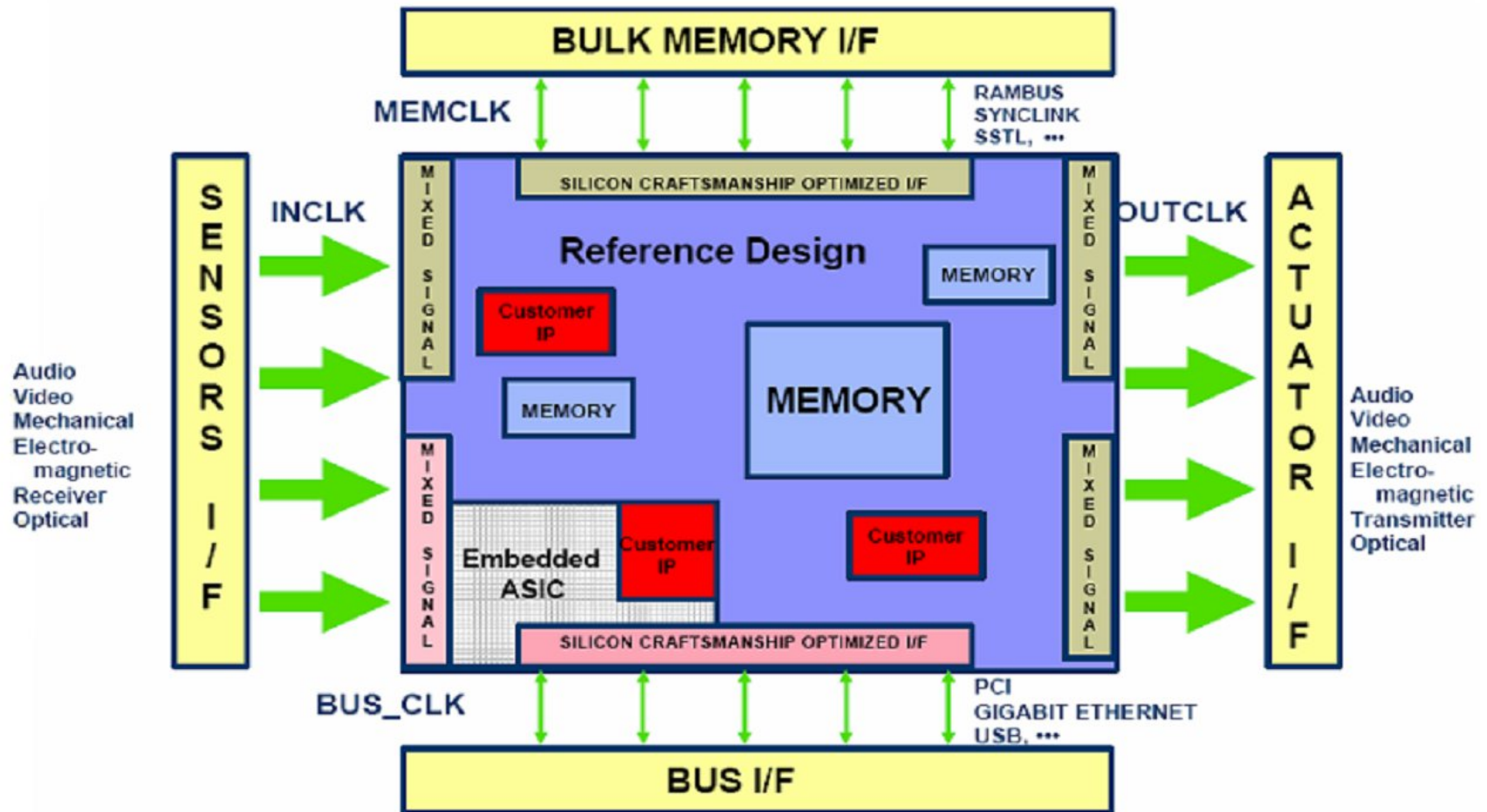
System-on-Chip Architecture:

A bus-based SoC



A Typical SoC Design Example

◆ The hardware view of soc



The History of the SoC

- The History of the System-on-Chip
 - o The Glorious Hope (1995-1999)
 - o The Reign of Terror: Reality & disappointment (1999-2001)
 - o Thermidor: Platform transformation (2000-2002)
 - o L'Avenir: The radiant future?

Chip Industry Foundations

Technology Development

Transistor



Dec 1947-Bell Labs

IC



1958 - TI

Standard MOS

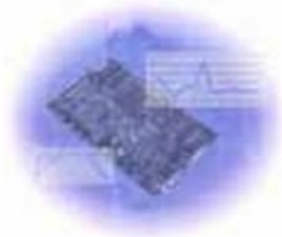


1967 - Fairchild

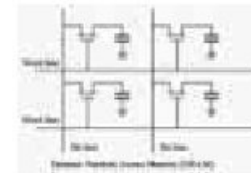
70s - Waves of New Products



Microprocessors



Digital Signal Processors



DRAM



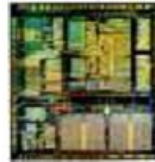
EPROM

Scaling for Performance

80s & 90s industry growth and scaling in computing power



1982 - Intel 80286
134 thousand transistors
12MHz; 68.7 mm²



1985 - Intel 80386
275 thousand transistors
33MHz; 104 mm²



1989 - Intel 80386
1.2 million transistors
50MHz; 163 mm²



1993 - Intel Pentium
3.1 million transistors
66MHz; 264 mm²



1997 - Intel Pentium II
7.5 million transistors
300MHz; 209 mm²



1999 - Intel Pentium III
28 million transistors
733MHz; 140 mm²



2000 - Intel Pentium 4
42 million transistors
1.5GHz; 224 mm²

IP

- A predefined, designed/verified, reusable building block for System-on-Chip
- Software IP, Silicon IP (Soft IP, Hard IP, ...)
- IP types
 - Foundation IP (cell library, gate array)
 - Standard IP (MPEG2/4, JPEG, USB, IEEE 1394, PCI...)
 - Star IP (ARM, MIPS, Rambus, ...)
- Ancillary characteristics
 - Deliverable at certain level, software/hardware interfaces
 - Modeling at different levels
 - Customizable, Configurable, Parameterizable

IP Library

- Synthesizable Hardware IPs

- AHB Arbiters
- AHB2AHB Bridge
- AHB2APB Bridge
- AHB Monitor
- AHB Matrix
- I2C/I2S
- UART Controller

- TFT- LCD Controller
- SDRAM
- Static Memory Controller
- DMA Controller
- Interrupt Controller
- On- chip memory controller

- External Bus Interface
- Multi- Channel Counter
- Multi- Channel Timer
- DCT/IDCT Accelerator
- H.264 ME/ITQ

- Simulation Models

- Virtual UART Terminal
- Virtual TFT- LCD Panel
- ARM7 Simulation Model

- Memory Models
Asynchronous SRAM, SSRAM, ZBT SRAM, DRAM,
Dual Port SRAM, SDRAM, Flash Memory, etc

Criteria in Selecting IP's

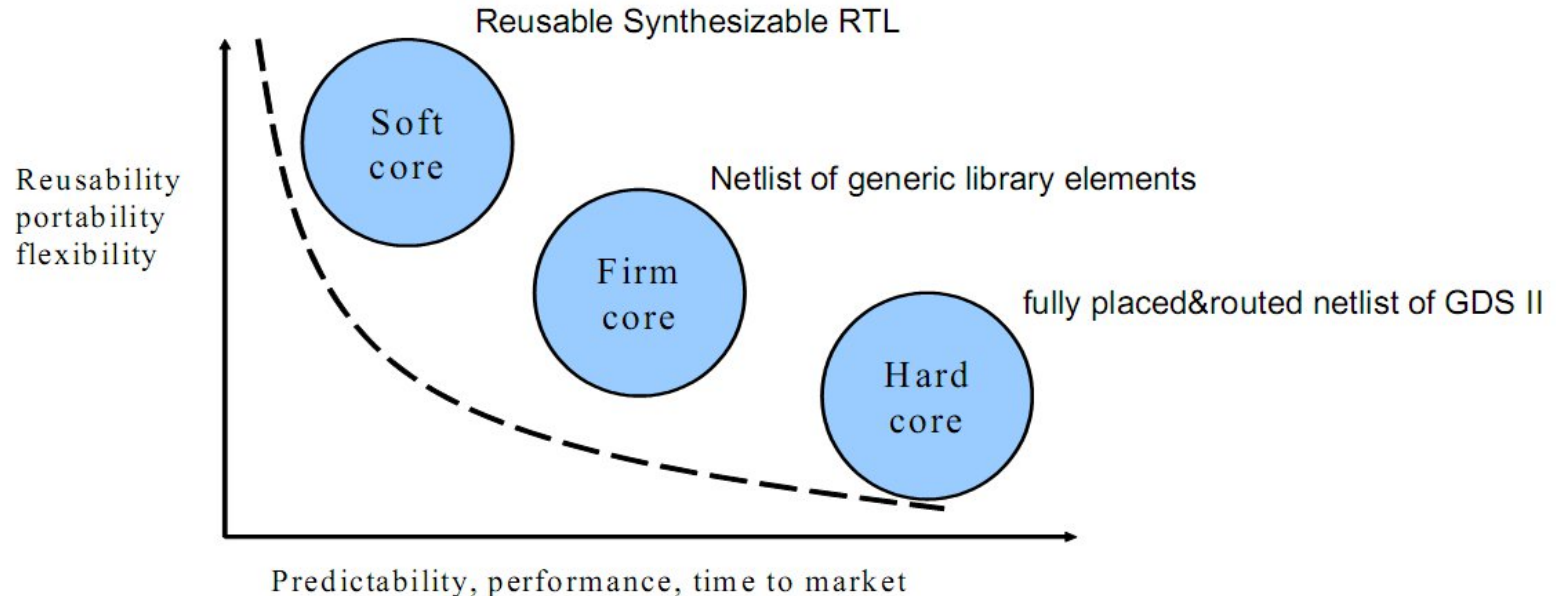
- Processor IP selection criteria
 - Power, performance, area, cost
 - Flexibility
 - Hardness (hard IP vs. soft IP)
 - Available system software
 - Development environment
 - Simulation model
 - Support library
 - Support OS
 - Inter-operability with other IP's

IP-based Design

Component-level Reuse

- ❑ Reuse of cores or IPs from in-house or third parties
- ❑ Programmable cores: ARM, Teak, etc
 - ❑ Software Development Kit (SDK)
- ❑ ASIC Cores: Application specific hardware engine
 - ❑ RTLs, Testbenches, Synthesis scripts

Forms of IPs



Problem of IP-based Design

Heterogeneity

- ❑ Complex SoCs are naturally a heterogeneous system
 - ❑ ASIC H/Ws, RISC cores, DSP cores
 - ❑ C/C++ softwares, assembly codes
 - ❑ On-chip/Off-chip memories
- ❑ Integration of heterogeneous IPs is also a difficult and time consuming work

Multi-domain, System-level Knowledges

- ❑ A system design team should manage many different knowledge and skills
 - 1) software programming skills,
 - 2) hardware description skills,
 - 3) system integration and verification skills
 - 4) virtual/rapid prototyping, chip back-end design skills

Tremendous Design Complexities

- ❑ Today's SoCs are more than 10M gates
- ❑ How can we describe/integrate/verify more than 10M gates from scratch??

Platform

- A fully defined bus structure and a collection of IP blocks
- A design methodology to support the feature of “Plugging and Playing”
- The definition of a platform is the result of a trade-off process involving reusability (programmability and configurability), cost and performance optimization.
- Enhance the differentiation

Platform-based Design

What is a platform?



Why a platform?

- ☐ Constructed by system experts for a specific application domains
 - ☐ The system expertise can capture good solutions and provides abstracted programming interfaces for the future customization
- ☐ Customized by application experts for a specific product
 - ☐ The most application engineers do not have expertise on the system architecture

Specification Languages

- HDL-based specification language
 - VHDL, Verilog
 - Benefit from existing design flow
 - Good for hardware description
- HLL-based specification language
 - SystemC, SpecC
 - Typically based on C/C++
 - Good for software/system description
- Mixed form
 - Superlog, CoWareC

Core Technologies

- IP Development
- System Architecture
- SoC Verification
- Embedded Software
- High Speed/Low Power Design

System Architecture Design

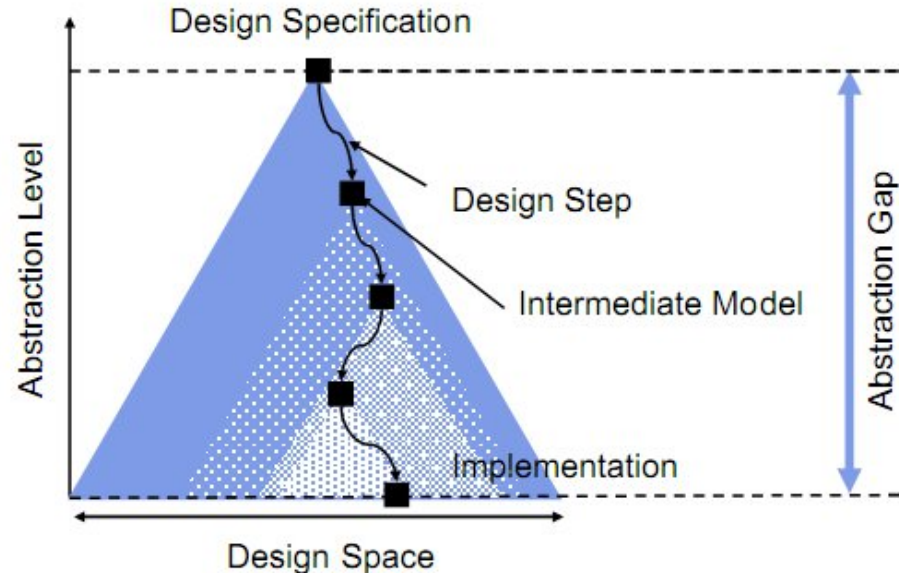
- Specification, Requirement, Functionalities
→ Architecture
 - C-level design, SystemC description and simulation
 - Advantages
 - Broader design space
performance, power, cost tradeoff
→ scalability, good for time-in-market
 - Early verification
module well-defined, partition, refinement
→ necessary for time-to-market
-

Interfacing IPs

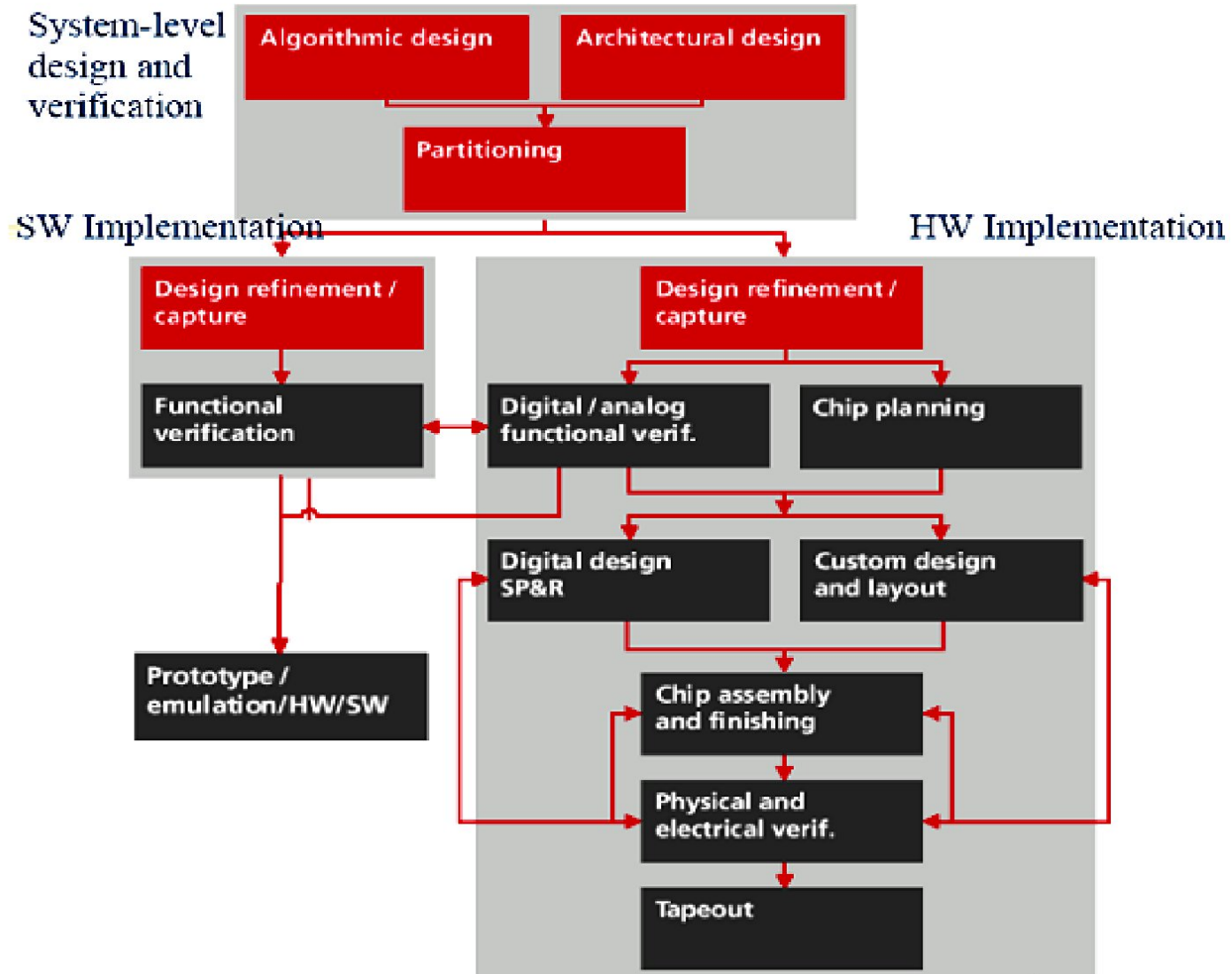
- PCB (=Processor + Peripheral) shrinks to SOC
- Interface between HW and HW
 - FIFO-based interface: I/O
 - On-chip bus is required
- On-chip bus: System Bus & Peripheral Bus

SoC Design Tasks

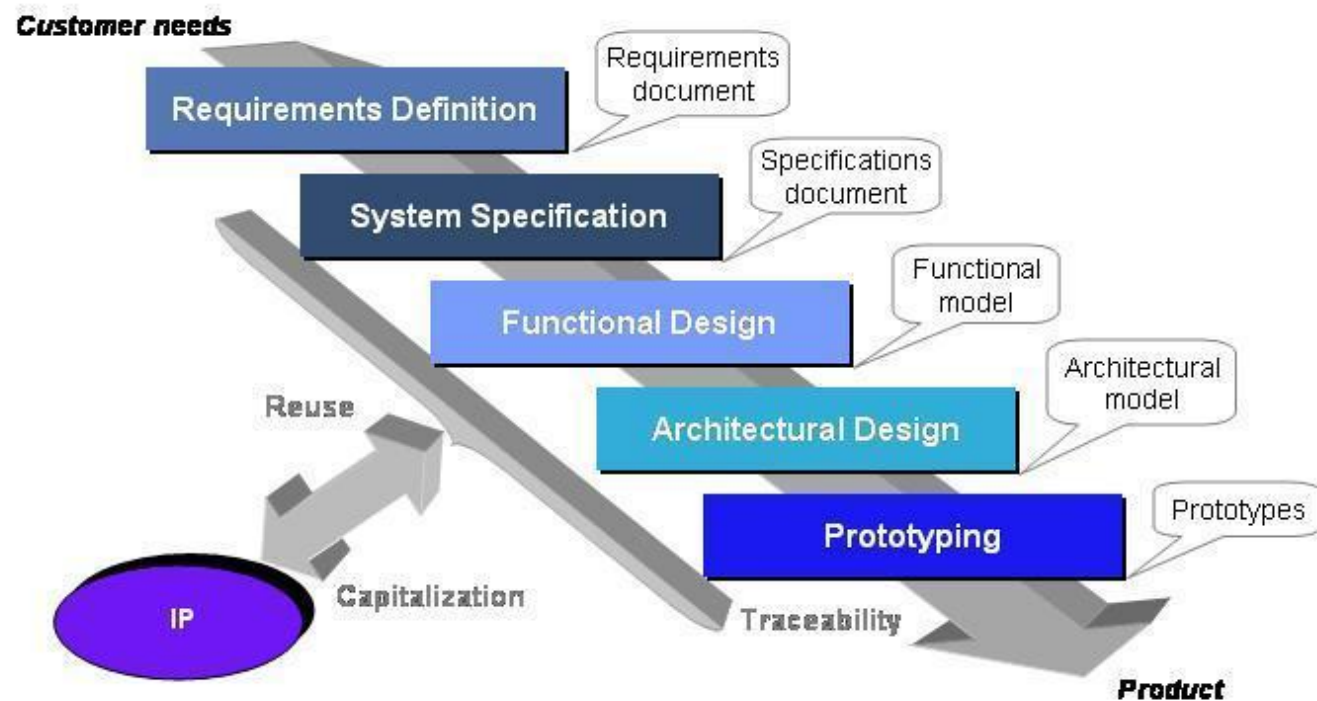
- Definition of system-level design specification (C-based, HDLs)
- Design evaluation and exploration
- Hardware/Software codesign
- Co-verification : co-simulation and co-emulation
- Debugging and diagnosis
- Rapid prototyping



SoC Design Flow



5 Design Step for SoC

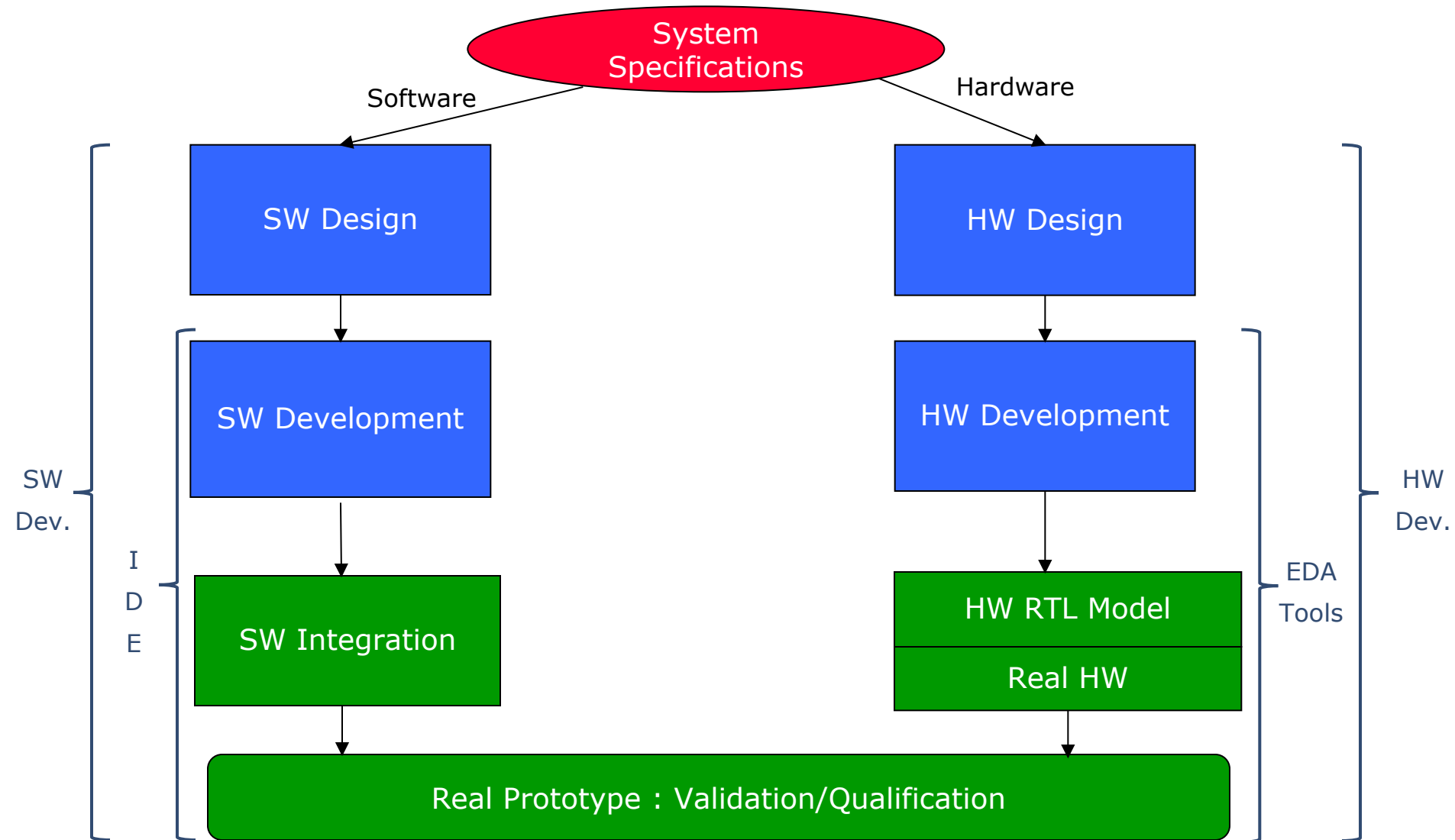


5 Design Step for SoC

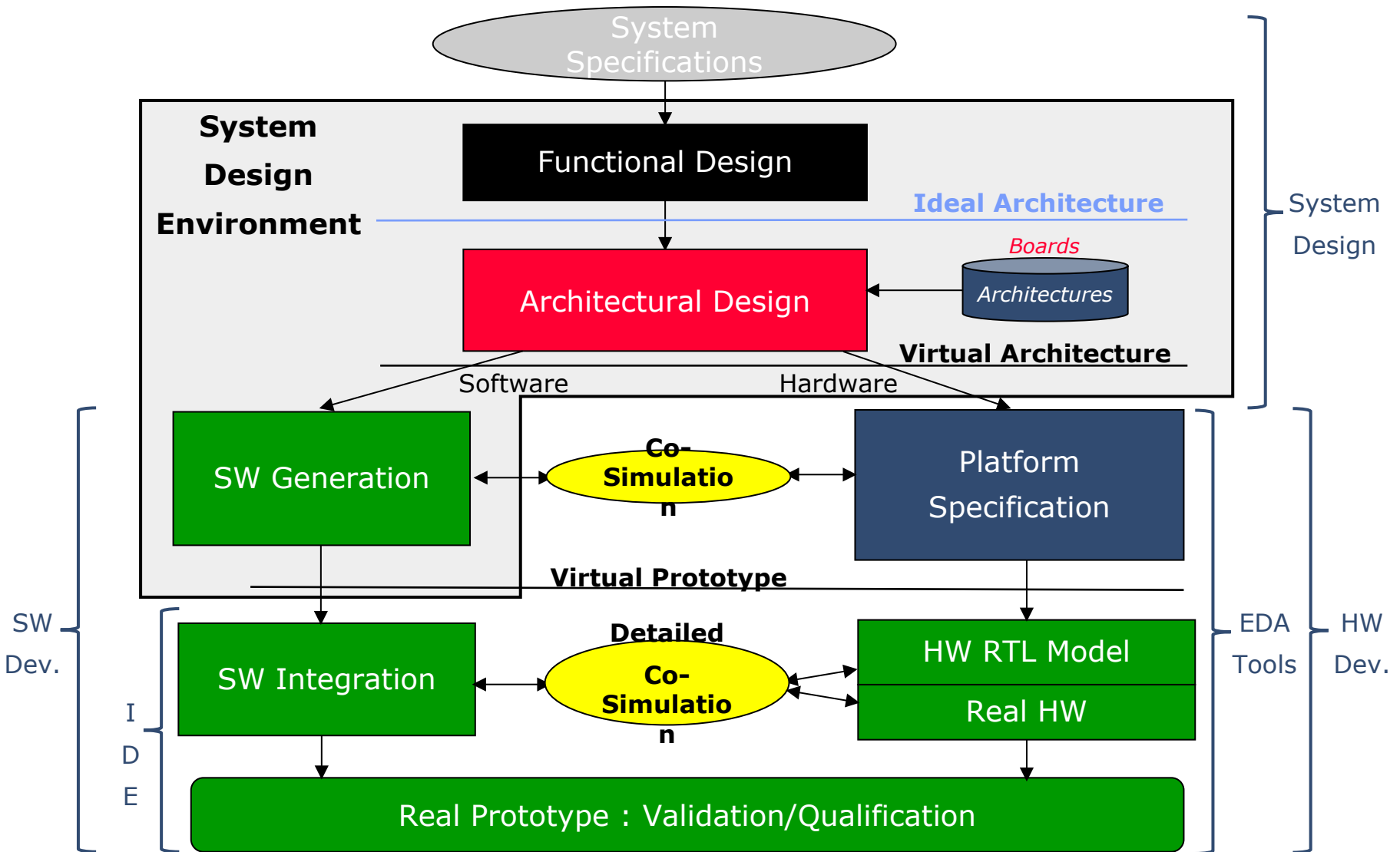
- **Requirements definition**
- **System specification**
- **Functional design**
- **Architectural design**
- **Prototyping**

Design steps	Purpose	Inputs	Outputs
1. Requirements definition	Understanding all interested parties' needs and documenting these needs as written definitions and descriptions.	Customer needs	Requirements document
2. System specification	Representing a purely external view of the system. Completely modeling the behavior of the system within its environment and listing all non-functional constraints.	Requirements document	Specifications document
3. Functional design	Finding a suitable internal logical (or functional) and technology-independent representation of the system from an application-oriented viewpoint.	Specifications document	Functional model
4. Architectural design	Defining the detailed system's physical (or hardware) architecture and the organization of the software application on each programmable processor.	Functional model Performances/cost constraints	Architectural model
5. Prototyping	Developing an operational system prototype in terms of hardware and software implementations.	Architectural model Technological & economical constraints	Prototype

Traditional Design Step for SoC

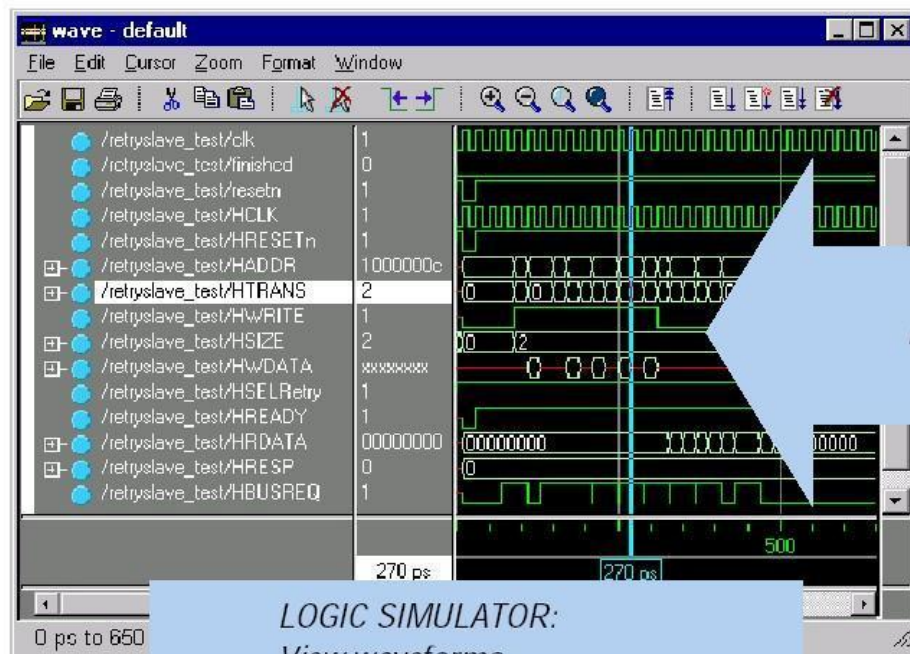


SoC Platform Architecture: Design Step for SoC

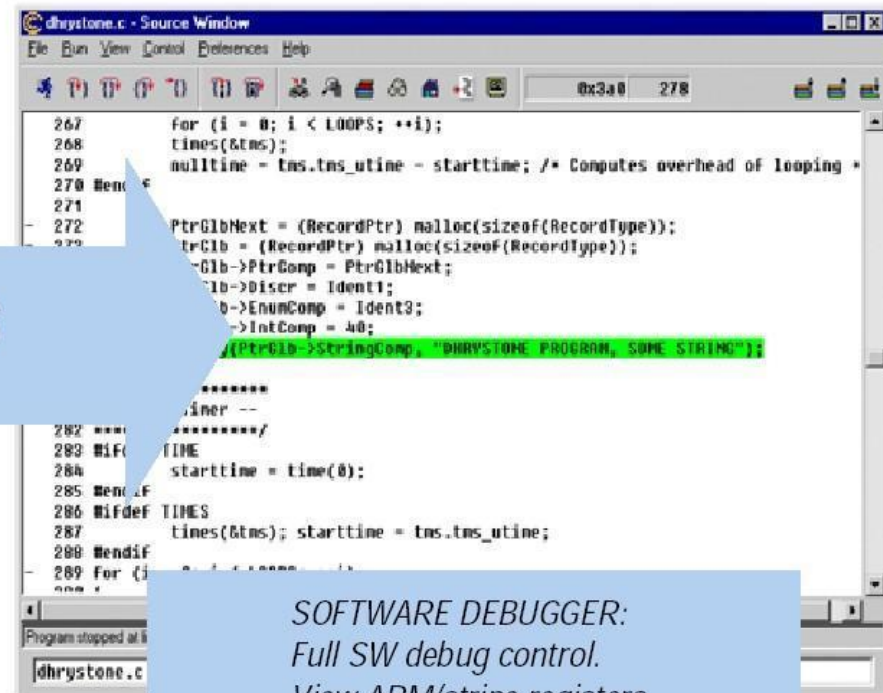


Co-Simulation

❑ Hardware/Software Co-Simulation



LOGIC SIMULATOR:
View waveforms
View signal values



SOFTWARE DEBUGGER:
Full SW debug control.
View ARM/stripe registers

ESS

S/W Requirements - Quartus

**Altera Subscriptions Include World-Class
OEM Synthesis & Simulation Tools**

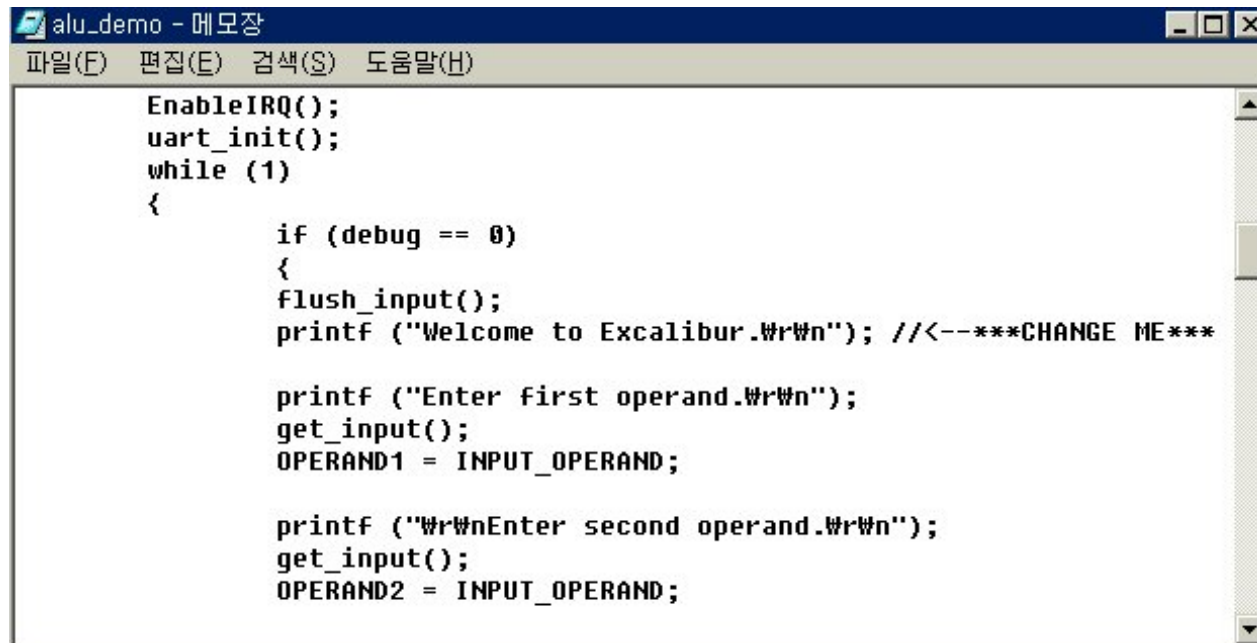
LEONARDO
spectrum **ModelSim**

- Exemplar Logic® LeonardoSpectrum™ -Altera Synthesis Tool
- Model Technology™ ModelSim® -Altera Simulation Tool

Download the Latest Versions at <http://www.altera.com>

Quartus II 2.0 Setup
Installing:

S/W Requirements – Verilog HDL, VHDL



```
EnableIRQ();
uart_init();
while (1)
{
    if (debug == 0)
    {
        flush_input();
        printf ("Welcome to Excalibur.\r\n"); //<--***CHANGE ME***

        printf ("Enter first operand.\r\n");
        get_input();
        OPERAND1 = INPUT_OPERAND;

        printf ("\r\nEnter second operand.\r\n");
        get_input();
        OPERAND2 = INPUT_OPERAND;
```

High Speed / Low Power Design

- Deep submicron effect
- High speed circuit design
- Low power / low voltage design
- Tradeoffs:
 - Cost vs. Functionality
 - Cost vs. Speed
 - Power vs. Speed

Platform-based Design

What is a platform?



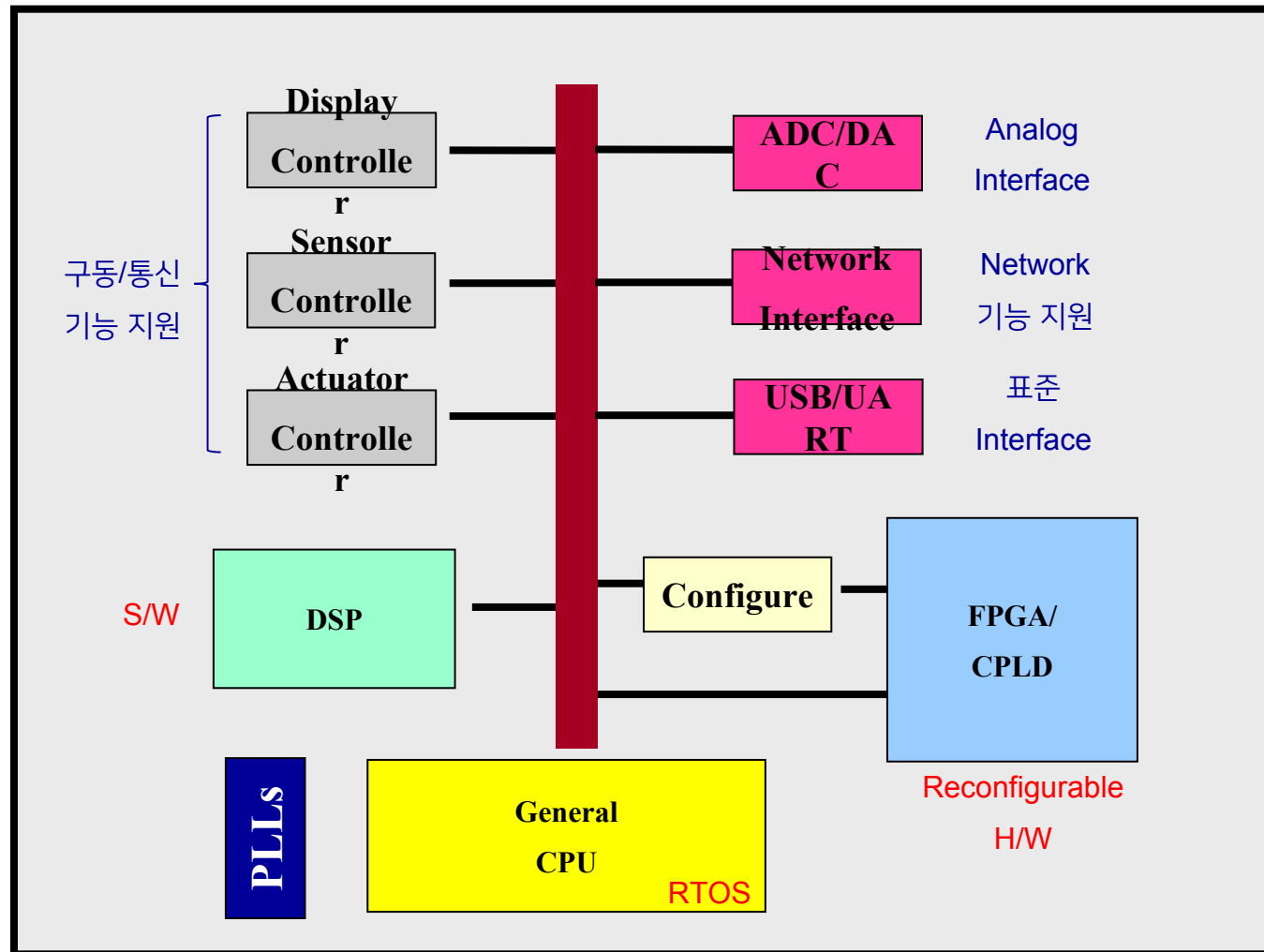
Why a platform?

- ☐ Constructed by system experts for a specific application domains
 - ☐ The system expertise can capture good solutions and provides abstracted programming interfaces for the future customization
- ☐ Customized by application experts for a specific product
 - ☐ The most application engineers do not have expertise on the system architecture

SoC Platform Prototype

- Products are needed
 - To increase performance, reduce costs and enhance features
 - The use of newer, faster and cheaper technologies
 - More functions and features to be placed on a single piece of silicon
 - Integrated in a system-on-chip with new functions added.
 - Peripheral Devices are needed
 - These include timers, DMA engines, interrupt controllers and memory controllers
 - In many cost-sensitive applications, a shared memory structure is utilized to reduce memory component costs
-

SoC Platform Architecture



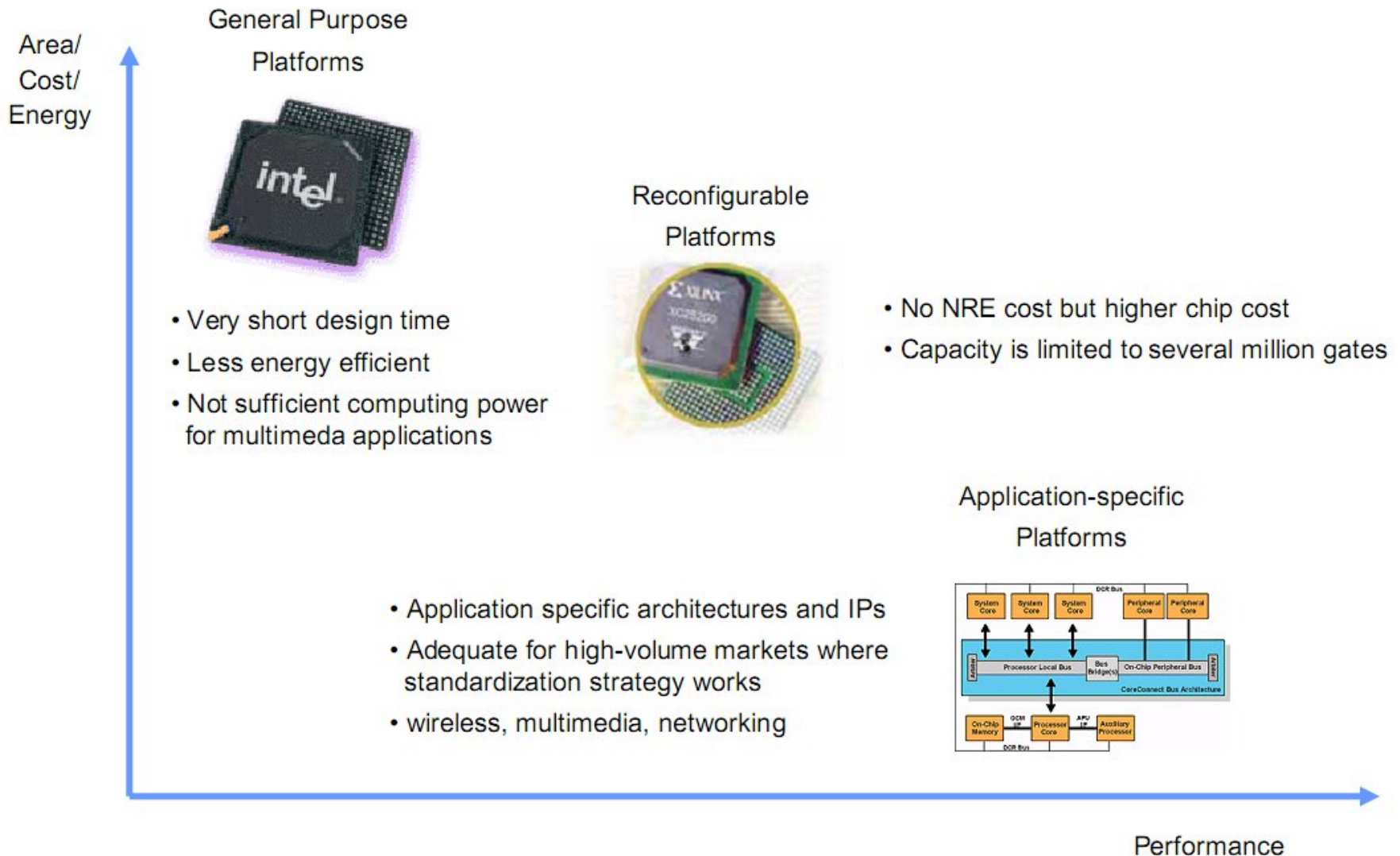
SoC Platform Architecture

- Embedded Processor
 - ARM : ARM922T, ARM926EJ-S, ARM1136
 - Altera : Excalibur (ARM922T, FPGA)
 - Motorola : PowerPC
 - Xilinx : Virtex II Pro (IBM405GP, FPGA)
 - MIPS : MIPS4K
- Re configurable FPGA
 - Xilinx Virtex II, Spartan
 - Altera Stratix, Cyclone, APEX, Excalibur
- Memory Block
 - SDRAM, DDRRAM
 - Flash Memory

SoC Platform Architecture

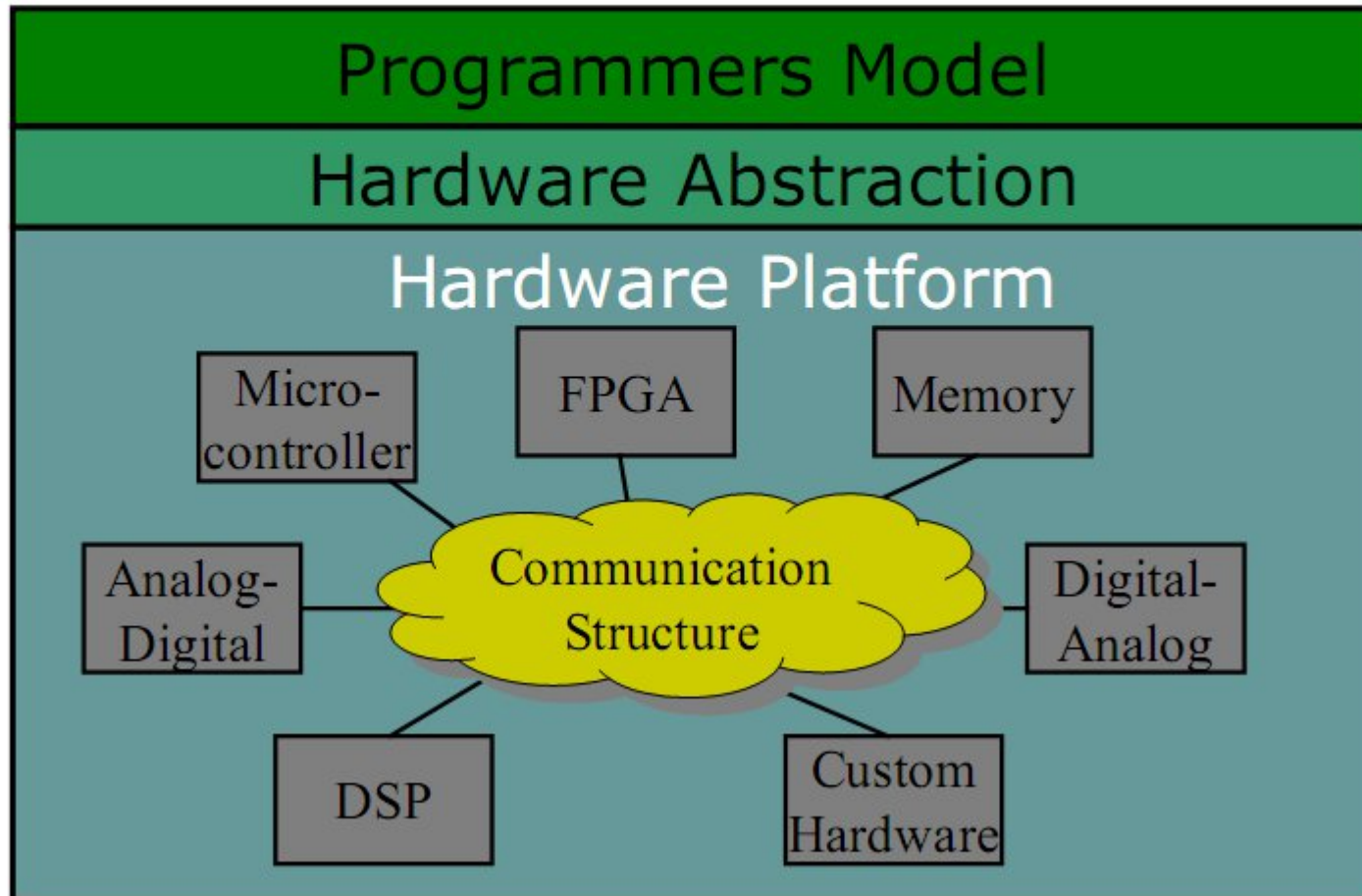
- All the peripheral functions required for a basic SOC
 - UART, Ethernet 10/100M, 1394, USB2.0
 - PCI, PC104, PCMCIA, CF+, Cardbus
 - DMA, Interrupt Request,
 - TFT LCD, Text LCD, FND
 - Key PAD, LED, PB Switch
- IP
 - MP3, MPEG4, H.264, MJPEG, JPEG, Memory Controller, PCI
 - USB Controller, UART Controller etc
- Bus Architecture
 - AMBA Spec. 2.0
- Operating System
 - Linux, Nucleus RTOS, Vxworks, Qplus

Platform Types



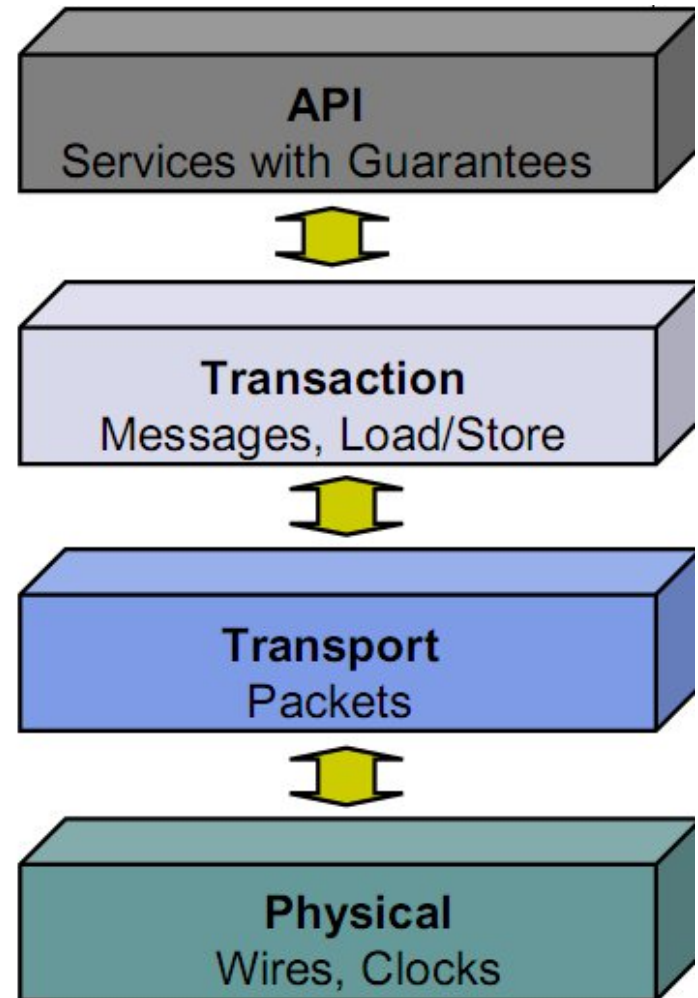
Platform-Based Design

The idea of a platform is to simplify the design process



System-on-Chip Platform

- Layered Concept allows to
 - Change the physical architecture of the SoC without affecting the application
 - Add new services on top of existing architecture
- Changes in one layer affect only the layer itself and its interfaces

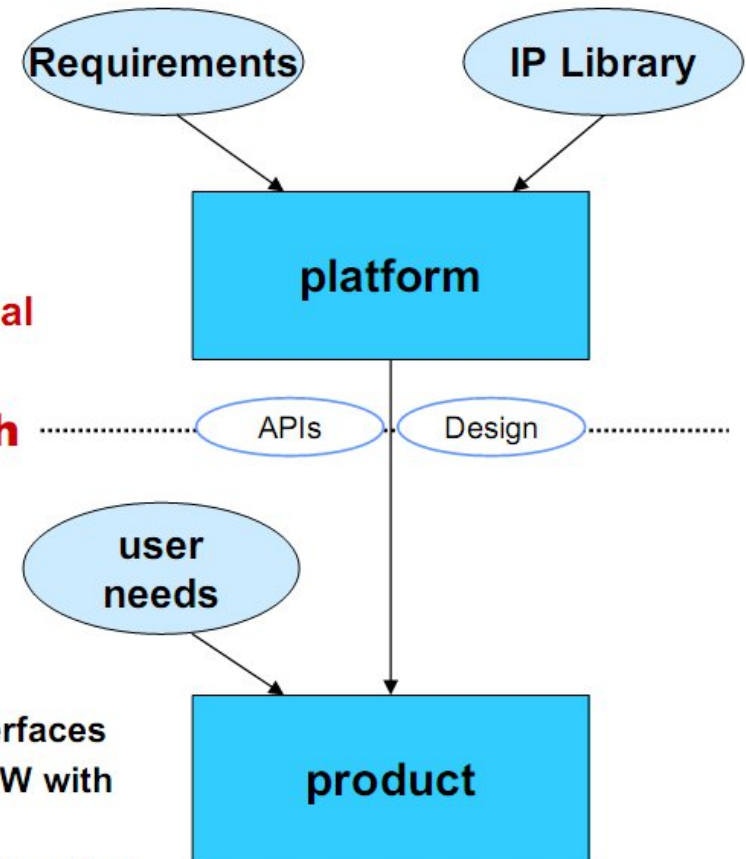


Two Phases of Platform-based Design

- Platform Development**
(by system experts)
- ☐ Bottom-up design
 - ☐ integrating IPs
 - ☐ Define APIs
 - ☐ H/W interfaces
 - ☐ S/W programming interfaces
 - ☐ Provides various abstraction level models of the platform
 - ☐ More difficult than conventional system design

Meet-in-the middle approach

- Derivative Design**
(by application experts)
- ☐ Top down design
 - ☐ Start design at the highest abstraction levels
 - ☐ Customize functionalities
 - ☐ Integrate HW IPs with HW Interfaces
 - ☐ Modify/describe application SW with SW APIs
 - ☐ Progressively change the abstraction levels into full RT-level



SoC Architecture Components

- SoC architecture is made up by the following:
 - (1) embedded processor
 - (2) processing engine (PE)
 - (3) on-chip memory
 - (4) off-chip memory
 - (5) i/o device controllers (peripherals)
 - (6) interconnection structure

Smart Phone SoC Example

Embedded Processor

- ❑ ARM 926EJ
(16KB I-Cache, 16-KB D-Cache)
- ❑ 16-bit teak DSP
(32kW program memory,
16KW X-data memory
16KW Y-data memory)

Processing Engines

- ❑ MPEG4 accelerator
- ❑ 2D/3D accelerator (MBX)

I/O Devices

- ❑ UART
- ❑ Timer
- ❑ LCD Controller
- ❑ USB Controller

Off-Chip Memory

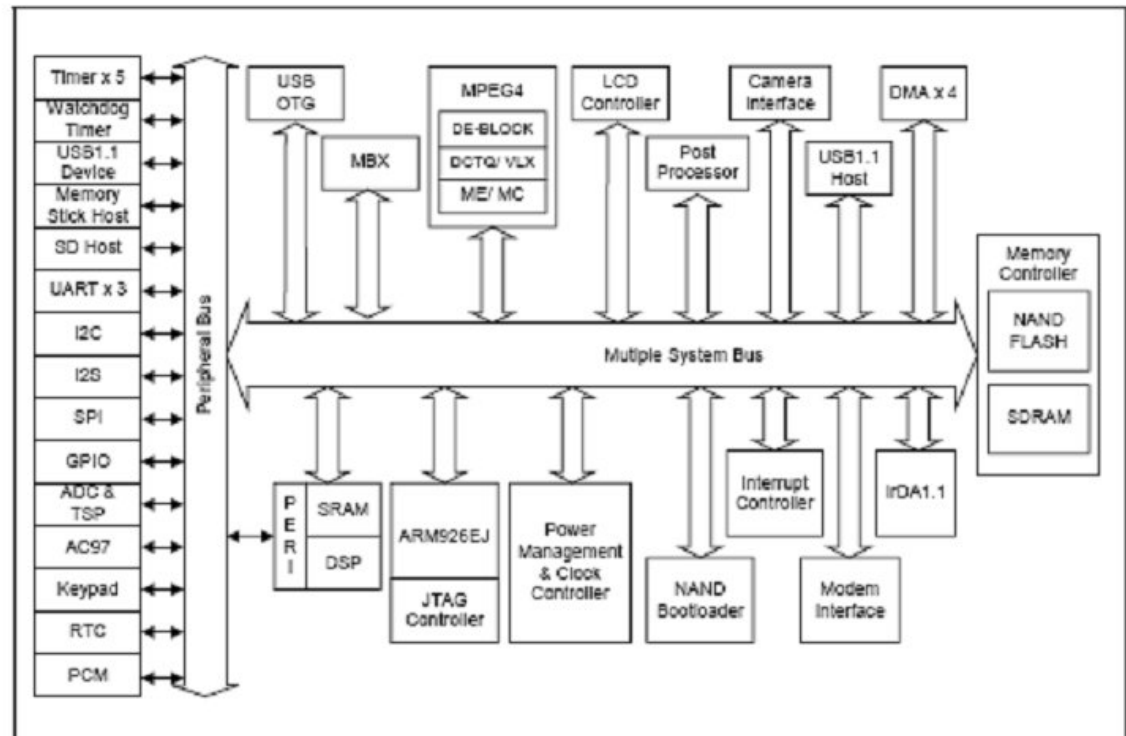
- ❑ NAND FLASH controller
- ❑ SDRAM controller

On-Chip Bus

- ❑ AHB
- ❑ APB

On-Chip Memory

- ❑ On-Chip SRAM
- ❑ On-Chip Flash



Smart Phone SoC: Embedded Processors

Embedded Processor

- ❑ ARM 926EJ
(16KB I-Cache, 16-KB D-Cache)
- ❑ 16-bit teak DSP
(32kW program memory,
16KW X-data memory
16KW Y-data memory)

Processing Engines

- ❑ MPEG4 accelerator
- ❑ 2D/3D accelerator (MBX)

I/O Devices

- ❑ UART
- ❑ Timer
- ❑ LCD Controller
- ❑ USB Controller

Off-Chip Memory

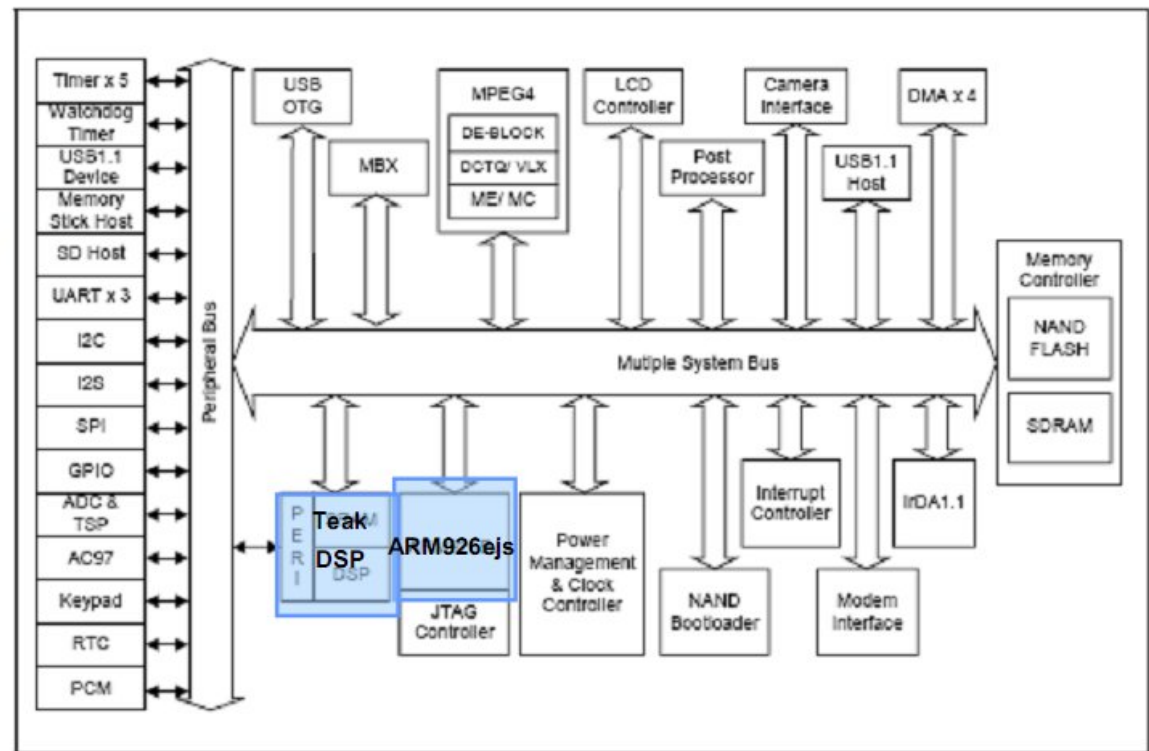
- ❑ NAND FLASH controller
- ❑ SDRAM controller

On-Chip Bus

- ❑ AHB
- ❑ APB

On-Chip Memory

- ❑ On-Chip SRAM
- ❑ On-Chip Flash



Smart Phone SoC: On-chip Bus

Embedded Processor

- ❑ ARM 926EJ
(16KB I-Cache, 16-KB D-Cache)
- ❑ 16-bit teak DSP
(32KW program memory,
16KW X-data memory
16KW Y-data memory)

Processing Engines

- ❑ MPEG4 accelerator
- ❑ 2D/3D accelerator (MBX)

Peripherals

- ❑ UART
- ❑ Timer
- ❑ LCD Controller
- ❑ USB Controller

Off-Chip Memory

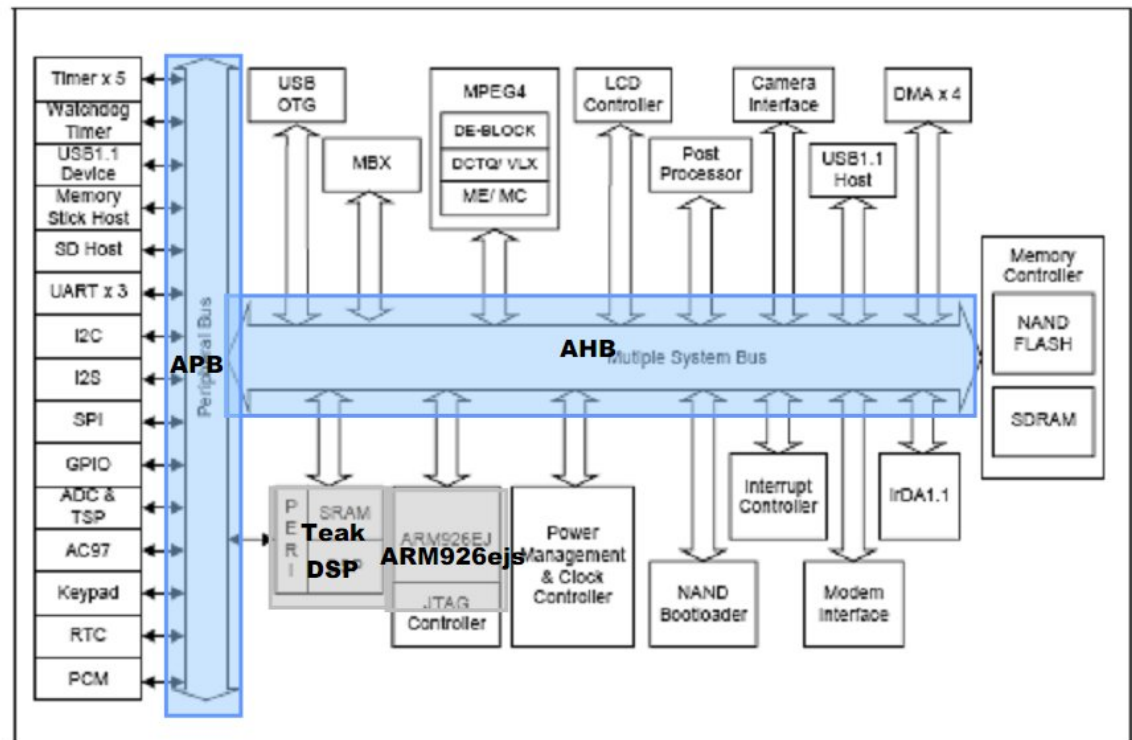
- ❑ NAND FLASH controller
- ❑ SDRAM controller

On-Chip Bus

- ❑ AHB
- ❑ APB

On-Chip Memory

- ❑ On-Chip SRAM
- ❑ On-Chip Flash



Smart Phone SoC: Peripherals

Embedded Processor

- ☐ ARM 926EJ
(16KB I-Cache, 16-KB D-Cache)
- ☐ 16-bit teak DSP
(32kW program memory,
16KW X-data memory
16KW Y-data memory)

Processing Engines

- ☐ MPEG4 accelerator
- ☐ 2D/3D accelerator (MBX)

Peripherals

- ☐ UART
- ☐ Timer
- ☐ LCD Controller
- ☐ USB Controller

Off-Chip Memory

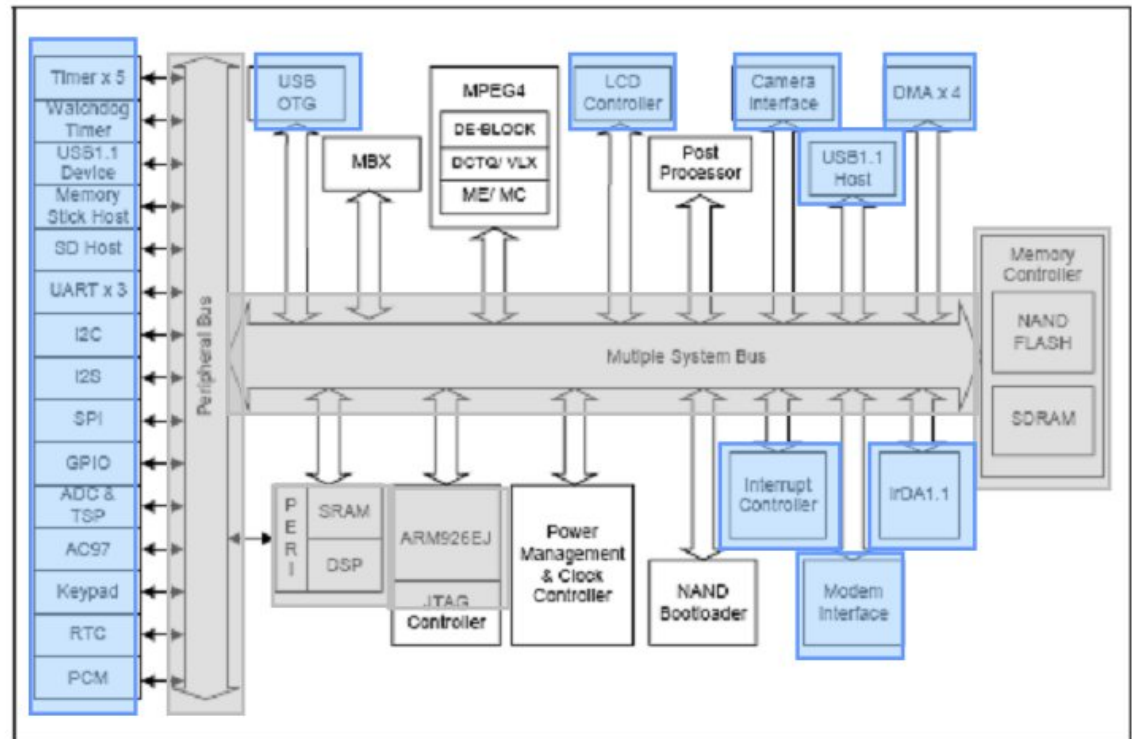
- ☐ NAND FLASH controller
- ☐ SDRAM controller

On-Chip Bus

- ☐ AHB
- ☐ APB

On-Chip Memory

- ☐ On-Chip SRAM
- ☐ On-Chip Flash



What do FPGAs Mean for the Future of Embedded Systems?

Implementation Options

- **Microprocessor**

The default choice

- **ASIC**

Got a couple of \$million?

- **FPGA**

Huh?





Which Way to Go?

ASICs

High performance

Low power

Low cost in
high volumes

FPGAs

Off-the-shelf

Low development cost

Short time to market

Reconfigurability

Microprocessors - PROs

- Reasonably cheap per unit costs
- Multiple configuration options
- Multiple vendors
- Integration of peripherals
- Memory capacity
- Secure
- Plenty of people can program them – low labour costs ?
- Power consumption
- Can be reprogrammed in the field

Microprocessors - CONs

- Architecture can not be altered
- Change of processor requires PCB revision
- Can't always get the peripheral mix you want – push some functionality into software or onto additional ICs.
- Development tools often limited to processors from the same vendor / family
- Sequential execution only

ASIC - PROs

- Superior power, speed, integration capability
- Absolute security – possible market monopoly
- Absolute control over device characteristics
- Lowest per unit costs
- High Entry costs provide a form of protection for vendors already in the market.

ASIC - CONS

- Spiralling setup costs
- Spiralling tool costs
- Limited number of people capable of designing such devices – high labour costs?
- Codevelopment of hardware and software is challenging
- Complex manufacturing processes can't always meet time to market demands
- Decreasing product lifetimes makes it harder to recoup initial costs

FPGA – PROs

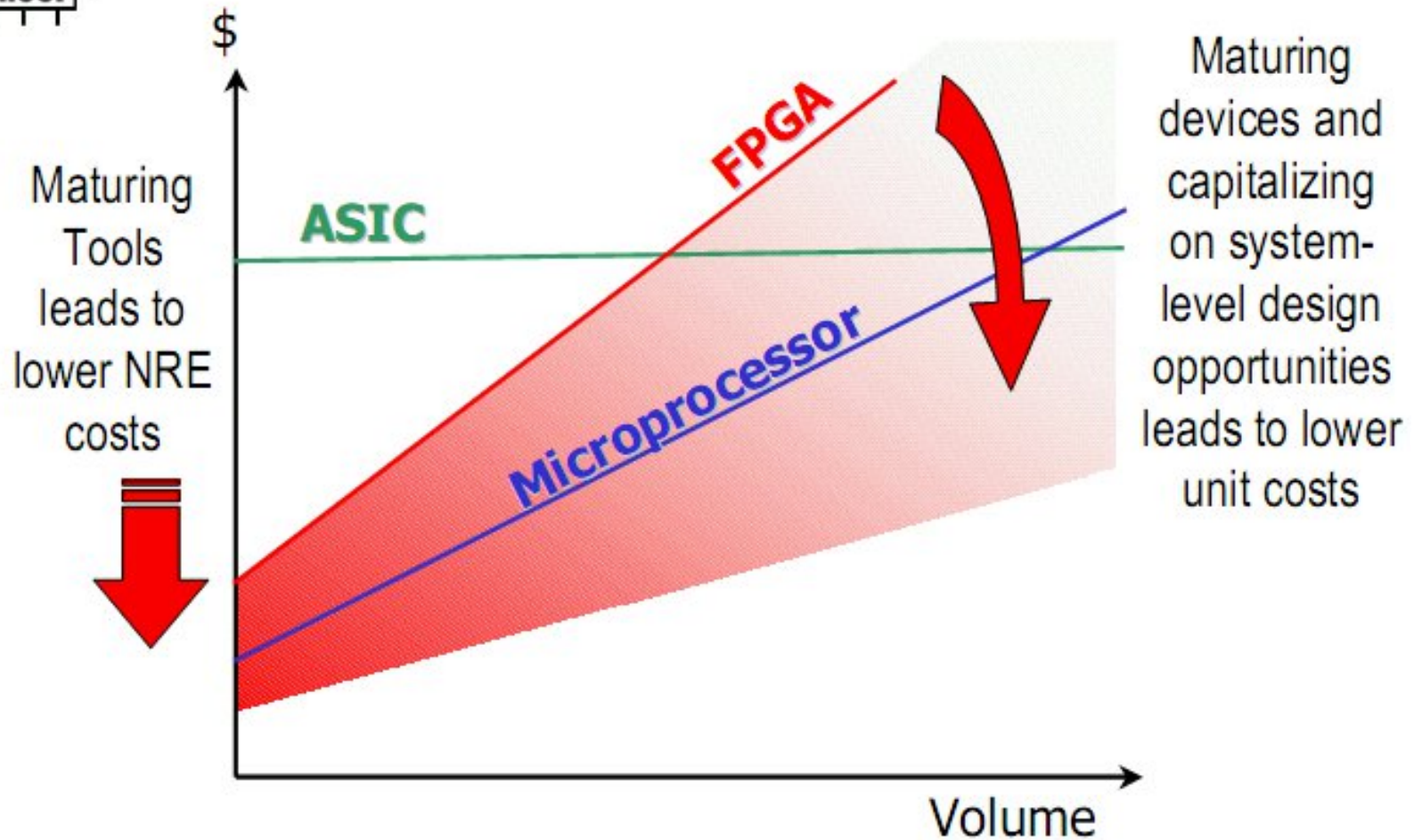
- Infinitely (Re)configurable – potential for run-time reconfiguration
- True parallelism
- High speed
- Architecture can be optimised for a given application
- Architecture can be changed without PCB revisions
- In field reconfigurable
- Decreasing per unit costs
- ASIC development path options
- Number and type of peripherals can be customized
- Decreased system costs through tighter integration

FPGA – CONS

- Limited on chip memory
 - Limited analogue capability
 - Power consumption
 - More complex PCBs
 - Configuration memory costs
 - More complex power supply requirements
 - Packaging options
 - Difficult to program (market perception?)
- Manufacturing cycle for ASIC is very costly, lengthy and engages lots of manpower
 - It takes ~5M dollars to make just one chip
 - It takes ~5M dollars to make 1,000 chips
 - Mistakes not detected at design time have large impact on development time and cost
 - FPGAs are perfect for rapid prototyping of digital circuits
 - Easy upgrades like in case of software
 - Unique applications
 - reconfigurable computing



Implementation Cost Comparison



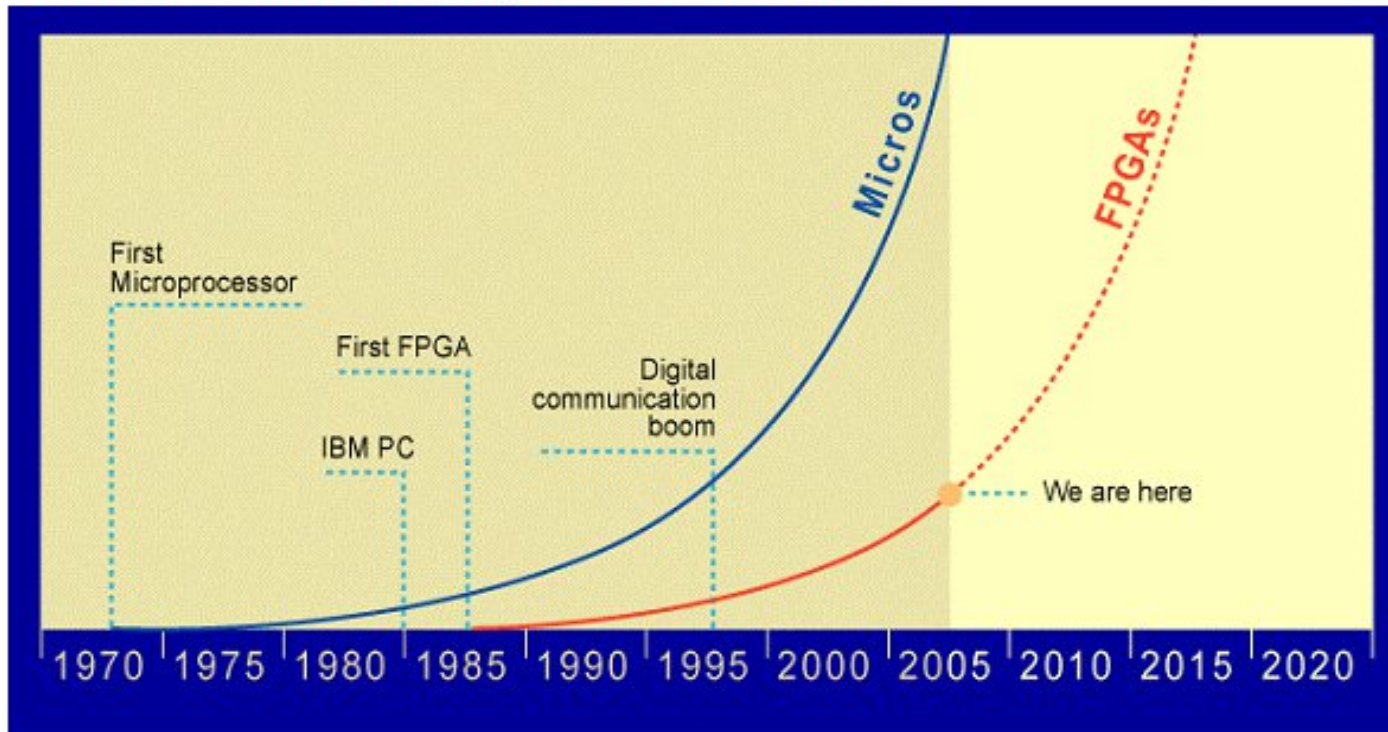
Gazing at the Crystal Ball

In order to determine the future, we need to look at the past:

- The introduction of the microprocessor represented more than a technological breakthrough – it heralded the last major paradigm shift in the electronics industry.
- Migration trend from *hardware* to *software*.
- Microprocessors have reached a commodity level where they are now regularly used in toys.
- It is hard to imagine a world without microprocessors.

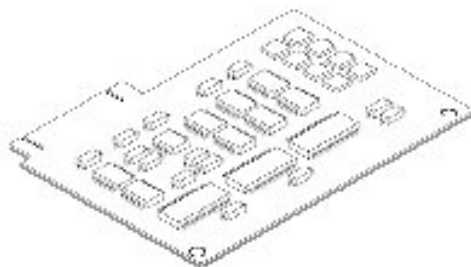
I Predict An FPGA In Your Future!

FPGAs capable of supporting 32-bit 'soft' processor cores are now achieving commodity pricing levels. Both Altera and Xilinx have a range of devices targeted for the embedded systems market.



FPGAs – the Future is Bright

- FPGAs represent the next logical extension in the hard to soft migration of system functionality.
- They allow for the extension of soft design methods to encompass the entire system. Processors, peripheral devices, additional logic, and software can all be easily changed.



Hard-wired Design

Design function is fixed and cannot be changed after manufacture.



Processor Based Design

Hard-wired components are fixed, but software can be changed after manufacture.



'Soft' Design

Both software and soft-wired components can be changed after manufacture – complete freedom in system implementation.

FPGA Paradigm Shift

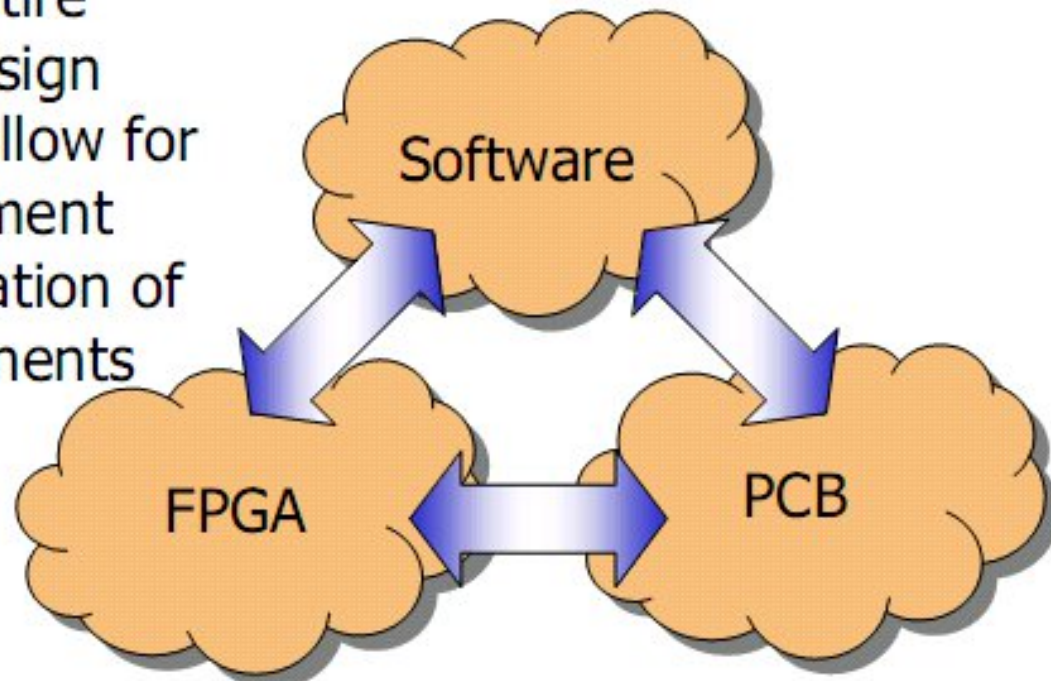
Designing with FPGAs requires a significant paradigm shift:

- System level approach is needed. What opportunities does a configurable platform bring to the application?
- Bigger processor is not always better – some tightly constrained blocks can be shifted with minimal effort to hardware. Consider PWM units, Event Capturing, Keypad scanning, etc.
- What additional features can be added 'for free'?

Catching the Wave

A key factor in the uptake of FPGAs for use in embedded systems is the general accessibility of affordable design tools.

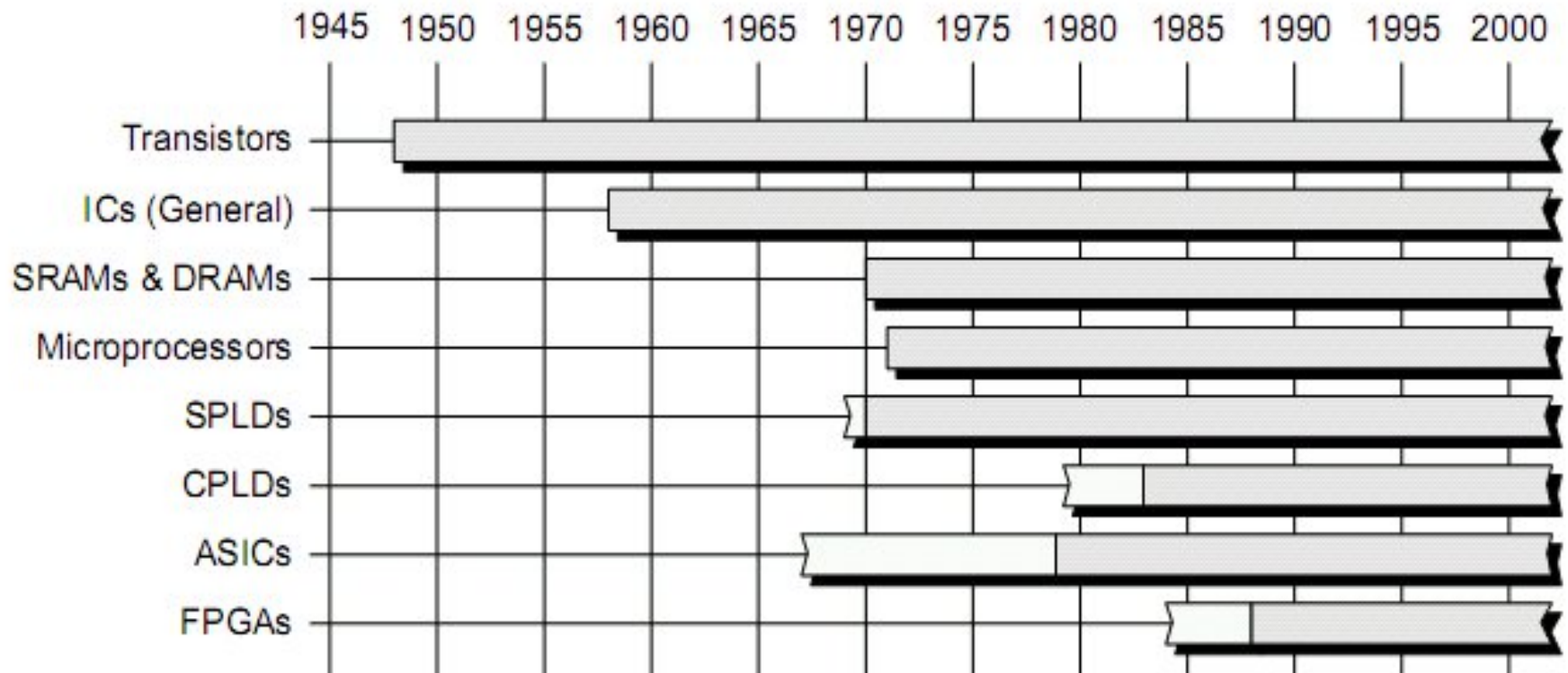
Tools are required that embrace the entire system level design paradigm and allow for the co-development and synchronization of all of its components



Glossary

- FPGA: Field Programmable Gate Array
- FPSC: Field Programmable System Chip (Hard core with FPGA resources)
- FPAA: Field Programmable Analogue Array (Anadigm)
- PAC: Programmable Analogue IC (Lattice Semiconductor)
- PLD: Programmable Logic Device
- CPLD: Complex Programmable Logic Device
- SPLD: Simple Programmable Logic Device
- EPLD: Erasable Programmable Logic Device
- PAL: Programmable Array Logic
- PLA: Programmable Logic Array
- PSoC: Programmable System on Chip
- JTAG (IEEE 1149.1): Joint Test Application Group
- LCA: Logic Cell Array (Xilinx FPGA).
- GAL: Generic Array Logic (TM Lattice)

Evolution of FPGA Hardware



Reference

- This document is used for educational purposes only
- This document contains material from other sources (incl. textbook, lecture notes, application notes, ...)
- Our thanks go to the contributors, references